


```

1 export async function exchange(exchangeRequest) {
2   ...
3   //get the exchange rate
4   const exchangeRate = rates[baseCurrency][counterCurrency];
5   //compute the requested (counter) amount
6   const counterAmount = baseAmount * exchangeRate;
7   ...
8   //all good, update balances
9   baseAccount.balance += baseAmount;
10  counterAccount.balance -= counterAmount;
11  ...
12 }

```

Código 1. Responsabilidades concentradas en la función `exchange` de `app/exchange.js`.

`state.js` no expone directamente sus variables internas, pero sus getters retornan las mismas referencias mutables almacenadas por el módulo. Luego de la inicialización, `exchange.js` conserva esas referencias y modifica los objetos sin pasar por una operación explícita del componente de persistencia. Por ello, la abstracción es permeable: reemplazar los JSON por un repositorio o una base de datos requeriría adaptar también la forma en que la lógica de negocio lee y actualiza el estado.

```

1 let accounts;
2 let rates;
3 let log;
4
5 export async function init() {
6   await stateInit();
7
8   accounts = stateAccounts();
9   rates = stateRates();
10  log = stateLog();
11 }

```

Código 2. Referencias mutables al estado obtenidas en `init` de `app/exchange.js`.

El endpoint `/exchange` también contiene una condición de carrera. La comprobación de saldo se ejecuta antes de dos operaciones asíncronas. Mientras una solicitud se encuentra suspendida en un `await`, otra puede comprobar el mismo saldo y aprobar la operación. Ambas solicitudes pueden completar los descuentos y dejar un saldo negativo. El siguiente código muestra la sección crítica, que no está protegida por exclusión mutua, serialización ni una transacción. Lo verificamos enviando diez cambios concurrentes de 80 USD contra un saldo interno de 100 USD: se aprobaron los diez y la cuenta terminó en -700 USD (Tabla 4).

```

1 // app/exchange.js (caso base)
2 if (counterAccount.balance >= counterAmount) {
3   if (await transfer(clientBaseAccountId, baseAccount.id, baseAmount)) {
4     if (
5       await transfer(counterAccount.id, clientCounterAccountId, counterAmount)
6     ) {
7       baseAccount.balance += baseAmount;
8       counterAccount.balance -= counterAmount;
9       exchangeResult.ok = true;
10      exchangeResult.counterAmount = counterAmount;
11    }
12  }
13 }

```

Código 3. Sección crítica sin exclusión mutua en `app/exchange.js`.

El registro de operaciones tampoco tiene una política de retención. Cada resultado se agrega a un arreglo en memoria y `/log` devuelve el arreglo completo. A medida que aumenta la cantidad de operaciones, crecen el consumo de memoria, el tamaño de la respuesta y el trabajo necesario para serializar el estado. Además, el arreglo completo se convierte a JSON y se reescribe cada 1s, por lo que el costo de cada persistencia aumenta con el historial.

```

1 // app/exchange.js (caso base)
2 log.push(exchangeResult);
3
4 // app/app.js (caso base)
5 app.get("/log", (req, res) => {
6   res.json(getLog());
7 });
8
9 // app/state.js (caso base)
10 scheduleSave(log, LOG, 1000);
11
12 function scheduleSave(data, fileName, period) {
13   setInterval(async () => {
14     await save(data, fileName);
15   }, period);
16 }

```

Código 4. Crecimiento, consulta y persistencia del log en app/exchange.js, app/app.js y app/state.js.

La escritura utiliza directamente el archivo definitivo. No se escribe primero a un archivo temporal ni se realiza un reemplazo atómico, por lo que una interrupción durante writeFile puede dejar un JSON incompleto. El manejo de errores de state.js registra el fallo, pero no lo propaga ni permite que la API informe que el estado no pudo persistirse.

Para verificar la pérdida de datos, ejecutamos un cambio, que la API confirmó con HTTP 200, y terminamos el proceso de forma abrupta antes del siguiente guardado. Antes de la terminación, /log contenía la operación y los saldos internos reflejaban el cambio (ARS 120000000 → 120100000, USD 60000 → 59934). Al reiniciar, el log volvió a estar vacío y los saldos a sus valores iniciales: la operación se perdió aunque las transferencias con el cliente ya se habían realizado.

```

1 async function save(data, fileName) {
2   const filePath = path.join(__dirname, fileName);
3   try {
4     await fs.promises.writeFile(filePath, JSON.stringify(data, null, 2));
5   } catch (err) {
6     console.error(`Error writing to ${filePath}:`, err);
7   }
8 }

```

Código 5. Escritura directa y no atómica del estado en app/state.js.

No hay un modelo de dominio, los importes monetarios se representan mediante Number, cuyo formato de punto flotante binario no representa exactamente todos los valores decimales. Sin una regla explícita de redondeo o una representación en unidades mínimas, el error puede acumularse en los saldos y en los importes entregados al cliente. Por ejemplo, fijando la tasa ARS/USD en 0.1 mediante PUT /rates y cambiando 3 ARS, la API responde counterAmount": 0.30000000000000004 en lugar de 0.3.

```

1 const exchangeRate = rates[baseCurrency][counterCurrency];
2 //compute the requested (counter) amount
3 const counterAmount = baseAmount * exchangeRate;

```

Código 6. Cálculo monetario con Number en app/exchange.js.

Finalmente, las validaciones HTTP comprueban únicamente la presencia mediante condiciones de truthiness. No se validan tipos, rangos, monedas soportadas ni coherencia entre moneda y cuenta. Por ejemplo, un importe negativo supera la validación inicial y puede alterar los saldos en el sentido contrario al esperado. Las monedas inexistentes también pueden producir excepciones al acceder a una tasa o cuenta no encontrada. A su vez, los rechazos de negocio se informan como HTTP 500, aunque no necesariamente representan una falla interna del servidor. La Tabla 3 muestra el resultado de ejecutar estas requests: además del importe negativo, un importe enviado como texto convierte el saldo de la cuenta interna en un string por concatenación.

```

1 // app/app.js (caso base)
2 if (
3   !baseCurrency ||
4   !counterCurrency ||
5   !baseAccountId ||
6   !counterAccountId ||
7   !baseAmount
8 ) {
9   return res.status(400).json({ error: "Malformed request" });
10 }

```

Código 7. Validación por presencia en app/app.js.

Cuadro 1. Hallazgos del análisis estático del caso base.

Hallazgo	Evidencia	QA afectados	Consecuencia posible	Evidencia de ejecución
Carrera concurrente	La verificación de saldo precede a dos operaciones asíncronas y el descuento ocurre después (Código 3).	Reliability, Security	Dos solicitudes pueden aprobarse con el mismo saldo y producir un balance negativo.	Diez cambios concurrentes de 80 USD contra un saldo de 100 USD: se aprueban todos y el saldo queda en -700 (Tabla 4).
Estado mutable expuesto	Los getters de <code>state.js</code> retornan objetos que <code>exchange.js</code> conserva y modifica.	Modificabilidad, Testability, Reliability	Las actualizaciones eluden una interfaz de repositorio y dificultan sustituir o simular la persistencia.	No aplica: se evidencia en el código (Código 2).
Log sin límite	Cada operación ejecuta <code>log.push</code> ; <code>/log</code> retorna el arreglo completo (Código 4).	Performance, Scalability, Availability	Aumentan memoria, tamaño de respuesta y tiempo de serialización con cada operación.	La latencia de <code>/log</code> crece durante toda la prueba de resistencia (Figura 17).
Reescritura completa	<code>scheduleSave</code> serializa y escribe el log completo cada 1s.	Performance, Efficiency, Reliability	El costo de I/O crece con el historial y una interrupción puede dejar el archivo incompleto.	El uso de CPU y memoria crece con carga constante (Figura 18).
Durabilidad diferida	Los cambios residen en memoria hasta el siguiente intervalo y la escritura no es atómica.	Reliability, Availability	Una terminación abrupta puede perder operaciones confirmadas o impedir cargar el JSON.	Un cambio confirmado con HTTP 200 y seguido de una terminación abrupta desaparece del log y de los saldos al reiniciar (Sección 1.2).
Dinero con Number	El importe se obtiene mediante <code>baseAmount * exchangeRate</code> sin redondeo explícito (Código 6).	Reliability, Security	Pueden aparecer residuos decimales y acumularse diferencias en los saldos.	Con tasa 0.1, cambiar 3 ARS entrega 0.30000000000000004 USD (Sección 1.2).
Validación incompleta	La API solo verifica truthiness y acepta valores negativos o combinaciones incoherentes (Código 7).	Security, Reliability, Testability	Entradas inválidas pueden alterar saldos o provocar excepciones no controladas.	Importes negativos o como texto alteran los saldos y una moneda inexistente termina el proceso (Tabla 3).

1.3. Infraestructura

La infraestructura se define mediante Docker Compose. Nginx publica la API en el puerto 5555; Grafana, Graphite/StatsD y cAdvisor conforman el subsistema de observabilidad. La API no publica directamente su puerto al host, por lo que las pruebas externas deben atravesar Nginx, al igual que el tráfico de los clientes.

El estado de la API se copia dentro de la imagen y no está asociado a un volumen. Por lo tanto, eliminar o recrear el contenedor descarta las modificaciones de sus archivos JSON. Grafana sí utiliza un volumen, mientras que Graphite no declara uno para conservar sus series. Estas decisiones limitan la durabilidad y la reproducibilidad de las mediciones entre recreaciones del entorno.

TODO: Agregar evidencia de ejecución sobre recuperación, persistencia y funcionamiento de cAdvisor en el equipo de pruebas.

2. ATRIBUTOS DE CALIDAD

2.1. Identificación de los QA claves

Se identifican los siguientes atributos de calidad como claves en el contexto del negocio. Adicionalmente, se citan especificaciones y requerimientos que motivaron estas afirmaciones.

2.1.1. Availability (disponibilidad)

Habilidad del sistema para reparar o enmascarar fallas de manera tal que el período fuera de servicio no exceda un determinado valor durante un lapso de tiempo especificado.

> La startup arVault es una fintech que opera una billetera digital, de reciente creación.

Al ser una billetera digital, el servicio permite manipular el dinero de los usuarios. Un servicio con baja o nula disponibilidad implicaría que los usuarios no pueden acceder a su propio dinero. Adicionalmente, el modelo de negocio se basa en que los usuarios operen, si el servicio no está disponible para permitir estas operaciones, se están perdiendo potenciales ganancias.

2.1.2. Performance

Habilidad del sistema para reaccionar ante ciertos eventos en un determinado tiempo.

> Si bien el lanzamiento fue exitoso, comenzaron a aparecer reclamos de los usuarios sobre problemas en el uso de la función de cambio de monedas.

Uno de los principales candidatos a explicar los reclamos es el endpoint /exchange. Una operación exitosa realiza dos transferencias secuenciales de entre 200 y 400ms cada una, por lo que su tiempo de respuesta esperado es mayor que el de los endpoints que solo consultan memoria. Esta hipótesis debe contrastarse con las pruebas de carga (Figura 5).

> las tasas de cambio no tienen gap entre el valor de compra y de venta.

La ausencia de gap significa que la compra y la venta utilizan la misma cotización; no implica por sí misma que la tasa se actualice en tiempo real. Sin embargo, una latencia elevada aumenta el tiempo entre la cotización y la finalización del cambio, y por ello incrementa el riesgo de operar con un valor que ya no represente las condiciones vigentes.

2.1.3. Scalability (escalabilidad)

Habilidad del sistema para incorporar recursos adicionales de manera efectiva. Efectividad = mejora mensurable de algún QA + no requerir un esfuerzo inapropiado + no interrumpir el funcionamiento.

> Para ganar usuarios, las tasas de cambio...

Se especifica que el objetivo a corto plazo es incrementar el número de usuarios. El sistema debe poder acompañar ese crecimiento incorporando recursos sin interrumpir el servicio ni exigir un esfuerzo inapropiado.

2.1.4. Reliability (confiabilidad)

Grado en que la arquitectura es susceptible a fallas a nivel de sistema cuando se producen fallas parciales en componentes, conectores o datos.

La confiabilidad es clave porque el servicio modifica saldos que representan dinero. Una respuesta exitosa con un saldo incorrecto constituye una falla aunque el proceso permanezca disponible. La condición de carrera y la persistencia diferida identificadas en la Sección 1.2 pueden producir pérdida de operaciones confirmadas, balances negativos o un estado distinto después de reiniciar el servicio.

2.1.5. Security (seguridad)

Capacidad del sistema de prevenir acciones maliciosas o accidentales fuera del uso esperado.

> No se valida que haya saldo en la cuenta del cliente, se espera que lo haga la UI y que no permita la operación.

Confiar en validaciones realizadas exclusivamente por el cliente permite que un consumidor invoque directamente la API con valores inesperados (Código 7). Aunque arVault cuenta con un componente previo de autenticación y autorización, esos mecanismos no sustituyen la validación de tipos, rangos e invariantes de negocio. Una operación inválida puede provocar pérdidas monetarias y reputacionales.

2.2. Otros QA menos relevantes

Además se mencionan otros QA que, aunque no resulten claves y por ende no sean una prioridad, son deseables, especialmente conforme el proyecto crezca.

2.2.1. Modifiability (modificabilidad)

Facilidad con la que el sistema puede adaptarse a cambios en sus requerimientos, tecnologías o entorno.

> No me gusta guardar todo en archivos .json, por ahora va, pero tendría que hacer algo distinto.

> Ver el tema del manejo de las cuentas, debería ser responsabilidad de otro servicio.

El propio README del servicio anticipa cambios concretos: reemplazar el almacenamiento en archivos (el fundador incluso propone migrar a Redis) y mover la gestión de cuentas a otro servicio. Además, arVault destinó los primeros fondos a agregar funcionalidad. La modificabilidad no resuelve los reclamos actuales de los usuarios, por lo que no la consideramos clave, pero determina el costo de aplicar las mejoras que sí lo son.

2.2.2. Efficiency (eficiencia)

Performance obtenida por recurso consumido.

> En docker-compose.yml tienen la solución tal como corre en el único servidor disponible al momento de comenzar el proyecto.

Al contar con un único servidor, los recursos que consume una funcionalidad quedan disponibles para el resto. Sin embargo, arVault acaba de recibir inversión y el problema que amenaza al negocio son los reclamos por el funcionamiento del cambio de monedas, no el costo de infraestructura. Por eso priorizamos la performance y la disponibilidad por sobre la eficiencia, aceptando consumir más recursos si eso las mejora (como hacen las propuestas de este trabajo).

2.2.3. Testability (testeabilidad)

Grado en que un componente o sistema de software permite ser probado para exhibir sus fallas; usualmente relacionado al esfuerzo necesario para llevar adelante las tareas de testing.

> los potenciales inversores se niegan a aportar más fondos si no se realiza una auditoría y se mejora el servicio.

Los usuarios no perciben directamente la testeabilidad, pero la auditoría exigida por los inversores requiere demostrar que el servicio se comporta correctamente. El repositorio no incluye pruebas automatizadas: el script `test` del package .json es el generado por defecto (`Error: no test specified`). Sin pruebas, cada corrección de los problemas de confiabilidad identificados corre el riesgo de introducir regresiones.

2.2.4. Manageability (maneabilidad)

Define la facilidad de gestión y monitoreo del sistema por parte de sus administradores.

> Actualiza el saldo de una cuenta. No me gusta cómo está hecho esto, otro servicio debería ocuparse de esto (el que hace las transferencias?)

Las cuentas internas y las tasas se administran a través de endpoints HTTP expuestos junto con los de los clientes, y el servicio original no emite métricas propias: para este trabajo tuvimos que instrumentarlo. Dado que se trata de un servicio con una única instancia y pocos operadores, no la consideramos clave, pero su falta dificulta diagnosticar problemas como los que motivaron los reclamos.

2.2.5. Simplicity (simplicidad)

Facilidad con la que la arquitectura de un sistema puede ser entendida, razonada y explicada.

> Su fundador, un desarrollador aficionado y entusiasta [...] se concentró en implementar rápidamente (bajo la consigna *fake it until you make it*) un núcleo de servicios que le permitieran conseguir más fondos a través de inversores.

El sistema fue desarrollado por una sola persona sin experiencia profesional y priorizando la velocidad. Una arquitectura simple facilita que el equipo (que hoy incluye a arquitectos externos) la entienda y la modifique. No es un QA clave, pero toda táctica que se proponga debe justificar la complejidad que agrega.

2.3. Decisiones de diseño actuales y su impacto en los QA

A continuación recorreremos los QA, claves y no claves, analizando cómo los afectan las decisiones de diseño del sistema recibido. La evidencia proviene del análisis del código (Sección 1.2), de la infraestructura (Sección 1.3) y de las pruebas sobre el sistema base (Sección 3).

2.3.1. Impacto en la Disponibilidad

- **Instancia única sin política de reinicio ni health check.** El `docker-compose.yml` declara una sola instancia de la API sin `restart` ni `healthcheck`. Cualquier falla que termine el proceso deja el servicio fuera de línea de forma indefinida. En el escenario de caos, una única request inválida tiró la API en el minuto 3:15 y no volvió a completarse ninguna request durante el resto de la prueba (Figura 21).
- **Errores no controlados en handlers asíncronos.** El handler de `/exchange` es una función `async` sin `try/catch`. Express 4 no captura el rechazo de la promesa, y Node.js 24 (la versión de la imagen) termina el proceso ante un rechazo no manejado. Por eso una excepción aislada, como la de una moneda inexistente, afecta a todos los usuarios.
- **Log y operaciones de negocio en el mismo proceso.** La degradación de `/log` arrastra a `/exchange`: en el escenario de resistencia aparecen fallas a partir del minuto 20 aunque la carga es constante (Figura 15).

2.3.2. Impacto en la Performance

- **Estado en memoria (positivo).** Las lecturas de cuentas y tasas no acceden a disco: `GET /rates` responde en torno a 1ms en P95 (Figura 5).
- **Transferencias secuenciales.** Cada cambio exitoso espera dos transferencias de entre 200 y 400ms, una después de la otra, por lo que `/exchange` no puede responder en menos de 400ms aun sin carga. Si fueran independientes podrían solaparse, pero la segunda depende del resultado de la primera (Código 3).
- **Serialización síncrona de un log sin límite.** `/log` serializa el arreglo completo con `res.json` (Código 4), operación síncrona que bloquea el event loop durante un tiempo proporcional al tamaño del log. A esto se suma la reescritura completa del archivo cada 1s. En el escenario de estrés, el CPU se satura y la latencia de `/exchange` comienza a crecer a partir del minuto 4 (Figuras 12 y 11).

2.3.3. Impacto en la Escalabilidad

- **Estado en memoria y en archivos locales.** Como se discutió en la Sección 1.1, agregar réplicas de la API produciría estados divergentes, por lo que el escalado horizontal no es posible sin mover el estado a un almacenamiento compartido.
- **Un único hilo de ejecución.** Node.js ejecuta el código de la aplicación en un solo hilo. En las pruebas de estrés y resistencia el CPU se satura en torno al 100 %, es decir un núcleo (Figuras 12 y 18), por lo que asignar más núcleos al contenedor no mejoraría la capacidad sin cambiar la arquitectura.
- **Costo creciente con el historial.** El costo de /log y de la persistencia crece con la cantidad de operaciones realizadas, de modo que más usuarios aceleran la degradación aunque la carga por segundo se mantenga (Figura 17).

2.3.4. Impacto en la Confiabilidad

- **Verificación de saldo sin exclusión mutua.** La condición de carrera del Código 3 permite aprobar dos operaciones con el mismo saldo.
- **Durabilidad diferida y escritura no atómica.** Las operaciones confirmadas al cliente pueden perderse si el proceso termina antes del siguiente guardado, y una interrupción durante la escritura puede dejar el archivo corrupto (Código 5).
- **Importes con Number.** El uso de punto flotante binario para dinero (Código 6) puede acumular diferencias en los saldos.

2.3.5. Impacto en la Seguridad

- **Validación por presencia.** La API acepta importes negativos, tipos incorrectos y monedas inexistentes (Código 7). Combinado con la falta de manejo de errores, una sola request malformada funciona como un ataque de denegación de servicio, como se evidencia en el escenario de caos (Figura 21).
- **Endpoints administrativos expuestos.** /accounts/<id>/balance y PUT /rates se publican a través del mismo nginx que los endpoints de los clientes. Si bien asumimos que el componente de autorización filtra quién puede invocarlos, la API no aplica ninguna defensa propia ni deja registro de estos cambios en el log.

2.3.6. Impacto en la Modificabilidad

- **Código reducido y dividido en tres módulos (positivo).** La separación entre app.js, exchange.js y state.js permite ubicar rápidamente cada responsabilidad.
- **Abstracción de persistencia permeable.** Los getters de state.js devuelven referencias mutables que exchange.js modifica directamente (Código 2). Reemplazar los archivos por una base de datos, como pide el fundador, obliga a modificar también la lógica de negocio.
- **Concentración de responsabilidades en exchange.js** (Código 1), que aumenta el acoplamiento entre cálculo, validación, transferencia y registro.

2.3.7. Impacto en la Testeabilidad

- **Estado global y dependencias fijas.** El estado se carga desde archivos al iniciar el módulo (Código 2) y el servicio de transferencias se invoca directamente, sin inyección de dependencias. Probar la lógica de cambio de forma aislada requiere preparar archivos reales y esperar los 200 a 400ms de cada transferencia simulada.
- **Ausencia de pruebas automatizadas,** como se mencionó en la Sección 2.2.3.

2.3.8. Impacto en la Eficiencia

- **Persistencia periódica completa.** Las cuentas y el log se serializan y escriben completos cada 1s, aunque no haya habido cambios o solo se haya agregado una operación (Códigos 4 y 5).
- **Log completo en memoria.** El uso de memoria crece durante toda la prueba de resistencia hasta un máximo de 22.5 % del límite del contenedor (Figura 18), sin que exista una política de retención.

2.3.9. Impacto en la Manejabilidad

- **Sin observabilidad propia.** El servicio recibido solo escribe mensajes en la consola; las métricas por endpoint y de volumen operado debieron agregarse para este trabajo (endpoint_metrics.js y currency_metrics.js). Como se vio en el escenario de caos, estas métricas se pierden cuando cae el proceso que las emite (Figura 23).
- **Estado dentro de la imagen.** El Dockerfile copia el directorio state en la imagen y no se declara un volumen (Sección 1.3), por lo que recrear el contenedor descarta el estado.
- **Configuración fija en el código.** El puerto y los períodos de persistencia (Código 4) no pueden ajustarse sin modificar el código y reconstruir la imagen.

2.3.10. Impacto en la Simplicidad

- **Un único proceso con almacenamiento en archivos (positivo).** El sistema completo puede entenderse leyendo tres archivos y levantarse con un solo contenedor, sin dependencias externas salvo el servicio de transferencias. Esta simplicidad es la que permitió construirlo rápidamente, y es también la causa de varias de las limitaciones anteriores.

2.4. Crítica

Las decisiones de diseño del sistema recibido son coherentes con el objetivo con el que fue construido: tener rápidamente un producto funcionando para conseguir inversores. Un único proceso con estado en memoria y persistencia en archivos es simple, fácil de desplegar y rápido para las consultas. Sin embargo, ese diseño se mantuvo cuando el servicio pasó a manejar dinero de usuarios reales, y los QA que sacrifica son precisamente los que identificamos como claves.

Los problemas más graves, en orden de impacto, son:

1. **Disponibilidad frágil:** una única request inválida deja el servicio fuera de línea de forma permanente, ya que se combinan la falta de validación, la falta de manejo de errores y la ausencia de cualquier mecanismo de recuperación (Sección 3.4).
2. **Degradación progresiva:** el log crece sin límite en el mismo proceso que atiende las operaciones de cambio, por lo que el servicio se degrada con el uso aunque la carga no aumente (Sección 3.3). Es la explicación más probable de los reclamos de los usuarios sobre la función de cambio.
3. **Correctitud de los saldos:** la condición de carrera, la durabilidad diferida y el uso de punto flotante (Sección 2.3.4) comprometen la integridad del dinero de los usuarios, lo cual es inaceptable para una auditoría.

Las propuestas de este trabajo atacan los tres puntos: la propuesta 1 (Sección 4) aísla el log de las operaciones de negocio, la propuesta 2 (Sección 5) agrega detección y recuperación de fallas, y la propuesta 3 (Sección 6) previene las caídas por requests inválidas y elimina la condición de carrera sobre los saldos mediante validación de entradas, manejo de excepciones y reserva de saldo. Quedan pendientes la representación decimal del dinero y la persistencia transaccional, que recomendamos como pasos siguientes, en particular al migrar el estado a una base de datos externa.

3. COMPORTAMIENTO DEL SERVICIO

3.1. Pruebas de carga

La prueba de carga se realizó usando una tasa constante de 40 usuarios durante 240s (4m). Cada usuario realiza un solo request, es decir se mantiene un throughput constante de 40 requests/s. No se realizó warm-up ni ramp para no ensuciar las métricas de promedios. Tampoco se realizaron recargas de saldo intermedias, se utilizó una carga inicial suficiente para cubrir toda la prueba.

Adicionalmente, aprovechando que siendo este el sistema inicial no se plantea una comparativa, decidimos usarla para ilustrar el formato de los gráficos y explicar los criterios compartidos entre todos los escenarios.

Se pueden encontrar los resultados completos en el Apéndice (Sección A.1).

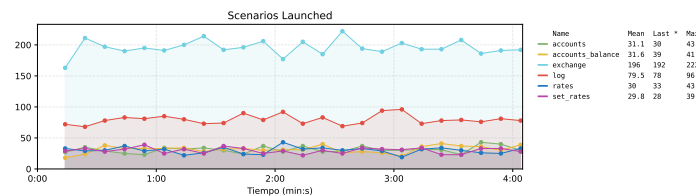


Figura 2. Para un primer diagnóstico del sistema se decidió utilizar todos los endpoints. Se asignaron 20 requests/s a /exchange por ser el endpoint principal en el contexto de negocio. Se asignaron 8 requests/s a /log por las hipótesis planteadas (Cuadro 1) sobre su significativo impacto en la disponibilidad y latencia del sistema. Se asignaron 3 requests/s al resto de endpoints. Para todos los gráficos salvo se indique lo contrario el eje x muestra el tiempo transcurrido de la prueba y cada punto agrupa una ventana de 10s. Adicionalmente en las leyendas de la derecha se muestran estadísticas relevantes al escenario, como por ejemplo el promedio, el último valor y/o el máximo.

3.1.1. Disponibilidad

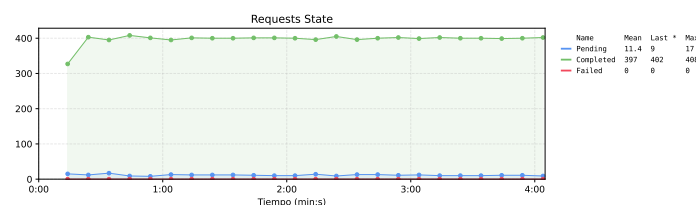


Figura 3. Para evaluar la disponibilidad del sistema monitoreamos el estado de las requests. Completed indica cuántas requests finalizaron correctamente en la ventana de 10s, mientras que Pending indica cuántas seguían en curso. En 4m con una carga de 40 requests/s, se puede observar que el sistema no presentó inconvenientes.

3.1.2. Latencia



Figura 4. Para evaluar la latencia del sistema optamos por utilizar dos metodologías, la primera monitorear los tiempos de respuestas desde la perspectiva del cliente a través de Artillery. Optamos por realizar el monitoreo en P50, P95 y P99, sin embargo una consideración importante para esta metodología es que no distingue entre endpoints. Una consecuencia de esto último es por ejemplo los saltos en P50 donde dependiendo si la ventana contiene mayormente requests rápidas o del endpoint /exchange, la mediana se desplaza entre ambos grupos.

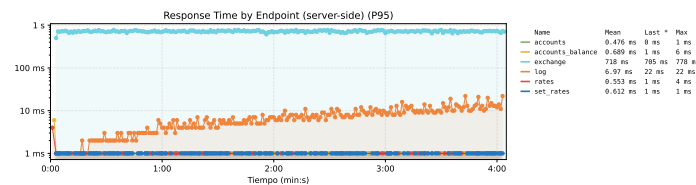


Figura 5. La siguiente metodología adoptada fue medir los tiempos de respuesta por endpoint desde la perspectiva del servidor. Se realizó agregando la recolección de métricas directamente en el servidor. Si bien en contraposición a la metodología anterior sí podemos discernir entre endpoints, optamos por representarlo solo para P95 para no saturar la representación. Para el eje y optamos por una escala logarítmica por la gran discrepancia entre los tiempos. Una observación que veníamos anticipando (Sección 2.1.2) es que los endpoints se agrupan en tres clases, /exchange lento por sus dependencias externas pero constante, /log que crece junto con el tamaño del propio log y los restantes los cuales son considerablemente más rápidos. En próximos escenarios nos vamos a limitar a monitorear un endpoint de cada grupo, es decir /exchange, /log y optamos por GET /rates como representante de los endpoints rápidos, de esta forma simplificar la representación.

3.1.3. Uso de recursos

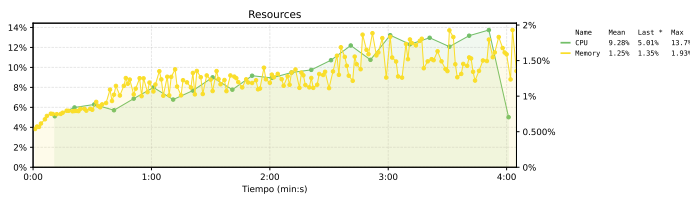


Figura 6. Para evaluar el uso de recursos del sistema, monitoreamos el uso del CPU y de la memoria. El uso del CPU se lee sobre el eje y al lado izquierdo mientras que el uso de la memoria al lado derecho. Nuevamente, mantiene el formato de tiempo en el eje x con ventanas de 10s. Una primera observación es que tanto el uso de CPU como el de memoria crece con el tiempo, se especula debido al crecimiento del log (Código 4).

3.1.4. Volumen operado por moneda

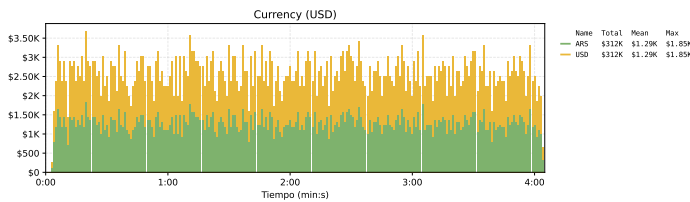


Figura 7. Como la primera métrica de negocio, se evaluó el volumen operado por cada moneda. Para esto se agregó la recolección de métricas directamente en el servidor. Optamos por representar cada moneda con su respectivo valor en dólares, a la cotización vigente en cada operación, para que los valores sean comparables. Los volúmenes de diferentes monedas operados en un mismo punto se apilan para representar el total operado en ese punto. Se mantiene el formato de tiempo en el eje x pero se redujo la ventana a 1s.

3.1.5. Volumen neto operado

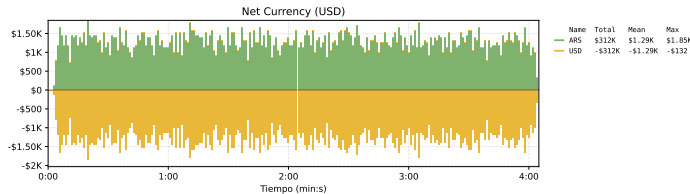


Figura 8. Para la segunda métrica de negocio, se evaluó el volumen neto operado por cada moneda. Nuevamente, a través de la recolección de métricas directamente en el servidor. De igual modo, con cada moneda en su respectivo valor en dólares. A diferencia del volumen operado, las ventas se representan como valores negativos y las compras con valores positivos. Se mantiene el formato de tiempo en el eje x y la ventana de 1s.

3.2. Pruebas de estrés

La prueba de estrés se realizó incrementando progresivamente la carga desde 40 hasta 180 en saltos de 20, cada salto con una duración de 45s, el tiempo total de la prueba es de 450s (7.5m). Se realizó un warm-up inicial de 5 usuarios durante 30s. Además, al final de la prueba se dejó una instancia de recovery de nuevamente 5 usuarios esta vez durante 60s. Tampoco se realizaron recargas de saldo intermedias y se simplifico el uso de endpoints dejando solo /exchange, /log y /rates luego de las conclusiones obtenidas en (Figura 2).

Se pueden encontrar los resultados completos en el Apéndice (Sección A.2).

3.2.1. Disponibilidad

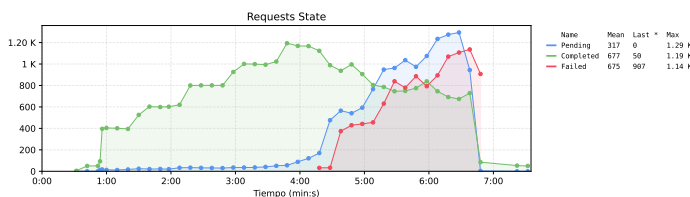


Figura 9. Se observa que el servidor bajo el crecimiento de la carga encuentra su punto de quiebre alrededor del minuto 4:30. A partir de ese momento, la disponibilidad del sistema se ve comprometida y se continúan completando solicitudes pero otras fallan.

3.2.2. Latencia

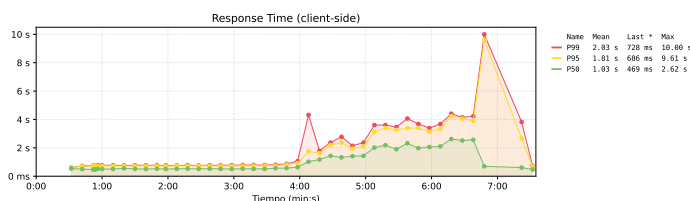


Figura 10. Se observa cómo previo a la pérdida de disponibilidad antes mencionada (minuto 4:30) el servidor ya empieza a presentar una degradación en los tiempos de respuesta desde el minuto 4.

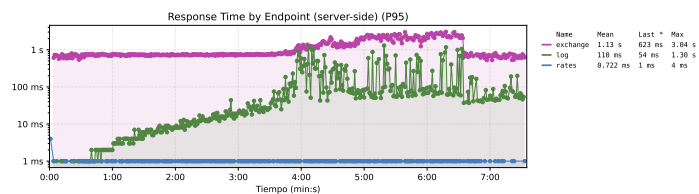


Figura 11. Se observa cómo el crecimiento en los tiempos de respuesta de /log, dado por el crecimiento del log, impacta directamente en la latencia del endpoint /exchange.

3.2.3. Uso de recursos

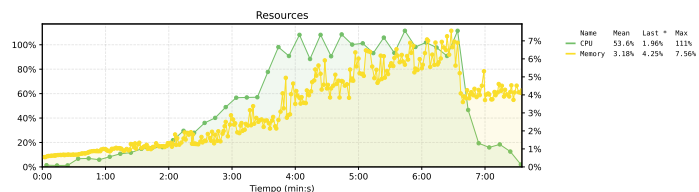


Figura 12. Se observa que el cuello de botella del servidor es el CPU, ya que alcanza su máxima capacidad en el minuto 4, junto con el comienzo de la degradación del servidor. Igualmente aunque en menor medida la memoria también muestra signos de crecimiento. Ambos factores generados por el crecimiento del log, lo que aumenta el costo de serialización para el CPU y el uso de memoria para almacenar los datos.

3.2.4. Volumen operado por moneda

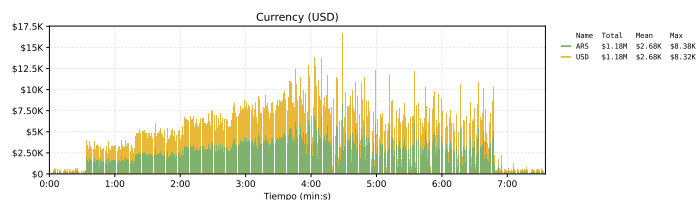


Figura 13. Se observa cómo se degrada el volumen operado a partir del minuto 4, esto implica potenciales pérdidas económicas tanto para usuarios como para la plataforma.

3.2.5. Volumen neto operado

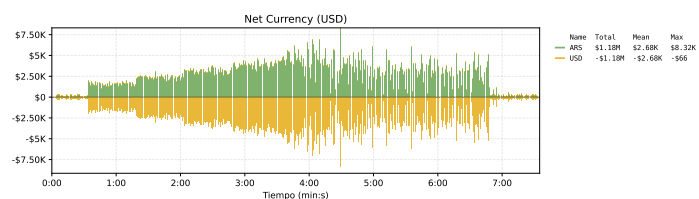


Figura 14. Para el volumen neto, se observan los mismos resultados expuestos para el volumen operado por moneda.

3.3. Pruebas de resistencia

La prueba de resistencia se realizó usando una tasa constante de 40 usuarios durante 2700s (45m). Se realizó un warm-up inicial de 5 usuarios durante 60s. Además, al final de la prueba se dejó una instancia de recovery de nuevamente 5 usuarios esta vez durante 120s (2m). Tampoco se realizaron recargas de saldo intermedias y se simplificó el uso de endpoints dejando solo /exchange, /log y /rates luego de las conclusiones obtenidas en (Figura 2).

Se pueden encontrar los resultados completos en el Apéndice (Sección A.3).

3.3.1. Disponibilidad

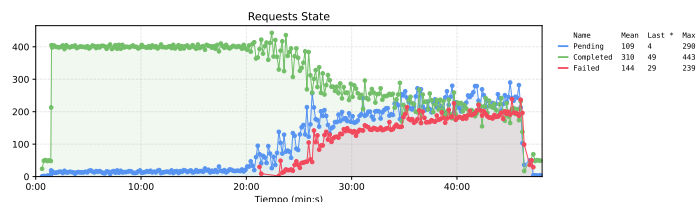


Figura 15. A diferencia del escenario de estrés (Figura 9), en este caso se observa que la pérdida de disponibilidad no es producto del aumento de la carga sino que también se produce la misma degradación bajo un escenario prolongado de carga continua. Las primeras requests fallidas empiezan a ocurrir posterior al minuto 20. A partir de ahí se observa un incremento sostenido en la cantidad de fallos hasta la fase de recuperación al final de la prueba.

3.3.2. Latencia

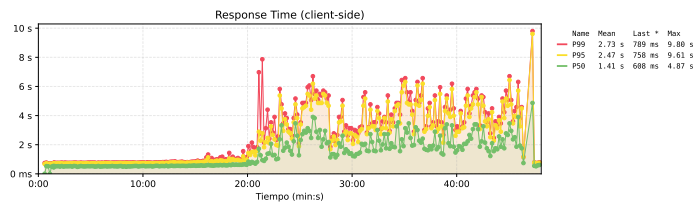


Figura 16. Se observa cómo nuevamente el aumento en la latencia desde la perspectiva del cliente es en el minuto 20, previo a la aparición de las primeras requests fallidas. Coherente con una degradación progresiva y no un error abrupto.

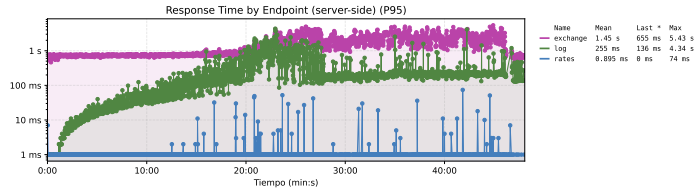


Figura 17. En cuanto a la latencia por endpoint se observan los mismos síntomas que los descritos en la prueba de estrés (Figura 11). El endpoint /log perjudica la latencia del endpoint /exchange conforme el tamaño del log aumenta y con este su tiempo de respuesta. Este comportamiento se evidencia a partir del minuto 20, consistente con las conclusiones de disponibilidad y latencia antes mencionadas.

3.3.3. Uso de recursos

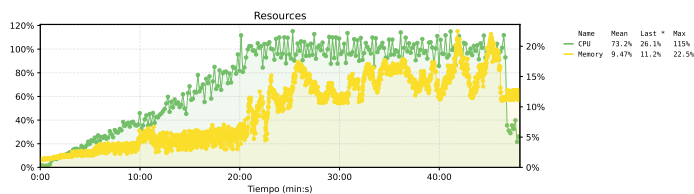


Figura 18. Se observa un incremento en el uso de recursos, especialmente de CPU, alcanzando este el máximo disponible en el minuto 20. El uso de memoria también siguió incrementando con respecto a las observaciones en las pruebas de carga (Figura 6), llegando hasta un máximo de 22.5 % de la memoria disponible.

3.3.4. Volumen operado por moneda

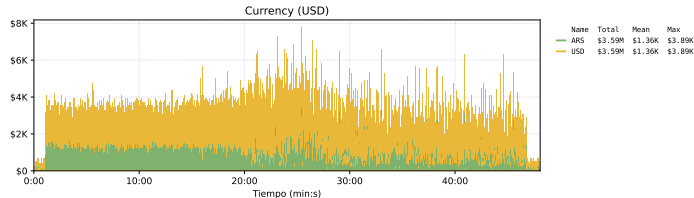


Figura 19. Se observa una degradación en el volumen operado a partir del minuto 20 con respecto a los minutos previos.

3.3.5. Volumen neto operado

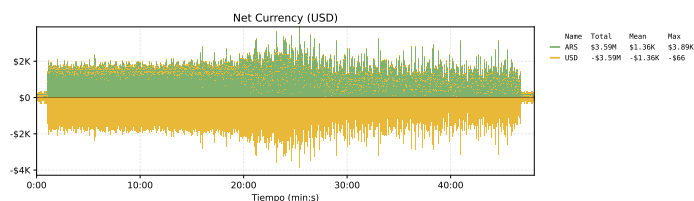


Figura 20. Se observa una tendencia similar en el volumen neto operado, con una disminución notable a partir del minuto 20.

3.4. Pruebas de caos

La prueba de caos se realizó usando una tasa constante de 40 usuarios durante 480s (8m). El reparto de requests se mantiene igual que en la prueba de resistencia (Figura 2). Adicionalmente se agrega una request malintencionada sabiendo que no hay un buen manejo de errores (Sección 2.3.1). Este tiene una probabilidad suficiente para aparecer entre 2 o 3 veces a lo largo de la prueba. Cada usuario realiza un solo request, es decir se mantiene un throughput constante de 40 requests/s. No se realizó warm-up ni ramp porque no aportaban información relevante para esta prueba. Tampoco se realizaron recargas de saldo intermedias, se utilizó una carga inicial suficiente para cubrir toda la prueba.

Se pueden encontrar los resultados completos en el Apéndice (Sección A.4).

3.4.1. Disponibilidad

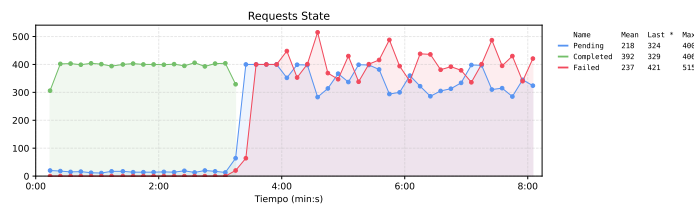


Figura 21. Se observa cómo el sistema cae cerca del minuto 3:15 en la primera request inválida. A partir de ese momento las requests no se completan y esperan un timeout completo de 10s. Se verifica que el sistema no se recupera como se mencionó en la Sección 1.1.

3.4.2. Latencia

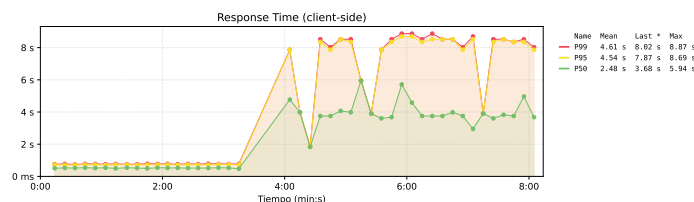


Figura 22. Posterior a la caída se observa el aumento en los tiempos de respuesta siendo estas devueltas por nginx como errores 502 en lugar de por el propio servidor.

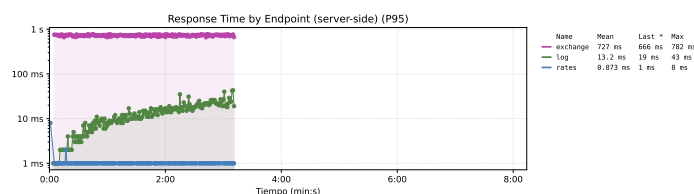


Figura 23. No se observan métricas posteriores al momento de la caída. Esto se debe a que el mecanismo para recolectar las métricas es parte del propio servidor y deja de funcionar cuando el servidor cae.

3.4.3. Uso de recursos

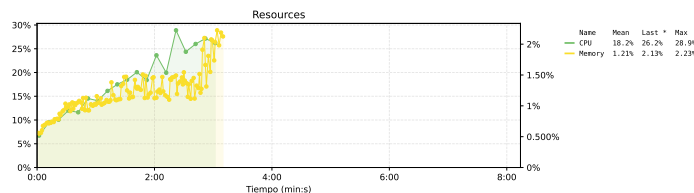


Figura 24. De igual manera, se dejan de reportar las métricas de uso de recursos posterior a la caída del servidor. Esto se debe a la caída del propio proceso.

3.4.4. Volumen operado por moneda

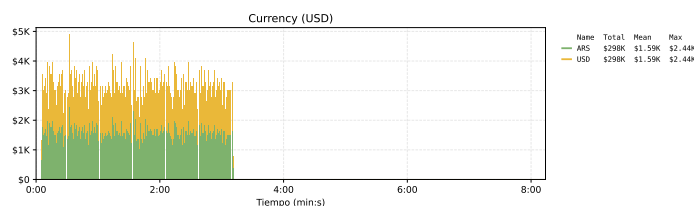


Figura 25. Posterior a la caída y como consecuencia de esta se deja de operar, el total se estanca en \$298K por moneda en 8m.

3.4.5. Volumen neto operado

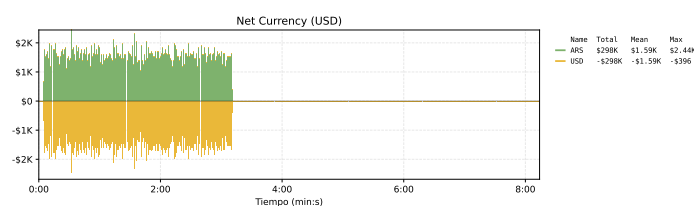


Figura 26. Análogamente, posterior a la caída y como consecuencia de esta se deja de operar y el volumen neto se estanca.

3.5. Resultados y conclusiones

Los resultados obtenidos del escenario de carga nos indican que podemos clasificar los endpoints en tres categorías (Figura 5) lo que nos permitirá simplificar nuestro análisis. Adicionalmente, los resultados del escenario de estrés (Sección 3.2) y resistencia (Sección 3.3) nos

indican que el sistema presenta una degradación significativa por la mala gestión del log. Esta observación motivará la propuesta 1 (Sección 4). Finalmente, los resultados del escenario de caos (Sección 3.4) nos indican que el sistema base no provee mecanismos de recuperación. Esta observación motivará la propuesta 2 (Sección 5).

4.1.4. Escalabilidad

El desacoplamiento permite asignar recursos de forma independiente a la API de exchange y al nodo de logs según la carga de cada uno: como se vio en el escenario de estrés, la API queda casi ociosa mientras el nodo de logs concentra el consumo (Figura 33). La API de exchange, en cambio, sigue sin poder replicarse, ya que mantiene las cuentas y las tasas en memoria (Sección 2.3.3).

El desacoplamiento es además una condición previa para escalar horizontalmente la consulta del log, pero la implementación actual no lo permite sin cambios. Replicar el nodo de logs requeriría:

- que las réplicas compartan el archivo (por ejemplo, mediante un volumen) o un almacenamiento externo, ya que actualmente el archivo vive dentro de cada contenedor;
- modificar la configuración de nginx, cuyo upstream apunta a un único contenedor (exchange-log-api-1);
- modificar la API de exchange, que abre una única conexión TCP con el nodo de logs al arrancar, por lo que todas las escrituras llegarían a una sola réplica.

4.1.5. Seguridad

El puerto TCP 3002 del nodo de logs acepta registros de cualquier cliente dentro de la red de Docker, sin autenticación ni validación del contenido. Cualquier contenedor comprometido podría insertar registros falsos en el log de transacciones, lo que afecta su integridad. El puerto no se publica al host y nginx solo expone /log, por lo que el riesgo queda acotado a la red interna.

4.1.6. Modificabilidad

Se separan responsabilidades que antes estaban mezcladas en `state.js` (Código 2): la API de exchange ya no conoce cómo se almacena ni cómo se consulta el log, lo que permitiría cambiar el almacenamiento (por ejemplo, a una base de datos) sin tocarla. La interfaz pública de /log se mantiene sin cambios para los clientes. En contrapartida, el protocolo entre ambos nodos (JSON delimitado por saltos de línea sobre TCP) es ad hoc, y los nombres de host y puertos están fijos en el código y en la configuración de nginx.

4.1.7. Costo y simplicidad

Se agrega un contenedor con los mismos límites de recursos que la API (1 CPU y 4 GB de memoria), lo que duplica los recursos reservados. Según las mediciones, la API de exchange queda prácticamente ociosa, con un 14.2 % de CPU en el escenario de estrés (Figura 33), por lo que sus límites podrían reducirse. La arquitectura suma un componente y un conector nuevo, así como un orden de arranque a respetar, aunque la complejidad agregada es menor que la de la propuesta 2 (Sección 5.1.6).

4.2. Pruebas de carga

Las pruebas de carga utilizan la misma configuración que en el sistema base (Sección 3.1), de modo que los resultados sean comparables. Con esta carga y esta duración el log todavía es chico, por lo que no esperamos que la degradación del sistema base llegue a manifestarse; el efecto principal de la táctica se observa en los escenarios de estrés y resistencia.

Ambos sistemas completaron todas las requests sin fallas, con un promedio de 385 requests completadas por ventana de 10s en la propuesta 1 y de 397 en el sistema base. La diferencia se debe a la primera ventana de la propuesta 1, que registra solo 38 requests por corresponder al inicio de la prueba; a partir de la segunda ventana ambos sistemas completan alrededor de 400. Los tiempos de respuesta de /exchange son equivalentes (P95 de 716ms frente a 718ms medido en el servidor, y P95 de 709ms frente a 712ms visto desde el cliente), ya que están acotados por las dos transferencias.

La diferencia aparece en /log, cuyo P95 promedio se reduce a la mitad (3.61ms frente a 6.97ms, con un máximo de 11ms frente a 22ms), y en el uso de recursos de la API, cuyo CPU promedio baja del 9.28 % al 4.02 % y su memoria del 1.25 % al 0.81 %, al dejar de mantener, serializar y persistir el log. El nodo de logs consume en promedio un 3.35 % de CPU, por lo que el consumo sumado de ambos nodos (7.37 %) es menor que el de la API del sistema base. Su memoria crece durante la prueba, del 0.5 % al 3.67 %; en el escenario de resistencia se observa que este crecimiento se da en los primeros minutos y luego se estabiliza en torno al 5 %, sin crecer con el tamaño del log (Figura 39).

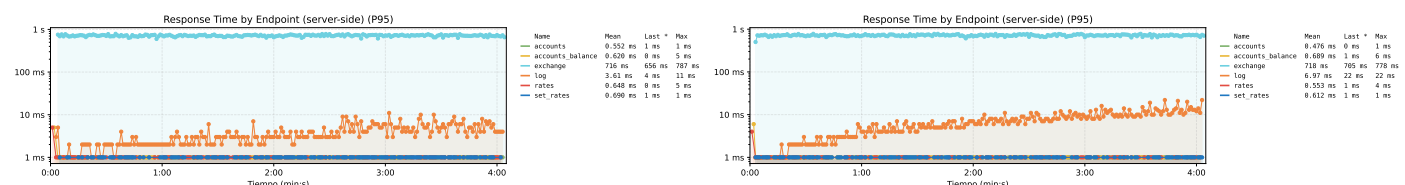


Figura 28. Comparativa de los resultados de latencia por endpoint entre la propuesta 1 (izquierda) y el sistema base (derecha) frente al escenario de carga. /exchange se mantiene en torno a 720ms (P95) en ambos casos, mientras que /log crece más lentamente en la propuesta 1, con un promedio de 3.61ms frente a 6.97ms.

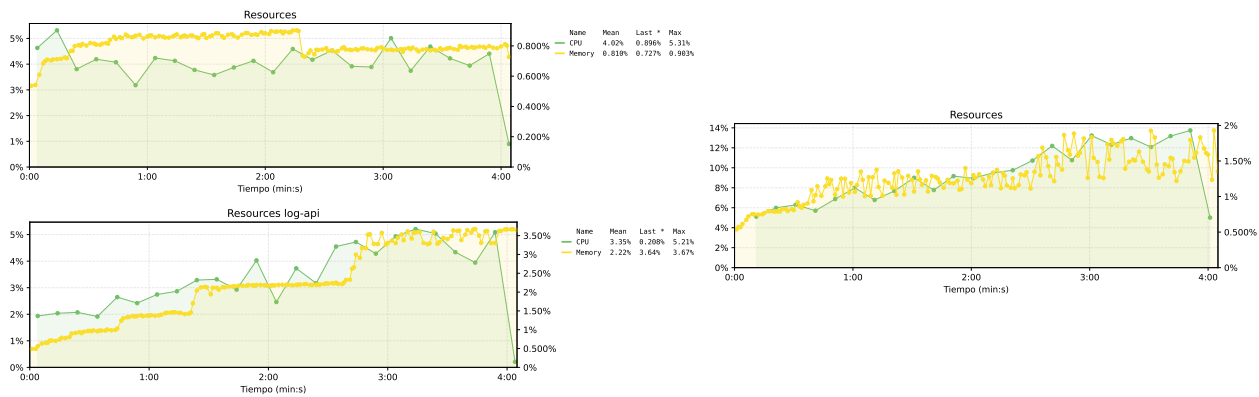


Figura 29. Comparativa del uso de recursos entre la propuesta 1 (izquierda, arriba la API de exchange y abajo el nodo de logs) y el sistema base (derecha) frente al escenario de carga. La API de exchange reduce su CPU promedio del 9.28 % al 4.02 %, y el consumo sumado de ambos nodos (7.37 %) es menor que el del sistema base.

Se pueden encontrar los resultados completos en el Apéndice (Sección A.5).

4.3. Pruebas de estrés

Las pruebas de estrés comparten la misma configuración con el sistema base de tal forma que los resultados sean comparables, ver Sección 3.2.

Se pueden encontrar los resultados completos en el Apéndice (Sección A.6).

4.3.1. Disponibilidad

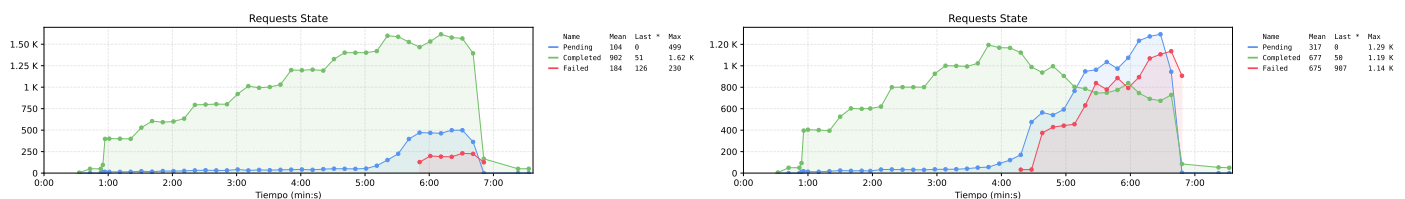


Figura 30. Comparativa de la disponibilidad ofrecida por la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de estrés. La propuesta 1 obtuvo un incremento en el promedio de respuestas satisfactorias de 225 respuestas adicionales, lo que representa un aumento del 33.23 % frente al caso base.

4.3.2. Latencia

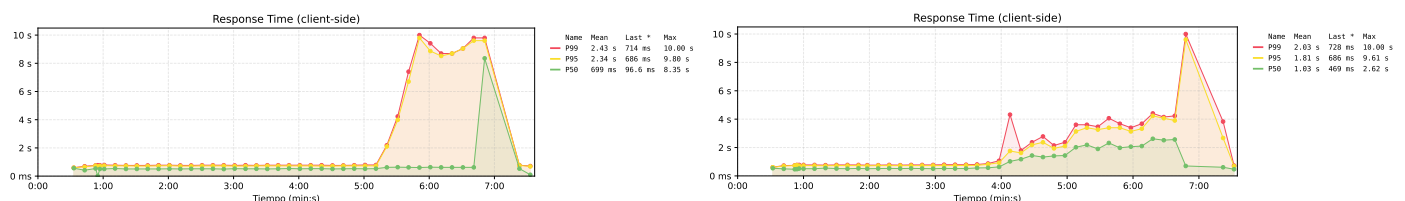


Figura 31. Comparativa de los resultados de latencia entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de estrés. Se observa que la propuesta 1 mejoró el promedio de latencia para P50. También se observa que empeoró en P95 y P99.

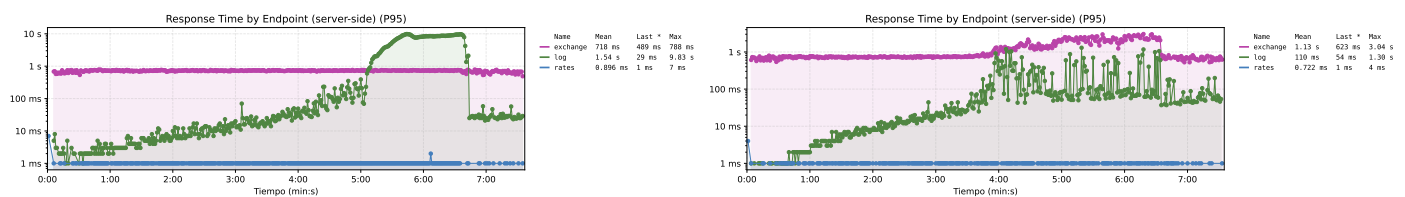


Figura 32. Comparativa de los resultados de latencia por endpoint entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de estrés. Con diferencia el análisis más interesante, la propuesta 1 permitió como se esperaba mejorar la latencia en el endpoint /exchange, se puede notar que este no se ve afectado por la degradación del endpoint /log. Como sí ocurría en el caso base, donde alrededor del minuto 4, los tiempos de respuesta de /exchange incrementaban, en este caso, los tiempos de respuesta se mantienen estables en 1s, acotado por la dependencia del servicio externo. Por otro lado, los tiempos de respuesta de /log empeoraron, sin embargo es esperable, la mayor disponibilidad antes mencionada implica que se registraron más transacciones, por lo que el tamaño del log necesariamente aumentó. Esto explica el aumento de los tiempos de respuesta visto desde el lado del cliente antes mencionados, sin embargo, las métricas se ven ensuciadas al mezclar todos los endpoints.

4.3.3. Uso de recursos

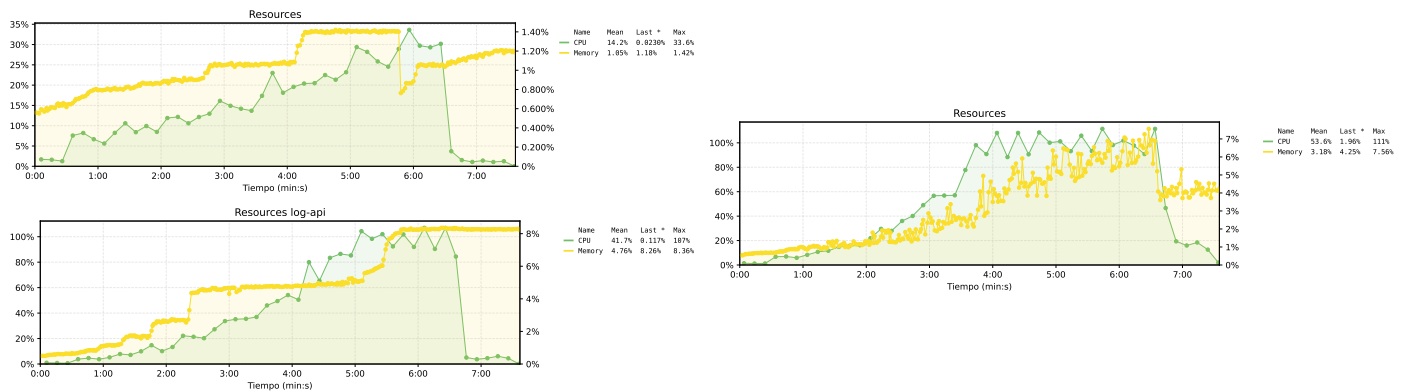


Figura 33. Comparativa de los resultados del uso de recursos entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de estrés. Destacar que consideramos la comparación con respecto a ambos nodos, el nodo de exchange y el de logs. Se puede observar que el nodo de logs mantiene la carga con respecto a la del sistema base, incluso la aumenta. Esto es esperable con el aumento de la disponibilidad, más requests completadas implican más entradas en el log. Sin embargo, el nodo de exchange apenas llegó a un 14.2 % de uso de CPU y 1.42 % de uso de memoria.

4.3.4. Volumen operado por moneda

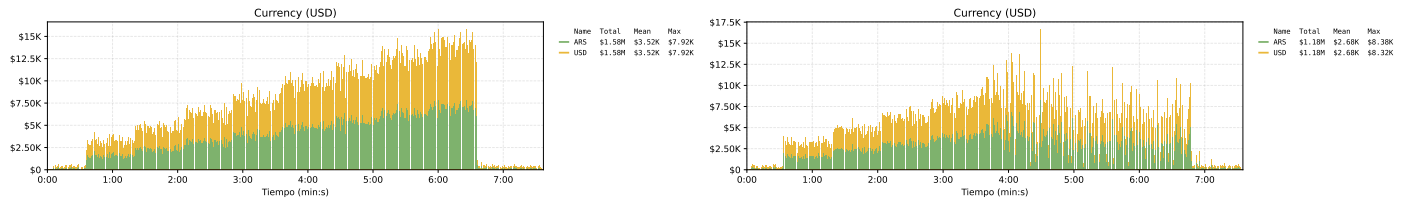


Figura 34. Comparativa de los resultados de volumen operado en cada moneda entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de estrés. Se observa que la propuesta 1 obtuvo un mayor volumen total siendo este 1.58M frente al 1.18M del sistema base, es decir una mejora del 33.90 %.

4.3.5. Volumen neto operado

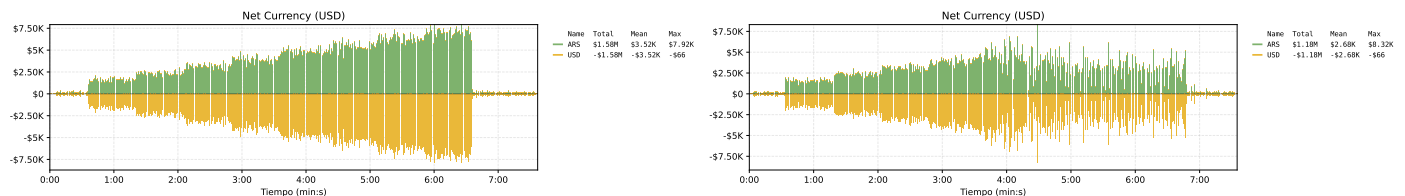


Figura 35. Comparativa de los resultados de volumen neto operado en cada moneda entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de estrés. Análogamente al volumen operado, se observa que el volumen neto también es mayor en la propuesta 1 frente al sistema base.

4.4. Pruebas de resistencia

Las pruebas de resistencia comparten la misma configuración con el sistema base de tal forma que los resultados sean comparables, ver Sección 3.3. Este es el escenario en el que esperamos ver el efecto de la táctica.

Se pueden encontrar los resultados completos en el Apéndice (Sección A.7).

4.4.1. Disponibilidad

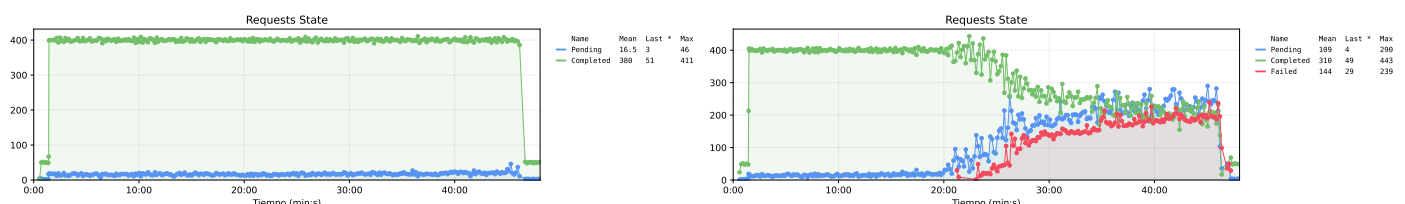


Figura 36. Comparativa de la disponibilidad ofrecida por la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de resistencia. El sistema base mostraba una degradación crítica en el minuto 20, mientras que la propuesta 1 logró completar satisfactoriamente todas las requests durante los 45m.

4.4.2. Latencia

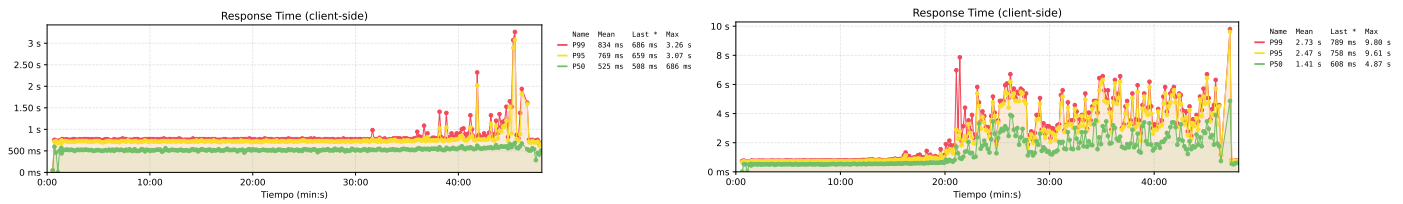


Figura 37. Comparativa de los resultados de latencia entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de resistencia. Consistente con los resultados de disponibilidad se muestra una gran mejora especialmente posterior al minuto 20. Sin embargo, la propuesta 1 sigue mostrando un principio de degradación en el minuto 40, esperable dado el crecimiento del log.

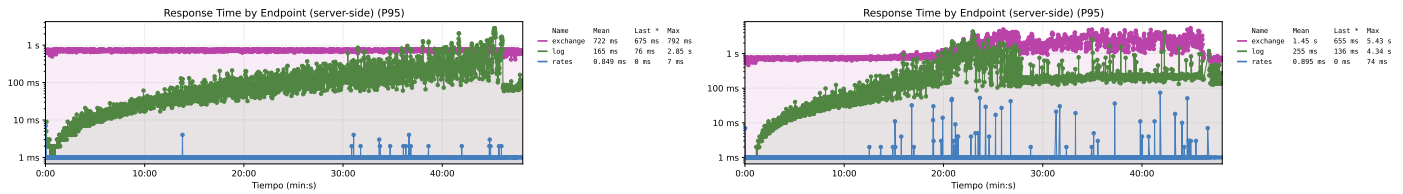


Figura 38. Comparativa de los resultados de latencia por endpoint entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de resistencia. Para el tiempo de respuesta desde la perspectiva del servidor, se muestra cómo en la propuesta 1 los tiempos del endpoint /log ya no “empujan” los tiempos del endpoint /exchange, sin embargo como ya se mencionó (Figura 32), el endpoint /log continúa degradando. Los tiempos de respuesta de /log entre ambos casos lamentablemente no son comparables puesto que menor disponibilidad implica un menor crecimiento del propio log.

4.4.3. Uso de recursos

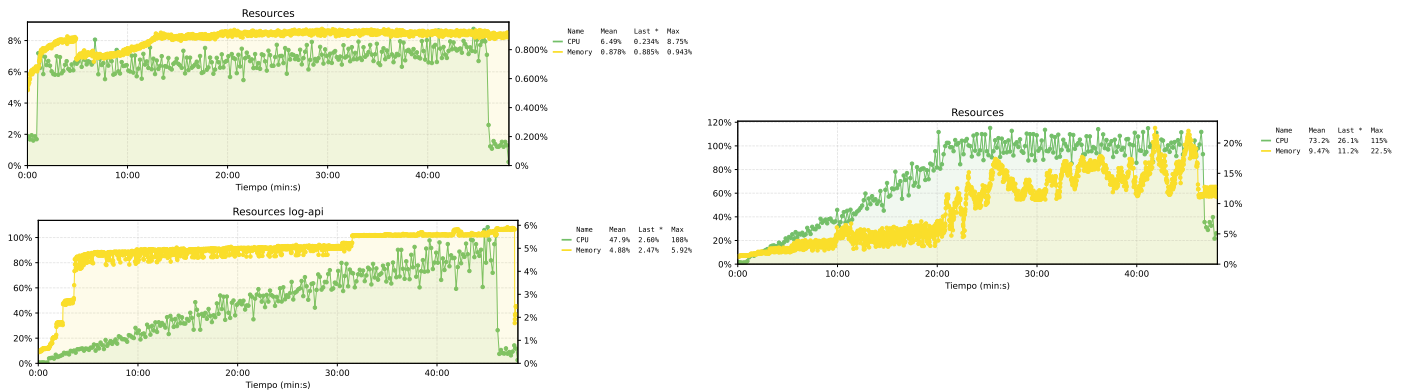


Figura 39. Comparativa de los resultados del uso de recursos entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de resistencia.

Destacar que ambos casos no son directamente comparables, la propuesta 1 dispone de dos contenedores/nodos independientes cada uno con los mismos recursos que el nodo monolítico del escenario base. Se observa que para el nodo de exchange-api los recursos son prácticamente ociosos con apenas un uso del 6.4 % de CPU y 0.8 % en memoria. También se observa en los recursos del nodo de logs, un uso más constante de la memoria, esperable al streamear los datos desde disco y no conservar todos los logs en memoria.

4.4.4. Volumen operado por moneda

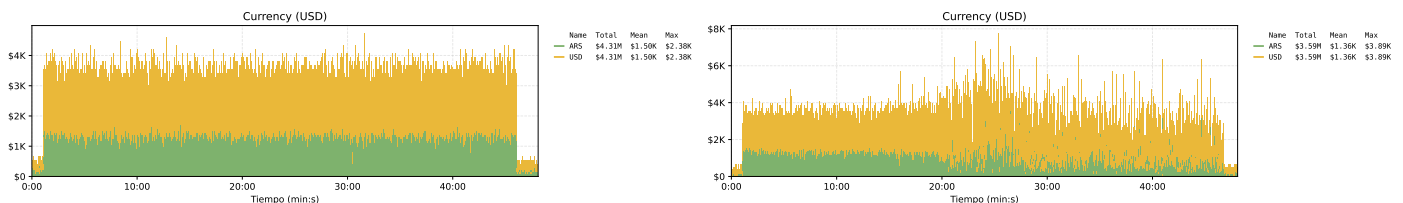


Figura 40. Comparativa de los resultados de volumen operado en cada moneda entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de resistencia. Se observa que la propuesta 1 obtuvo un mayor volumen total siendo este 4.31M frente a 3.59M del sistema base, es decir una mejora del 20.06 %.

4.4.5. Volumen neto operado

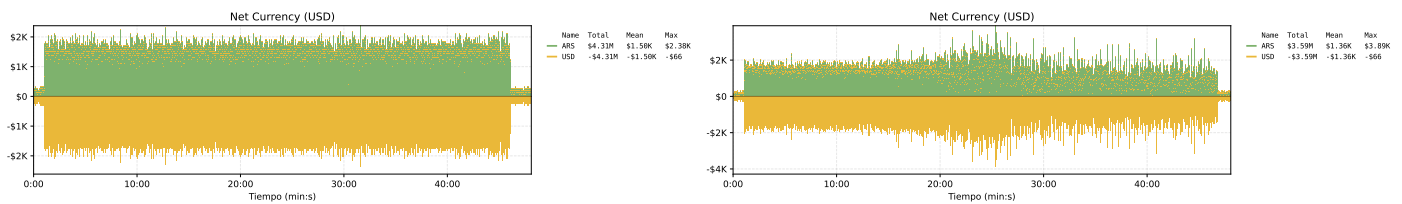


Figura 41. Comparativa de los resultados de volumen neto operado en cada moneda entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de resistencia. Análogamente al volumen operado, se observa que el volumen neto también es mayor en la propuesta 1 frente al sistema base en la misma proporción.

4.5. Pruebas de caos

No ejecutamos el escenario de caos sobre la propuesta 1, ya que la táctica no modifica ninguno de los elementos que determinan su resultado en el sistema base (Sección 3.4):

- la validación de `/exchange` en `app.js` sigue siendo por presencia (Código 7), por lo que la request con la moneda inexistente XXX es aceptada;
- `exchange.js` sigue accediendo a `rates[baseCurrency][counterCurrency]` sin verificar que la moneda exista, lo que lanza una excepción;
- el handler de `/exchange` sigue siendo una función `async` sin `try/catch`, por lo que la excepción termina el proceso (Sección 2.3.1);
- el `docker-compose.yml` de la propuesta no agrega una política de reinicio ni un health check para la API.

Por lo tanto, esperamos el mismo resultado que en el sistema base: la API termina con la primera request inválida y no vuelve a atender requests durante el resto de la prueba. El nodo de logs no cambia este resultado, ya que la propuesta solo lo desacopla de las operaciones de negocio y no interviene en el procesamiento de `/exchange`. La recuperación frente a este tipo de fallas es el objetivo de la propuesta 2 (Sección 5).

4.6. Resultados y conclusiones

La propuesta 1 demostró ser capaz de aislar la degradación producida por la consulta de logs de las operaciones de negocio del sistema, permitiendo ofrecer una mejor disponibilidad a los usuarios comunes (Figura 36). Por otro lado, la consulta de logs sigue degradándose con el tiempo. El desacoplamiento es un primer paso para escalar horizontalmente la lectura del log, pero para lograrlo las réplicas del nodo de logs deberían compartir el almacenamiento, y `nginx` y la API de `exchange` deberían poder distribuir las lecturas y las escrituras entre ellas (Sección 4.1.4). Como se mencionó en la Sección 4, la solución de fondo es paginar la consulta.

5. PROPUESTA 2

El escenario de caos del sistema base (Sección 3.4) mostró que una única request inválida alcanza para terminar el proceso de la API y que, a partir de ese momento, el servicio permanece interrumpido hasta que alguien reinicie la API manualmente: no existe ningún mecanismo de detección ni de recuperación de fallas. Si bien la causa raíz (la falta de validación de las requests, ver Código 7) debe corregirse, no es posible garantizar que no existan otras fallas que terminen el proceso. Por eso la propuesta 2 no apunta a evitar la falla, sino a acotar el tiempo que el servicio permanece caído cuando esta ocurre.

Para ello aplicamos las siguientes tácticas de disponibilidad:

- **Heartbeat (detección de fallas):** la API envía cada 1s un datagrama UDP a cada uno de los watchdogs indicando que está viva.
- **Monitor / watchdog (detección de fallas):** se agrega un componente independiente de la API que, si no recibe un heartbeat de la API durante 3s, la considera caída.
- **Restart (recuperación):** al detectar la caída, el watchdog reinicia el contenedor de la API a través de la API de Docker (montando `/var/run/docker.sock`).
- **Redundancia activa del monitor:** para que el watchdog no sea un nuevo punto único de falla, se despliegan tres instancias. Solo una de ellas (el líder) actúa sobre la API; el líder se elige mediante el algoritmo *Bully* y el resto le envía heartbeats. Si el líder deja de responder durante 3s, se inicia una nueva elección.

Clasificamos la redundancia del monitor como activa, y no como pasiva, porque las tres instancias procesan las mismas entradas en paralelo: la API envía su heartbeat a los tres watchdogs y cada uno actualiza su registro del último heartbeat recibido, aunque solo el líder actúe sobre la API (Código 8). En una redundancia pasiva, en cambio, solo la instancia principal procesaría las entradas y las demás recibirían actualizaciones periódicas de su estado.

```

1 // app/heartbeat.js (propuesta 2): la API notifica a los tres watchdogs
2 const WATCHDOGS = ["watchdog-1", "watchdog-2", "watchdog-3"];
3 setInterval(() => {
4     for (const host of WATCHDOGS) {
5         socket.send("api", WATCHDOG_PORT, host, () => {});
6     }
7 }, HEARTBEAT_PERIOD);
8
9 // app/watchdog.js (propuesta 2): todos registran el heartbeat...
10 if (type == "api") {
11     lastApiHeartbeat = Date.now();
12 }
13
14 // ...pero solo el lider actua sobre la API
15 setInterval(function() {
16     if (leader == ID) {
17         checkApi();
18     } else {
19         // seguidor: heartbeat al lider y nueva eleccion si no responde
20     }
21 }, HEARTBEAT_PERIOD);

```

Código 8. Heartbeat de la API a los tres watchdogs y supervisión a cargo del líder en `app/heartbeat.js` y `app/watchdog.js`.

La consecuencia práctica es que, cuando un seguidor es elegido líder, ya conoce el estado de la API y puede comenzar a supervisarla de inmediato, sin necesidad de sincronizar estado con el líder anterior. Si la redundancia fuera pasiva, un seguidor que no hubiera recibido los heartbeats podría declarar caída a una API sana al asumir el liderazgo, o demorar la detección de una caída real hasta reconstruir ese estado.

Cabe destacar que esta propuesta se implementó sobre el sistema base y no sobre la propuesta 1, de modo que los resultados reflejen solo el efecto de estas tácticas y sean comparables con los del sistema base.

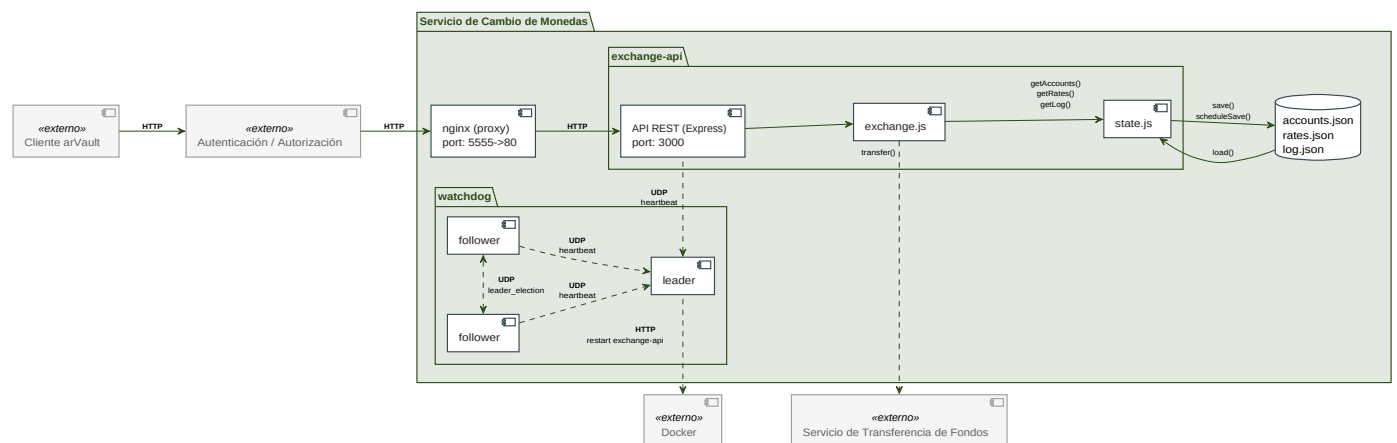


Figura 42. Diagrama Components & Connectors de la propuesta 2. Respecto al sistema base agrega tres watchdogs (un líder y dos seguidores) que monitorean la API mediante heartbeats y la reinician a través de Docker.

5.1. Impacto en los QA

5.1.1. Disponibilidad

Es el QA que la propuesta busca mejorar. Frente a fallas que terminan el proceso (*crash*), el tiempo durante el cual el servicio no está disponible pasa de ser indefinido a estar acotado por el tiempo de detección (hasta 3s de timeout más 1s del periodo de chequeo) sumado al tiempo de reinicio del contenedor. Como se verá en las pruebas de caos (Sección 5.5), esto se traduce en caídas de pocos segundos en lugar de la pérdida total del servicio.

Sin embargo, el heartbeat solo indica que el proceso está vivo y que su event loop avanza, no que responda en un tiempo aceptable. Una degradación de performance como la causada por el log (Sección 3.3) no es detectada, y por lo tanto no esperamos mejoras en los escenarios de estrés y resistencia.

El valor de 3s para el timeout es un tradeoff: reducirlo disminuye el tiempo de recuperación, pero aumenta el riesgo de falsos positivos. Dado que el heartbeat se envía desde el mismo event loop que atiende las requests, una operación síncrona larga (por ejemplo, serializar un log muy grande) podría retrasarlo y provocar el reinicio de una API que en realidad seguía funcionando, descartando todas las requests en curso.

5.1.2. Confiabilidad

El reinicio no resuelve la pérdida de datos. El estado se mantiene en memoria y se persiste a disco cada 1s (ver Código 5), por lo que ante un crash se pierden las operaciones realizadas desde el último guardado, además de las requests que estaban siendo procesadas. Como se utiliza `docker restart` y no se recrea el contenedor, el sistema de archivos se conserva y la API retoma desde el último estado guardado. La evidencia de esto se observa en la Figura 58, donde el tiempo de respuesta de /log sigue creciendo después de cada reinicio en lugar de volver a su valor inicial.

Además, la táctica no corrige la causa de la falla: la request inválida sigue terminando el proceso cada vez que llega (Figura 56). La propuesta mitiga el efecto, pero la validación de las requests sigue siendo necesaria.

5.1.3. Performance

El único trabajo agregado a la API es el envío de 3 datagramas UDP/s, que consideramos despreciable frente al procesamiento de las requests. Los watchdogs corren en contenedores separados, por lo que no compiten directamente con la API, aunque sí comparten el host. En las pruebas de estrés (Sección 5.3) y resistencia (Sección 5.4) no se observan diferencias atribuibles a la táctica.

5.1.4. Seguridad

Para poder reiniciar la API, los watchdogs montan el socket de Docker del host. Esto les otorga control total sobre el daemon de Docker, que es equivalente a tener acceso de administrador al host. Una vulnerabilidad en el watchdog compromete todo el servidor. Además, los mensajes UDP entre la API y los watchdogs no están autenticados, por lo que cualquier proceso dentro de la red podría enviar heartbeats falsos (enmascarando una caída) o mensajes de elección.

5.1.5. Escalabilidad y modificabilidad

El nombre del contenedor supervisado (`exchange-api-1`) y la lista de watchdogs están fijos en el código. Esto impide escalar horizontalmente la API sin modificar el watchdog, y agregar o quitar watchdogs requiere modificar tanto el watchdog como la API. Por otro lado, la API no conoce al watchdog más allá del envío del heartbeat, por lo que la lógica de supervisión puede evolucionar sin cambios en el servicio.

5.1.6. Simplicidad

La arquitectura pasa de tener un único proceso a tener cuatro, e incorpora un algoritmo distribuido de elección de líder que agrega nuevos modos de falla: por ejemplo, durante una partición de red podrían existir dos líderes y ambos reiniciarían la API. Una alternativa más simple, como la política `restart: on-failure` de Docker, cubriría el caso de crash con menos componentes, pero no detectaría un proceso colgado y dejaría la recuperación atada al daemon de Docker del host.

5.1.7. Costo y eficiencia

La propuesta agrega tres contenedores. A diferencia de la API, que tiene un límite de 1 CPU y 4 GB de memoria, los watchdogs no declaran límites de recursos en el `docker-compose.yml`. Como todo el sistema corre en un único servidor (Sección 2.2.2), un watchdog con un consumo anómalo competiría por los recursos con la API sin ninguna cota. Además, los watchdogs se construyen a partir de la misma imagen que la API (`build: ./app`), por lo que cada uno incluye el código y el estado de la aplicación aunque solo ejecuta `watchdog.js`. A la inversa, la dependencia `dockerode`, que solo usa el watchdog, se agrega al `package.json` de la API.

De los tres watchdogs, solo el líder supervisa la API; los otros dos permanecen en espera y únicamente intercambian heartbeats con él. Este es el costo de la redundancia del monitor: dos contenedores que no aportan trabajo mientras el líder funcione, a cambio de que la supervisión sobreviva a su caída.

Sobre la API, el costo es el envío de 3 datagramas UDP/s. En el escenario de carga no se observa una diferencia en el uso de recursos frente al sistema base (9.68 % frente a 9.28 % de CPU promedio, Sección 5.2), por lo que la táctica no reduce la eficiencia de la API.

TODO: Agregar captura de `docker stats` con el consumo de los tres watchdogs durante una prueba, ya que el dashboard solo grafica el contenedor de la API.

Cuadro 2. Costo agregado por cada propuesta respecto del sistema base, según su docker-compose.yml y las mediciones de las pruebas.

	Propuesta 1	Propuesta 2
Contenedores agregados	1 (log-api)	3 (watchdog-1 a watchdog-3)
Límites de recursos	1 CPU y 4 GB, iguales a los de la API	Sin límites
Conectores nuevos	TCP de la API al nodo de logs; HTTP de nginx al nodo de logs	UDP de la API a los watchdogs; UDP entre watchdogs (elección y heartbeats); API de Docker
Dependencias de despliegue	La API no atiende requests hasta conectarse con el nodo de logs	Acceso al socket de Docker del host
Uso de CPU de la API	14.2 % en estrés (el base satura el núcleo)	Equivalente al base (9.68 % frente a 9.28 % en carga)
QA que mejora	Performance y disponibilidad ante la degradación del log	Disponibilidad ante caídas del proceso log

En comparación con la propuesta 1 (Sección 4.1.7), la propuesta 2 agrega más contenedores y conectores, pero reserva menos recursos de forma explícita, ya que sus contenedores no declaran límites. La propuesta 1 duplica los recursos reservados, pero libera a la API: su uso de CPU cae del núcleo saturado al 14.2 % en el escenario de estrés. La propuesta 2, en cambio, no cambia el consumo de la API, ya que no busca mejorar la performance sino acotar el tiempo fuera de servicio.

5.2. Pruebas de carga

Las pruebas de carga utilizan la misma configuración que en el sistema base (Sección 3.1), de modo que los resultados sean comparables. Con esta carga el sistema base no presenta fallas ni caídas, por lo que el watchdog no debería intervenir y el escenario permite aislar el costo de la táctica en condiciones normales.

Ambos sistemas completaron todas las requests sin fallas, con un promedio de 394 requests completadas por ventana de 10s en la propuesta 2 y de 397 en el sistema base. Los tiempos de respuesta son prácticamente idénticos: el P95 de /exchange medido en el servidor es de 719ms frente a 718ms, y, desde el cliente, el P95 es de 712ms en ambos casos y el P99 de 761ms frente a 759ms. El uso de recursos de la API también es equivalente, con un promedio de CPU del 9.68 % frente a 9.28 % y de memoria del 1.26 % frente a 1.25 %. Esto confirma que el envío de los heartbeats no tiene un costo observable sobre la API (Sección 5.1.3).

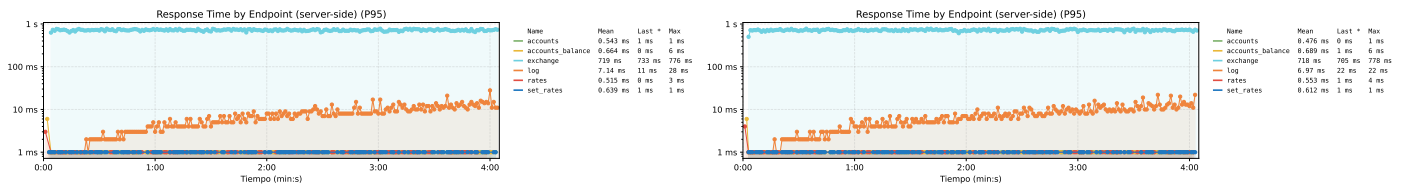


Figura 43. Comparativa de los resultados de latencia por endpoint entre la propuesta 2 (izquierda) y el sistema base (derecha) frente al escenario de carga. Las series de ambos sistemas se superponen: /exchange se mantiene estable en torno a 720ms (P95) y /log crece con el tamaño del log de la misma forma en ambos casos.

Se pueden encontrar los resultados completos en el Apéndice (Sección A.8).

5.3. Pruebas de estrés

Las pruebas de estrés utilizan la misma configuración que en el sistema base (Sección 3.2), de modo que los resultados sean comparables. En este escenario el sistema se degrada por sobrecarga, pero el proceso no termina, por lo que esperamos que el watchdog no intervenga.

Se pueden encontrar los resultados completos en el Apéndice (Sección A.9).

5.3.1. Disponibilidad

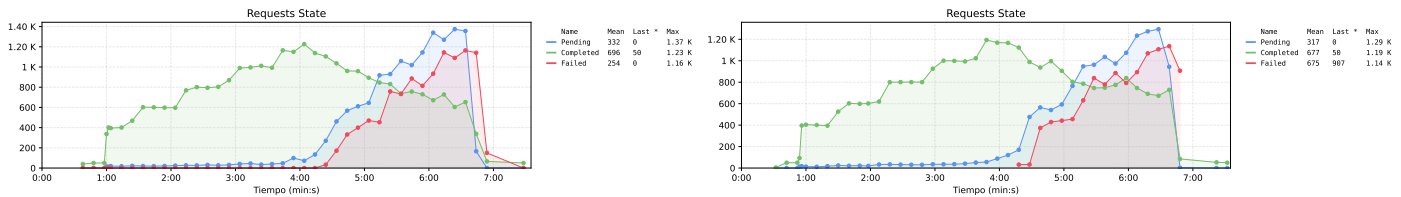


Figura 44. Comparativa de la disponibilidad ofrecida por la propuesta 2 (izquierda) y el sistema base (derecha) frente a un escenario de estrés. Ambos sistemas alcanzan su punto de quiebre en el mismo momento, alrededor del minuto 4:30, y presentan picos de fallas similares (1.16K frente a 1.14K por ventana). El promedio de respuestas satisfactorias por ventana es prácticamente el mismo (696 frente a 677). Como se esperaba, la táctica no mejora la disponibilidad frente a la sobrecarga.

5.3.2. Latencia

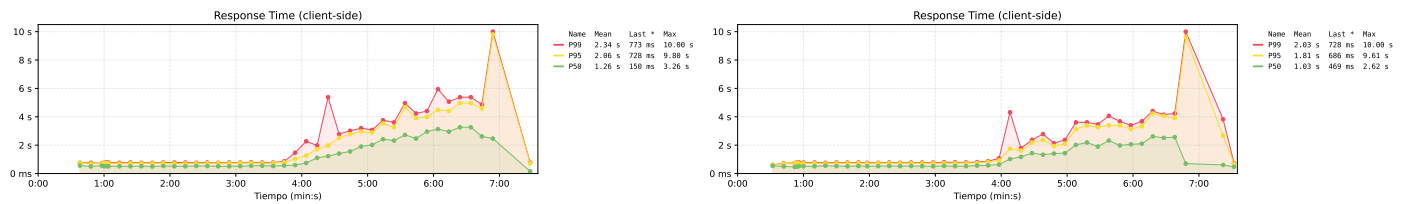


Figura 45. Comparativa de los resultados de latencia entre la propuesta 2 (izquierda) y el sistema base (derecha) frente a un escenario de estrés. Ambos se mantienen estables hasta el minuto 4 y luego se degradan de la misma forma. La propuesta 2 muestra un P50 promedio mayor (1.26s frente a 1.03s), pero consideramos que la diferencia no es atribuible a la táctica, ya que el único trabajo agregado a la API es el envío de 3 datagramas UDP/s y el uso de CPU es equivalente en ambos casos (Figura 47).

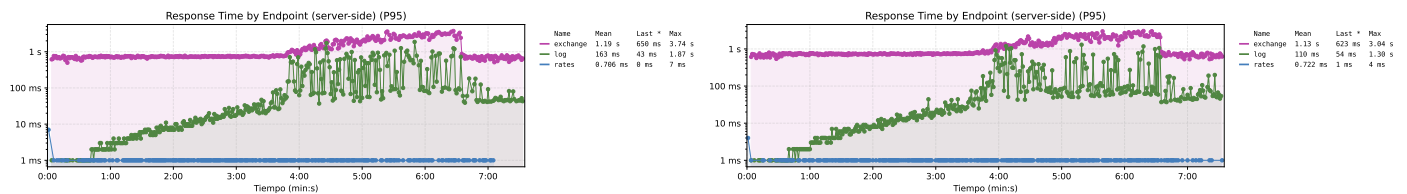


Figura 46. Comparativa de los resultados de latencia por endpoint entre la propuesta 2 (izquierda) y el sistema base (derecha) frente a un escenario de estrés. Se reproduce el mismo comportamiento del sistema base: el crecimiento del log degrada /log y arrastra a /exchange a partir del minuto 4. La propuesta 2 no aísla esta degradación, lo que es consistente con el objetivo de la táctica.

5.3.3. Uso de recursos

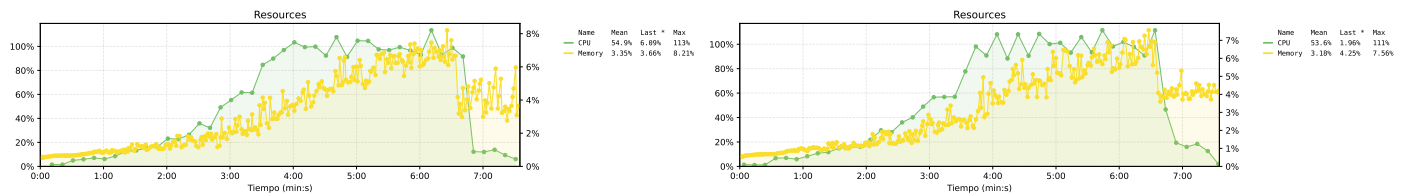


Figura 47. Comparativa de los resultados del uso de recursos entre la propuesta 2 (izquierda) y el sistema base (derecha) frente a un escenario de estrés. El uso de recursos de la API es equivalente en ambos casos (54.9 % frente a 53.6 % de CPU promedio). El uso de CPU nunca cae a cero durante la prueba, lo que indica que el watchdog no reinició la API: el heartbeat siguió llegando a pesar de la saturación.

5.3.4. Volumen operado por moneda

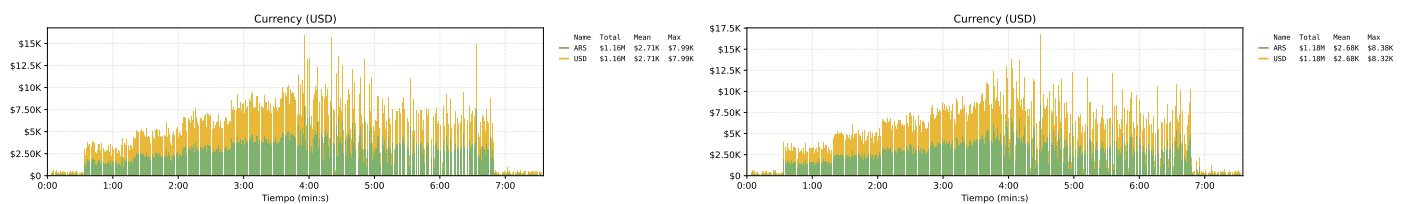


Figura 48. Comparativa de los resultados de volumen operado en cada moneda entre la propuesta 2 (izquierda) y el sistema base (derecha) frente a un escenario de estrés. El volumen total es prácticamente el mismo, 1.16M frente a 1.18M del sistema base (una diferencia del 1.7 %).

5.3.5. Volumen neto operado

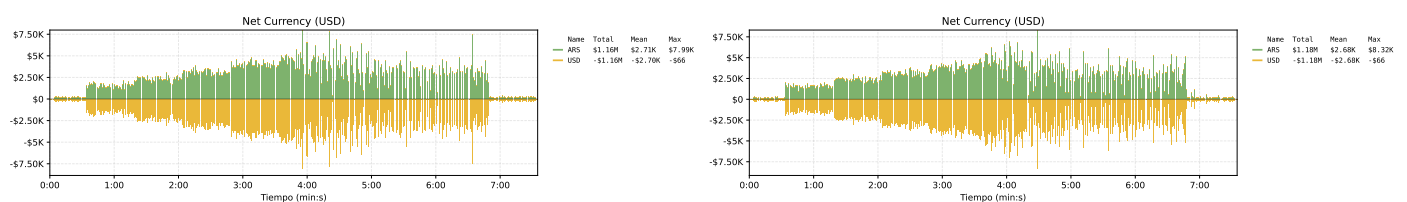


Figura 49. Comparativa de los resultados de volumen neto operado en cada moneda entre la propuesta 2 (izquierda) y el sistema base (derecha) frente a un escenario de estrés. Análogamente al volumen operado, no se observan diferencias significativas.

5.4. Pruebas de resistencia

Las pruebas de resistencia utilizan la misma configuración que en el sistema base (Sección 3.3), de modo que los resultados sean comparables. Al igual que en el escenario de estrés, la degradación se debe al crecimiento del log y no a una caída del proceso.

Se pueden encontrar los resultados completos en el Apéndice (Sección A.10).

5.4.1. Disponibilidad

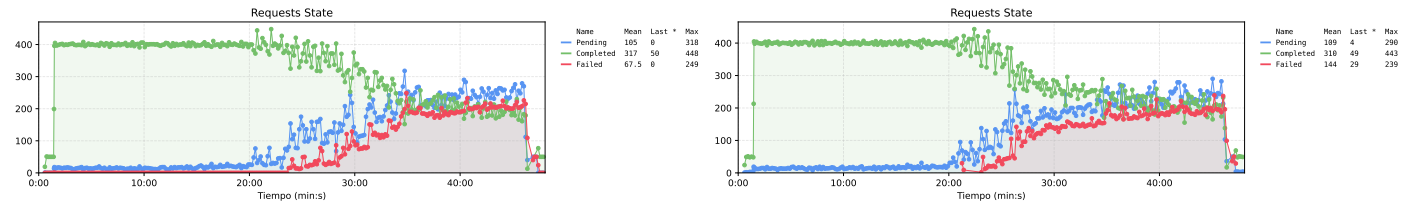


Figura 50. Comparativa de la disponibilidad ofrecida por la propuesta 2 (izquierda) y el sistema base (derecha) frente a un escenario de resistencia. Ambos sistemas comienzan a degradarse cerca del minuto 20 y hacia el minuto 35 alcanzan el mismo nivel de fallas. El promedio de respuestas satisfactorias por ventana es prácticamente el mismo (317 frente a 310). La propuesta 2 no mejora la disponibilidad frente a la degradación progresiva.

5.4.2. Latencia

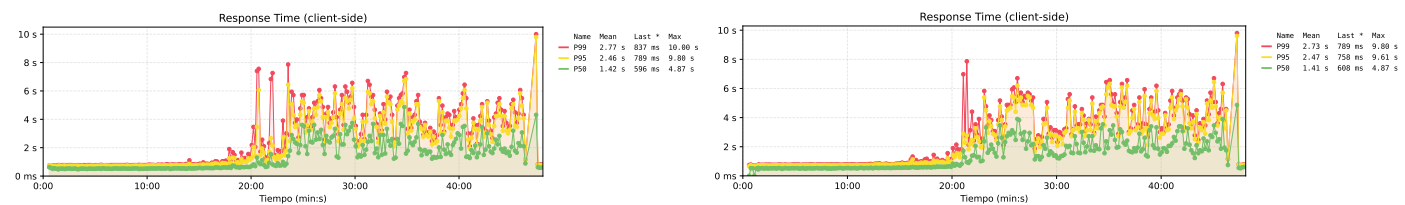


Figura 51. Comparativa de los resultados de latencia entre la propuesta 2 (izquierda) y el sistema base (derecha) frente a un escenario de resistencia. Los resultados son prácticamente idénticos: P50 promedio de 1.42s frente a 1.41s y P95 de 2.46s frente a 2.47s.

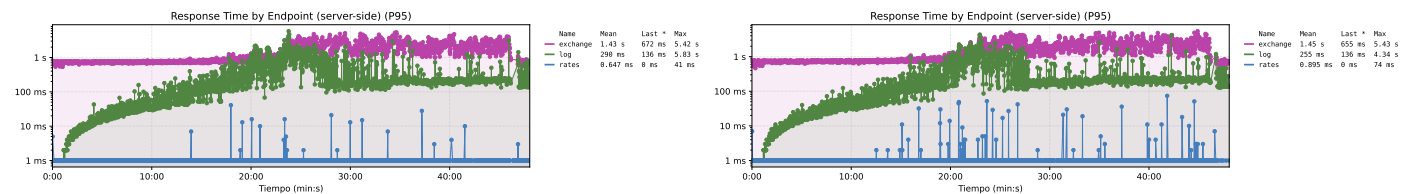


Figura 52. Comparativa de los resultados de latencia por endpoint entre la propuesta 2 (izquierda) y el sistema base (derecha) frente a un escenario de resistencia. En ambos casos el tiempo de respuesta de /log crece durante toda la prueba y arrastra a /exchange, que supera 1s a partir del minuto 20.

5.4.3. Uso de recursos

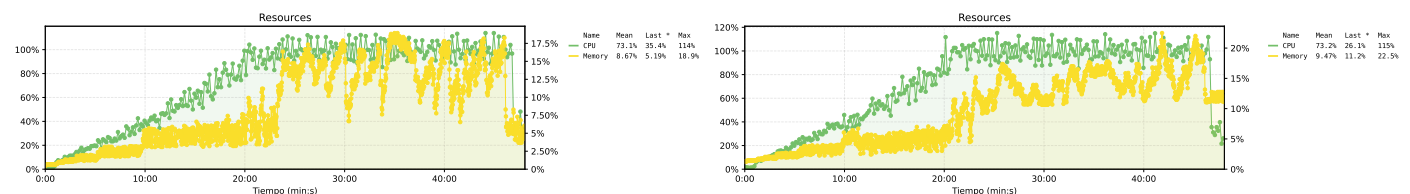


Figura 53. Comparativa de los resultados del uso de recursos entre la propuesta 2 (izquierda) y el sistema base (derecha) frente a un escenario de resistencia. El uso de CPU es equivalente (73.1 % frente a 73.2 % promedio) y alcanza la saturación a partir del minuto 20 en ambos casos. Nuevamente el uso de CPU nunca cae a cero, por lo que no hubo reinicios. Esto evidencia la principal limitación de la táctica: el sistema pasó más de 20m degradado sin que el watchdog lo detectara, ya que el heartbeat solo verifica que el proceso esté vivo.

5.4.4. Volumen operado por moneda

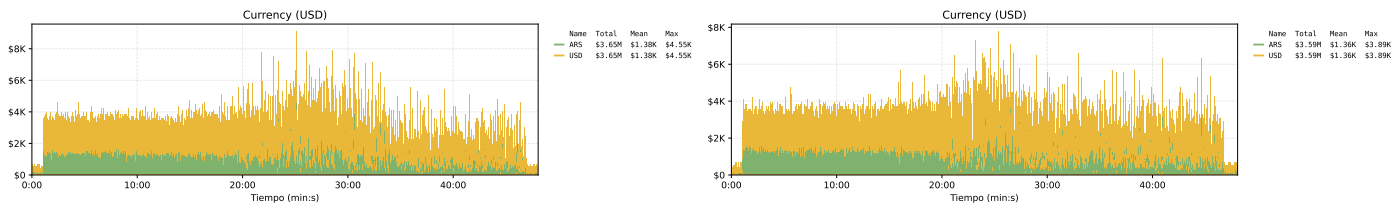


Figura 54. Comparativa de los resultados de volumen operado en cada moneda entre la propuesta 2 (izquierda) y el sistema base (derecha) frente a un escenario de resistencia. El volumen total es prácticamente el mismo, 3.65M frente a 3.59M del sistema base (una diferencia del 1.7 %).

5.4.5. Volumen neto operado

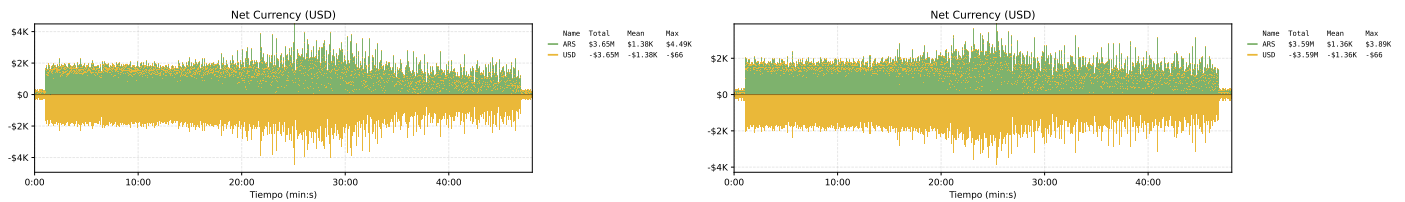


Figura 55. Comparativa de los resultados de volumen neto operado en cada moneda entre la propuesta 2 (izquierda) y el sistema base (derecha) frente a un escenario de resistencia. Análogamente al volumen operado, no se observan diferencias significativas.

5.5. Pruebas de caos

Las pruebas de caos utilizan la misma configuración que en el sistema base (Sección 3.4), de modo que los resultados sean comparables. Este es el escenario en el que esperamos ver el efecto de la táctica.

Se pueden encontrar los resultados completos en el Apéndice (Sección A.11).

5.5.1. Disponibilidad

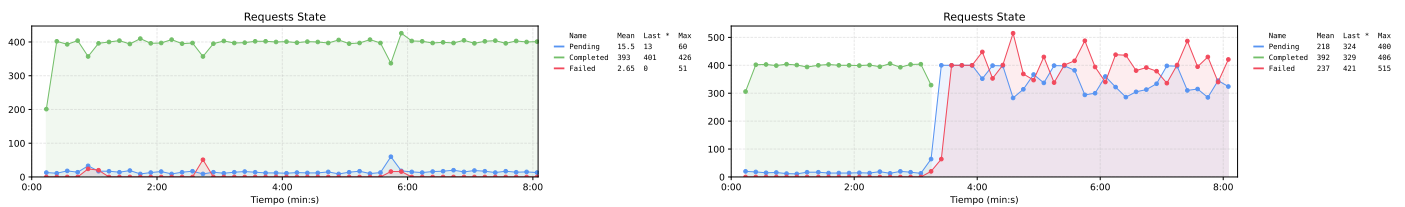


Figura 56. Comparativa de la disponibilidad ofrecida por la propuesta 2 (izquierda) y el sistema base (derecha) frente a un escenario de caos. El sistema base cae en el minuto 3:15 y no vuelve a completar ninguna request durante el resto de la prueba. La propuesta 2 sufre tres caídas pero se recupera de cada una: las fallas se limitan a picos puntuales de a lo sumo 51 requests en una ventana de 10s y el sistema vuelve inmediatamente a completar alrededor de 400 requests por ventana, es decir, las 40 requests/s generadas. Luego de cada caída se observa un pico de requests completadas (hasta 426), correspondiente a las requests acumuladas durante la caída.

5.5.2. Latencia

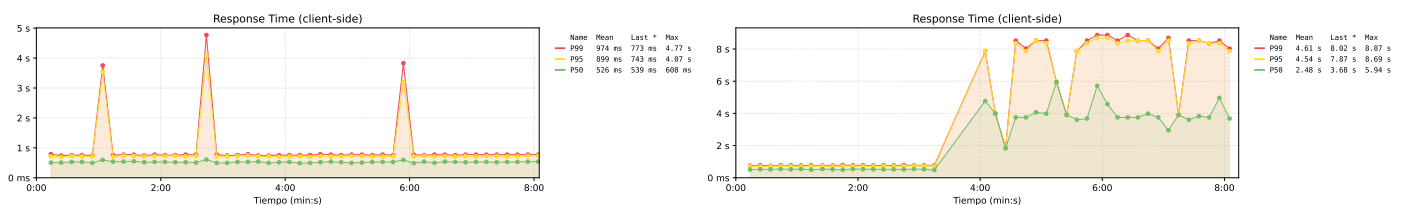


Figura 57. Comparativa de los resultados de latencia entre la propuesta 2 (izquierda) y el sistema base (derecha) frente a un escenario de caos. En el sistema base, luego de la caída, la latencia queda dominada por el timeout de 10s. En la propuesta 2 cada caída se refleja en un pico de latencia de entre 3.2s y 4.8s (P99), consistente con el tiempo de detección configurado (3s sin heartbeat más hasta 1s del período de chequeo) sumado al reinicio del contenedor. Fuera de esos picos la latencia se mantiene estable, con un P50 promedio de 526ms.

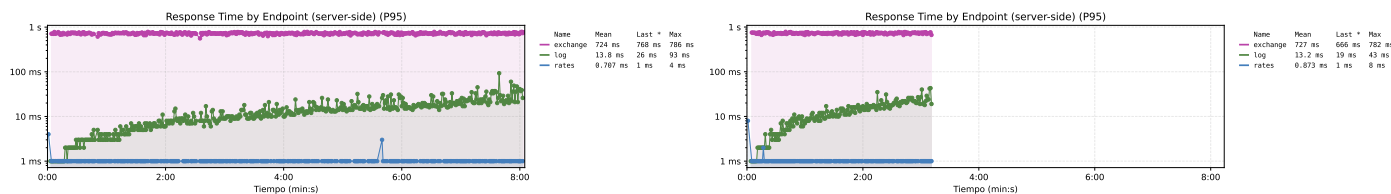


Figura 58. Comparativa de los resultados de latencia por endpoint entre la propuesta 2 (izquierda) y el sistema base (derecha) frente a un escenario de caos. A diferencia del sistema base, en la propuesta 2 las métricas del servidor se siguen reportando durante toda la prueba, ya que el proceso que las emite vuelve a estar disponible. El tiempo de respuesta de /log sigue creciendo después de cada reinicio en lugar de volver a su valor inicial, lo que evidencia que el estado persistido en disco se conserva entre reinicios.

5.5.3. Uso de recursos

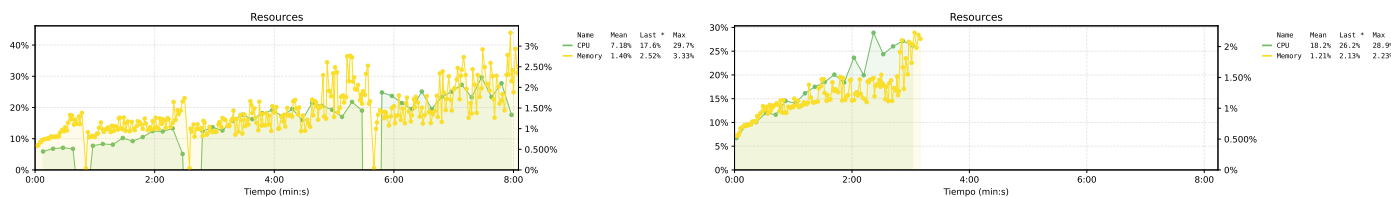


Figura 59. Comparativa de los resultados del uso de recursos entre la propuesta 2 (izquierda) y el sistema base (derecha) frente a un escenario de caos. En el sistema base las métricas dejan de reportarse tras la caída. En la propuesta 2 se observan tres cortes breves en los que CPU y memoria caen a cero, que coinciden con los reinicios del contenedor, y luego el reporte se retoma.

5.5.4. Volumen operado por moneda

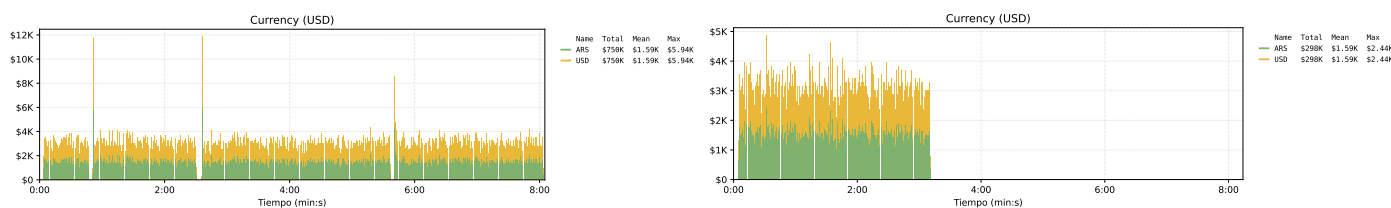


Figura 60. Comparativa de los resultados de volumen operado en cada moneda entre la propuesta 2 (izquierda) y el sistema base (derecha) frente a un escenario de caos. El sistema base se estanca en \$298K por moneda tras la caída, mientras que la propuesta 2 alcanza \$750K por moneda, un aumento del 152 %. Los tres huecos breves en el gráfico corresponden a las caídas, y el pico inmediatamente posterior a cada uno, a las operaciones acumuladas durante la caída.

5.5.5. Volumen neto operado

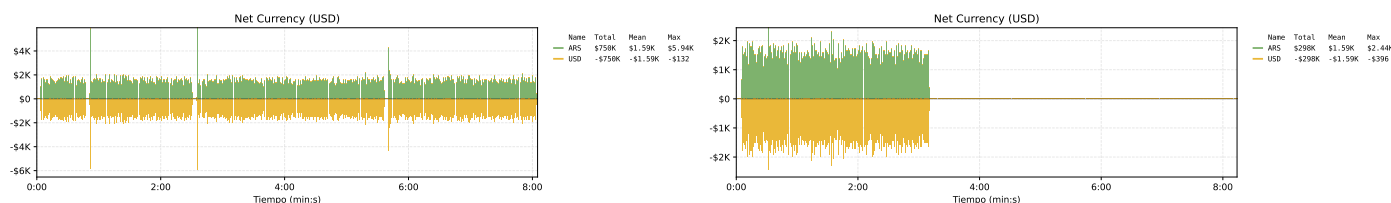


Figura 61. Comparativa de los resultados de volumen neto operado en cada moneda entre la propuesta 2 (izquierda) y el sistema base (derecha) frente a un escenario de caos. Análogamente al volumen operado, en la propuesta 2 el volumen neto continúa creciendo durante toda la prueba en lugar de estancarse tras la primera caída.

5.6. Resultados y conclusiones

La propuesta 2 cumple su objetivo frente a fallas que terminan el proceso: en el escenario de caos, el sistema pasa de quedar fuera de servicio de forma permanente a recuperarse en pocos segundos, y el volumen operado aumenta un 152 % (Figura 60). El tiempo de recuperación observado es consistente con los parámetros configurados (Figura 57).

Por otro lado, los escenarios de estrés y resistencia no muestran diferencias frente al sistema base. Esto es esperable, ya que el heartbeat verifica que el proceso esté vivo y no que responda en un tiempo aceptable: el sistema puede estar degradado durante más de 20m sin que el watchdog intervenga (Figura 53). Para cubrir este caso sería necesario un chequeo de salud a nivel de aplicación (por ejemplo, medir el tiempo de respuesta de un endpoint), o bien atacar la causa de la degradación como lo hace la propuesta 1 (Sección 4). Ambas propuestas son complementarias.

Los costos de las tácticas son una arquitectura más compleja, con un algoritmo distribuido y tres componentes nuevos (Sección 5.1.7), y un impacto negativo en la seguridad por el acceso de los watchdogs al socket de Docker. Además, la propuesta no evita la pérdida de las

operaciones en curso ni de las que no se llegaron a persistir, y no reemplaza la validación de las requests, que sigue siendo necesaria para eliminar la causa de la falla.

6. PROPUESTA 3

La propuesta 2 (Sección 5) acota el tiempo que el servicio permanece caído, pero no evita la caída: la request inválida sigue terminando el proceso cada vez que llega (Figura 56). Además, ninguna de las propuestas anteriores resuelve los problemas de correctitud de los saldos identificados en el análisis del código (Sección 2.3.4). La propuesta 3 ataca la causa de estas fallas en lugar de sus efectos.

Para ello aplicamos las siguientes tácticas:

- **Validación de entradas (seguridad, prevención de fallas):** la API valida tipos, rangos y monedas soportadas antes de procesar cada request, y rechaza con HTTP 400 las que no cumplen. Se reemplaza la validación por presencia del sistema base (Código 7).
- **Manejo de excepciones (disponibilidad):** los handlers capturan los errores no previstos y responden HTTP 500 sin terminar el proceso. Además, los rechazos de negocio (por ejemplo, falta de fondos) pasan a informarse como errores del cliente (4xx) en lugar de 500.
- **Reserva de saldo (confiabilidad):** el saldo de la cuenta interna se descuenta antes de iniciar las transferencias y se devuelve si alguna falla. De esta forma, la verificación y el descuento ocurren sin un `await` intermedio, eliminando la condición de carrera del Código 3.

Al igual que la propuesta 2, esta propuesta se implementó sobre el sistema base, de modo que los resultados solo reflejan el efecto de estas tácticas y sean comparables contra el sistema base.

Esta propuesta no altera la arquitectura: no agrega componentes ni conectores, por lo que el diagrama Components & Connectors es el mismo que el del sistema base (Figura 1). Los cambios se limitan al código de la API.

La validación se concentra en un módulo nuevo, `validation.js`, con una función por endpoint que retorna el primer error encontrado o `null` si la request es válida. Una moneda se considera soportada solo si existe una cuenta interna en ella, y un par solo si existe su tasa. Las cuentas de los clientes pertenecen al servicio de transferencias, por lo que de ellas solo puede validarse el formato.

```

1 // app/validation.js (propuesta 3)
2 export function validateExchangeRequest(body) {
3   const { baseCurrency, counterCurrency, baseAccountId,
4         counterAccountId, baseAmount } = body ?? {};
5
6   const currencyError =
7     validateCurrency("baseCurrency", baseCurrency) ??
8     validateCurrency("counterCurrency", counterCurrency);
9   if (currencyError) {
10     return currencyError;
11   }
12   if (baseCurrency == counterCurrency) {
13     return "baseCurrency and counterCurrency must be different";
14   }
15   if (!isPositiveNumber(getRates()[baseCurrency]?.[counterCurrency])) {
16     return `No exchange rate for ${baseCurrency}/${counterCurrency}`;
17   }
18   // ... validacion de baseAccountId y counterAccountId
19   if (!isPositiveNumber(baseAmount)) {
20     return "baseAmount must be a positive number";
21   }
22   return null;
23 }
24
25 // app/app.js (propuesta 3)
26 const validationError = validateExchangeRequest(req.body);
27 if (validationError) {
28   return res.status(400).json({ error: "Malformed request", obs: validationError });
29 }

```

Código 9. Validación de `/exchange` en `app/validation.js` y su uso en `app/app.js`.

El handler de `/exchange` captura los errores y los deriva a un manejador global, ya que Express 4 no captura los rechazos de promesas. El manejador responde HTTP 500 sin terminar el proceso. Los rechazos de negocio se informan con un código acorde: 409 cuando la cuenta interna no tiene fondos y 502 cuando falla el servicio de transferencias.

```

1 // app/app.js (propuesta 3)
2 app.post("/exchange", async (req, res, next) => {
3   // ... validacion
4   try {
5     const exchangeResult = await exchange({ ...req.body });
6     if (exchangeResult.ok) {
7       res.status(200).json(exchangeResult);
8     } else {
9       res.status(REJECTION_STATUS[exchangeResult.obs] ?? 500).json(exchangeResult);
10    }
11  } catch (err) {
12    next(err);
13  }
14 });
15
16 app.use((err, req, res, next) => {
17   if (err.status >= 400 && err.status < 500) {
18     return res.status(err.status).json({ error: "Malformed request" });
19   }
20   console.error(`Unexpected error on ${req.method} ${req.path}:`, err);
21   res.status(500).json({ error: "Internal server error" });
22 });

```

Código 10. Manejo de excepciones en app/app.js.

Finalmente, la reserva de saldo mueve el descuento de la cuenta interna al momento de la verificación. Como Node.js ejecuta el código de la aplicación en un único hilo, la verificación y el descuento no pueden intercalarse con otra request al no existir un `await` entre ambos, por lo que no es necesario un lock. Si la operación no se completa, ya sea por un rechazo o por una excepción, el bloque `finally` libera la reserva.

```

1 // app/exchange.js (propuesta 3)
2 if (counterAccount.balance >= counterAmount) {
3   counterAccount.balance -= counterAmount; // reserva, sin await intermedio
4
5   let transferred = false;
6   try {
7     transferred = await transferFunds(/* ... */);
8   } finally {
9     if (!transferred) {
10      counterAccount.balance += counterAmount; // libera la reserva
11    }
12  }
13
14   if (transferred) {
15     baseAccount.balance += baseAmount;
16     exchangeResult.ok = true;
17     exchangeResult.counterAmount = counterAmount;
18   }
19 }

```

Código 11. Reserva de saldo en app/exchange.js. Comparar con el Código 3.

6.1. Impacto en los QA

6.1.1. Disponibilidad

Es uno de los QA que la propuesta busca mejorar. A diferencia de la propuesta 2, que detecta la falla y se recupera, esta propuesta la previene: una request inválida es rechazada antes de acceder a tasas o cuentas inexistentes, y un error no previsto queda contenido en la request que lo produjo en lugar de terminar el proceso.

TODO: Completar con la evidencia del escenario de caos (Sección 6.4).

La táctica solo cubre las fallas originadas por el procesamiento de una request. Una terminación del proceso por otras causas (falta de memoria, error del runtime o del host) sigue dejando el servicio caído, por lo que es complementaria con la propuesta 2.

6.1.2. Confiabilidad

Es el otro QA que la propuesta busca mejorar. La reserva de saldo garantiza que dos operaciones concurrentes no puedan aprobarse con el mismo saldo, por lo que la cuenta interna no puede quedar en negativo.

En el escenario de concurrencia (Sección 6.5), diez cambios concurrentes que el sistema base aprobaba en su totalidad, dejando la cuenta interna en -700 USD, se resuelven en la propuesta 3 con un único cambio aprobado y la cuenta en 19.99 USD (Tabla 4).

Como contrapartida, mientras una operación está en curso el saldo reservado no está disponible para otras, por lo que una operación que luego falla puede provocar el rechazo de otra que sí tenía fondos. Consideramos preferible rechazar una operación válida antes que aprobar una que deje un saldo negativo.

La propuesta no resuelve la durabilidad diferida ni la escritura no atómica del estado (Código 5), ni el uso de Number para los importes (Código 6), que sigue produciendo residuos como el mostrado en la Sección 1.2. Ambos problemas requieren cambiar la representación y la persistencia del estado, lo que recomendamos abordar junto con la migración a una base de datos externa.

6.1.3. Seguridad

La validación impide que un consumidor que invoque directamente la API altere los saldos con importes negativos o provoque una denegación de servicio con una request malformada, que en el sistema base alcanzaba para tirar el servicio (Sección 3.4).

Para evidenciarlo, ejecutamos las mismas requests inválidas contra ambos sistemas, partiendo del estado inicial de `app/state/` en cada caso (Tabla 3). En el sistema base, un importe negativo ejecuta el cambio en sentido inverso (la cuenta interna en ARS entrega 100000 y la de USD recibe 66), y un importe enviado como texto supera la validación y convierte el saldo en un string por concatenación ("`120000000100000`"), corrompiendo la cuenta para todas las operaciones siguientes. En la propuesta 3 todas las requests se rechazan con HTTP 400 sin modificar el estado.

Cuadro 3. Resultado de requests inválidas sobre el sistema base y la propuesta 3.

Request	Sistema base	Propuesta 3
<code>baseAmount: -100000</code>	200; ARS interno 120000000 → 119900000, USD interno 60000 → 60066	400; saldos sin cambios
<code>baseAmount: "100000"</code>	200; saldo ARS interno pasa a ser el string " <code>120000000100000</code> "	400; saldos sin cambios
Misma moneda (USD/USD)	500	400
<code>PUT /rates con rate: -1</code>	200; la tasa USD/ARS queda en -1	400; tasa sin cambios
Moneda xxx	Sin respuesta; el proceso termina y no responde requests posteriores	400; el proceso sigue respondiendo

TODO: Adjuntar captura de la ejecución (terminal o Postman) en el entorno de pruebas.

6.1.4. Performance

La validación agrega comprobaciones en memoria de costo constante por request, que consideramos despreciables frente a los 400 a 800ms de las transferencias. La reserva de saldo no agrega esperas, ya que no utiliza un lock. No se espera que la propuesta mejore la degradación producida por el log, que se mantiene igual que en el sistema base.

TODO: Confirmar con las pruebas de estrés y resistencia (Secciones 6.2 y 6.3).

6.1.5. Modificabilidad

La validación queda centralizada en `validation.js` y separada de la lógica de cambio, por lo que modificar una regla no requiere tocar `exchange.js`. Las monedas soportadas no están fijas en el código sino que se derivan de las cuentas internas y las tasas existentes, de modo que agregar una moneda no requiere modificar la validación. Por otro lado, la validación depende de `exchange.js` para consultar cuentas y tasas, y los códigos HTTP de los rechazos se asocian al texto del motivo (REJECTIONS), por lo que agregar un nuevo motivo de rechazo requiere actualizar también `app.js`.

La reserva de saldo, en cambio, depende de que Node.js ejecute el código en un único hilo. Si el estado se mueve a una base de datos externa o se replica la API (como requeriría escalar horizontalmente), la verificación y el descuento dejan de ser atómicos y deben reemplazarse por una operación atómica o una transacción del motor.

6.1.6. Testeabilidad

Los validadores son funciones que reciben el cuerpo de la request y retornan un mensaje, por lo que pueden probarse de forma aislada sin levantar el servidor ni esperar las transferencias. Además, los códigos de respuesta permiten distinguir en las pruebas, y en las métricas, los errores del cliente (4xx) de los rechazos por falta de fondos (409) y de las fallas del servidor o sus dependencias (5xx). En el sistema base todos se informaban como 500. La propuesta no agrega pruebas automatizadas, por lo que la ausencia señalada en la Sección 2.2.3 se mantiene.

6.1.7. Simplicidad

Es la propuesta de menor complejidad: no agrega componentes, conectores ni dependencias de despliegue, y los cambios se concentran en la API. El costo es un aumento del código de la API y de la cantidad de casos que deben contemplarse al modificarla.

6.2. Pruebas de estrés

Las pruebas de estrés comparten la misma configuración con el sistema base de tal forma que los resultados sean comparables, ver Sección 3.2. En este escenario no hay requests inválidas, por lo que esperamos que la propuesta no presente diferencias frente al sistema base.

Se pueden encontrar los resultados completos en el Apéndice (Sección A.12).

6.2.1. Latencia

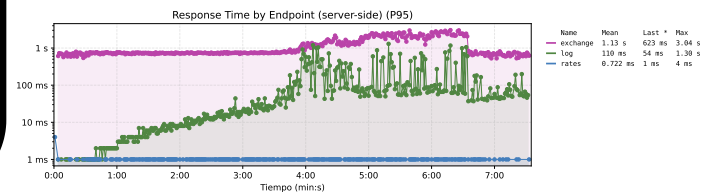
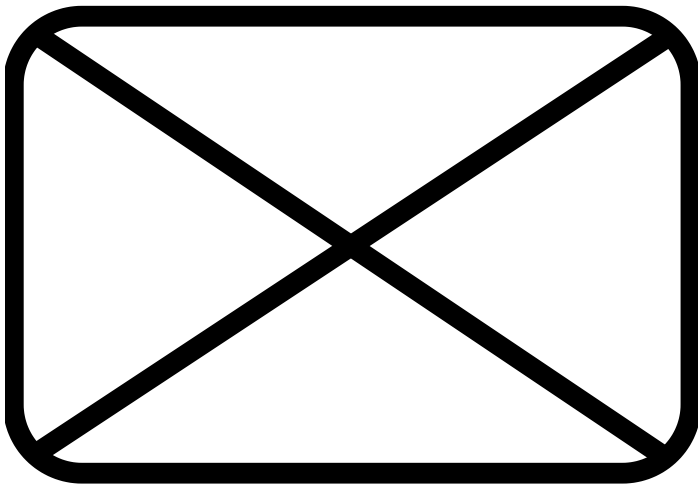


Figura 62. Comparativa de los resultados de latencia por endpoint entre la propuesta 3 (izquierda) y el sistema base (derecha) frente a un escenario de estrés.

TODO: Completar.

6.2.2. Uso de recursos

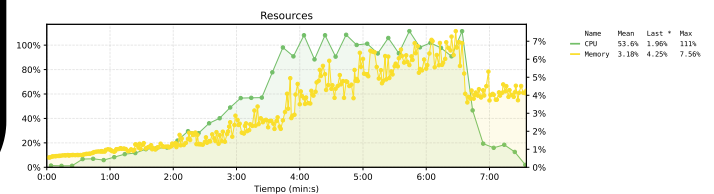
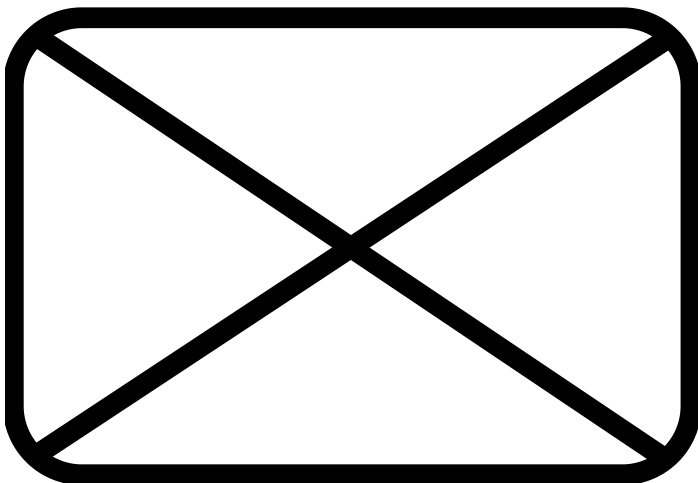


Figura 63. Comparativa de los resultados del uso de recursos entre la propuesta 3 (izquierda) y el sistema base (derecha) frente a un escenario de estrés.

TODO: Completar.

6.3. Pruebas de resistencia

Las pruebas de resistencia comparten la misma configuración con el sistema base de tal forma que los resultados sean comparables, ver Sección 3.3. Al igual que en el escenario de estrés, no esperamos diferencias frente al sistema base.

Se pueden encontrar los resultados completos en el Apéndice (Sección A.13).

6.3.1. Disponibilidad

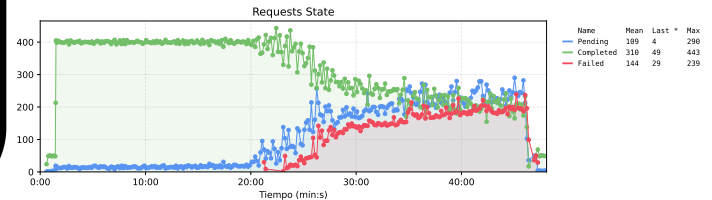
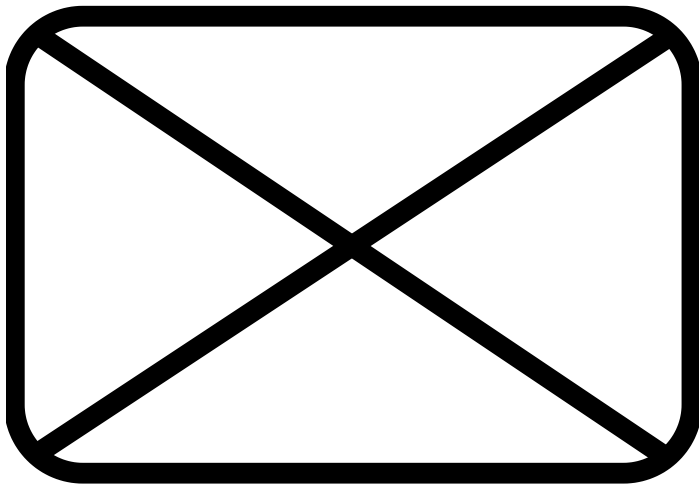


Figura 64. Comparativa de la disponibilidad ofrecida por la propuesta 3 (izquierda) y el sistema base (derecha) frente a un escenario de resistencia.

TODO: Completar.

6.3.2. Latencia

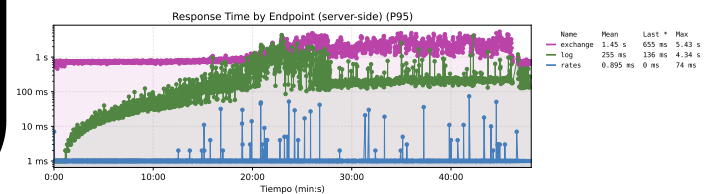
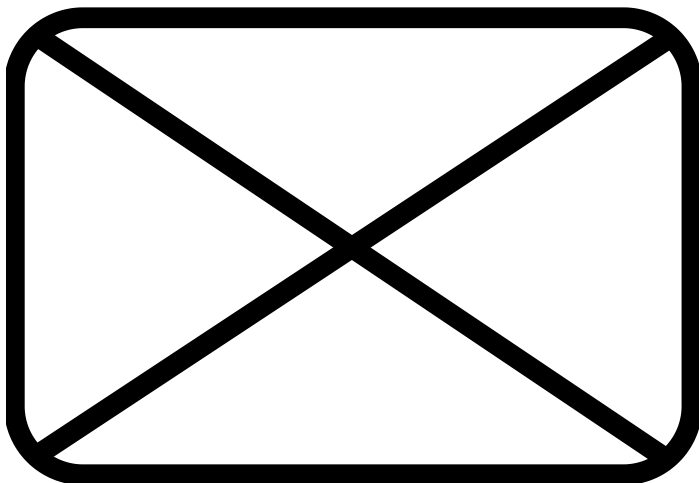


Figura 65. Comparativa de los resultados de latencia por endpoint entre la propuesta 3 (izquierda) y el sistema base (derecha) frente a un escenario de resistencia.

TODO: Completar.

6.4. Pruebas de caos

Las pruebas de caos comparten la misma configuración con el sistema base de tal forma que los resultados sean comparables, ver Sección 3.4. La request inválida utiliza una moneda inexistente (XXX). Este es el escenario en el que esperamos ver el efecto de la validación y del manejo de excepciones: el sistema no debería caer y la request inválida debería rechazarse con HTTP 400.

Se pueden encontrar los resultados completos en el Apéndice (Sección A.14).

6.4.1. Disponibilidad

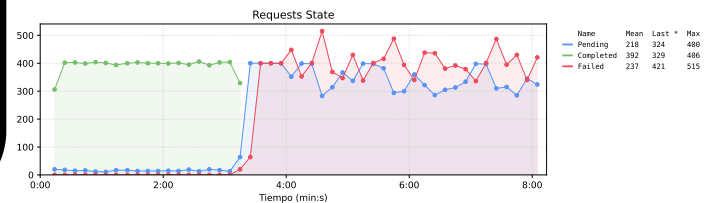
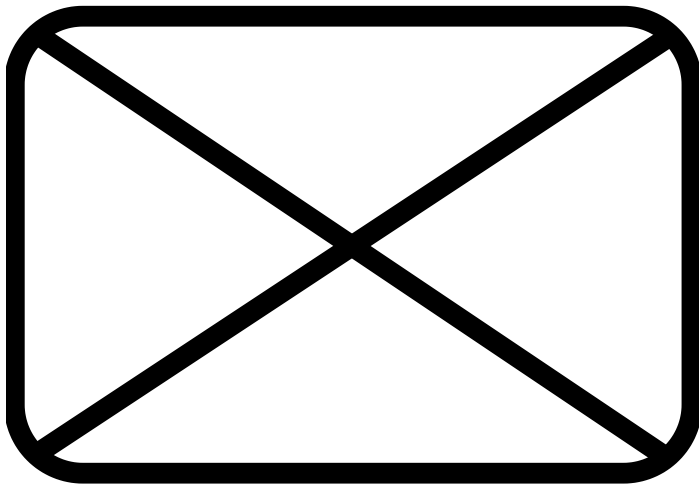


Figura 66. Comparativa de la disponibilidad ofrecida por la propuesta 3 (izquierda) y el sistema base (derecha) frente a un escenario de caos.

TODO: Completar. Comparar también con la propuesta 2 (Figura 56): P2 se recupera de cada caída, P3 no cae.

6.4.2. Latencia

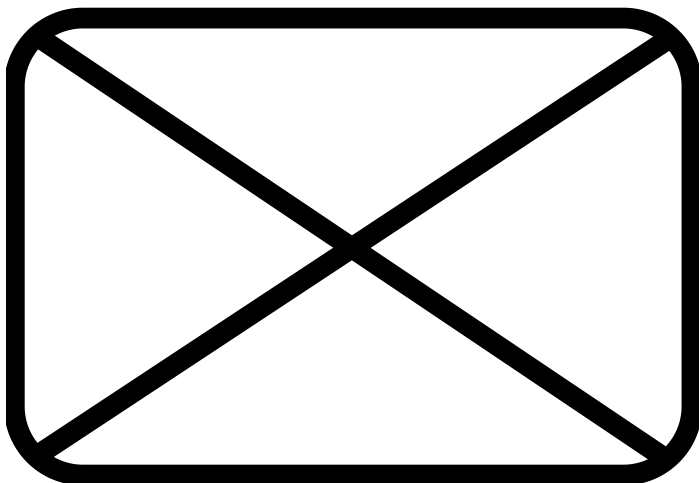


Figura 67. Comparativa de los resultados de latencia entre la propuesta 3 (izquierda) y el sistema base (derecha) frente a un escenario de caos.

TODO: Completar. Esperado: sin picos, a diferencia de la propuesta 2.

6.4.3. Volumen operado por moneda

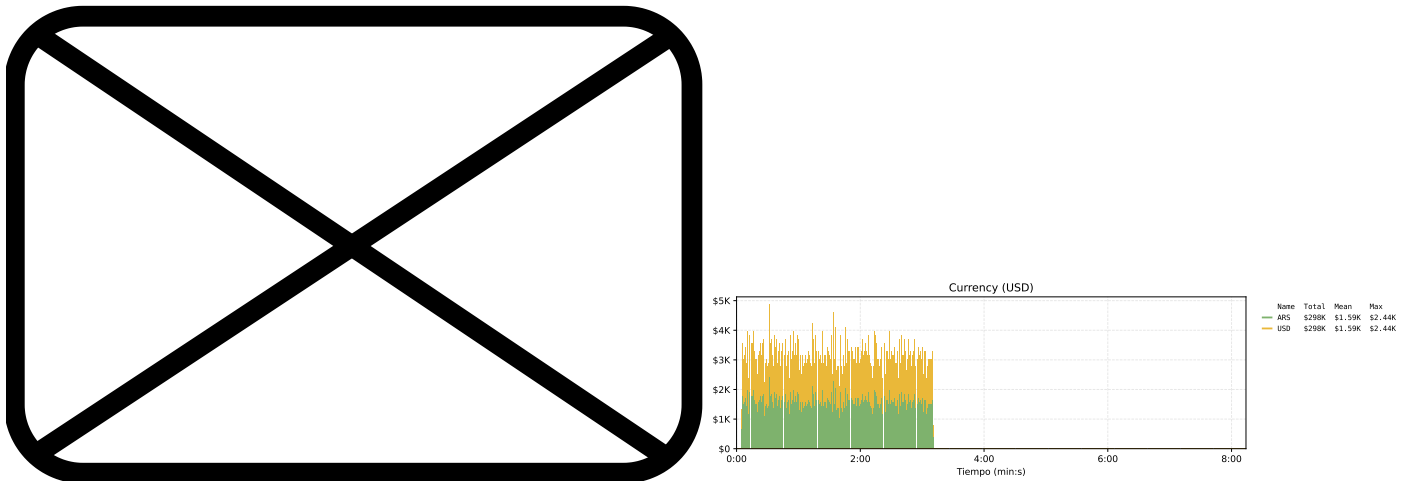


Figura 68. Comparativa de los resultados de volumen operado en cada moneda entre la propuesta 3 (izquierda) y el sistema base (derecha) frente a un escenario de caos.

TODO: Completar con el total y el porcentaje frente al base (\$298K) y a la propuesta 2 (\$750K).

6.5. Pruebas de concurrencia

Este escenario tiene como objetivo evidenciar la condición de carrera identificada en el Código 3, por lo que se ejecuta tanto sobre el sistema base como sobre la propuesta 3. Lo realizamos de dos formas complementarias.

La primera (`perf/race.sh`) es una prueba acotada que permite verificar el saldo exacto: se fija el saldo de la cuenta interna en USD en 100 y se envían N cambios concurrentes de 121213 ARS, equivalentes a unos 80 USD cada uno. Como el saldo alcanza para un solo cambio, el resultado correcto es que se apruebe uno, se rechace el resto y el saldo final sea de unos 20 USD.

La segunda (`perf/race.yaml`) reproduce la situación con carga sostenida para observarla en el dashboard: se fija el saldo de la cuenta interna en USD en 8000 y se envían 40 cambios/s de 100000 ARS (unos 66 USD cada uno) durante 60s. El saldo alcanza para unos 121 cambios de los 2400 enviados. Dado que cada cambio demora entre 400 y 800ms, en todo momento hay entre 16 y 32 cambios en curso que pueden haber verificado el mismo saldo.

6.5.1. Saldo de la cuenta interna

Cuadro 4. Resultado de `perf/race.sh` con un saldo inicial de 100 USD en la cuenta interna.

Sistema	Concurrentes	Respuestas	Saldo final (USD)
Base	2	2×200	-60.00116
Propuesta 3	2	$1 \times 200, 1 \times 409$	19.99942
Base	10	10×200	-700.0058
Propuesta 3	10	$1 \times 200, 9 \times 409$	19.99942

En el sistema base se aprueban todos los cambios y la cuenta interna termina en negativo en proporción a la cantidad de requests concurrentes: arVault entrega dólares que no tiene. En la propuesta 3 se aprueba un único cambio, el resto se rechaza con HTTP 409 y el saldo final es el mismo independientemente de la concurrencia.

TODO: Adjuntar captura de `./race.sh` en el entorno de pruebas para ambos sistemas.

6.5.2. Estado de las requests

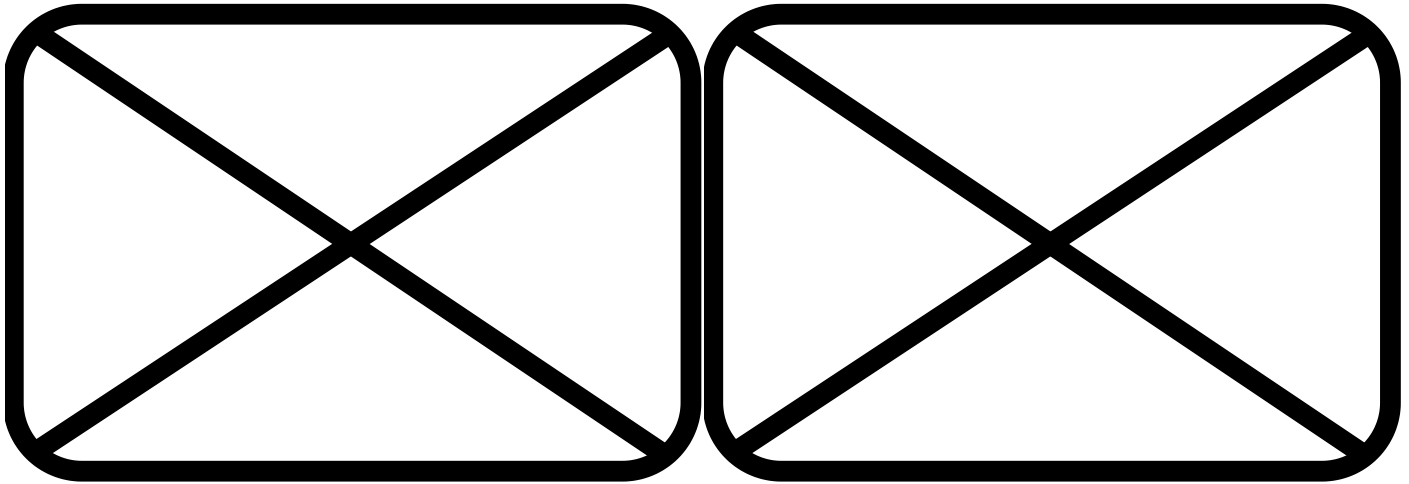


Figura 69. Comparativa del estado de las requests entre la propuesta 3 (izquierda) y el sistema base (derecha) frente al escenario de concurrencia. Notar que el panel clasifica las respuestas 4xx como *Limited*: en la propuesta 3 los cambios sin fondos se informan como 409, mientras que en el sistema base se informaban como 500 (*Failed*).

TODO: Completar. Esperado: en el base se aprueban más de los ~121 cambios que cubre el saldo; en la propuesta 3 las excedentes se rechazan. Complementar con el volumen neto en USD, que en el base debería superar los 8000.

6.6. Resultados y conclusiones

La propuesta 3 ataca la causa de las fallas que las propuestas anteriores no resuelven. La validación de entradas y el manejo de excepciones impiden que una request inválida altere los saldos o termine el proceso (Tabla 3), y la reserva de saldo elimina la condición de carrera: diez cambios concurrentes que en el sistema base dejaban la cuenta interna en -700 USD se resuelven con un único cambio aprobado (Tabla 4).

TODO: Agregar el resultado del escenario de caos (caídas, requests completadas y volumen frente al base y a la propuesta 2).

A diferencia de las propuestas 1 y 2, no agrega componentes, conectores ni recursos, y no se espera un impacto en la performance, ya que solo suma comprobaciones en memoria. **TODO:** Confirmar con estrés y resistencia, o aclarar que no se ejecutaron y por qué.

La propuesta no reemplaza a las anteriores. No resuelve la degradación producida por el log, que sigue arrastrando a /exchange (propuesta 1), ni las caídas del proceso por causas ajenas a las requests (propuesta 2), ni la pérdida de datos por la persistencia diferida o los residuos del punto flotante. Además, la reserva de saldo depende de que el estado viva en un único proceso: al migrarlo a una base de datos externa o replicar la API, la verificación y el descuento deben reemplazarse por una operación atómica del motor. Las tres propuestas son complementarias y, en conjunto, atacan los tres problemas señalados en la Sección 2.4.

7. CONSIDERACIONES

7.1. Organización del repositorio

La rama `main` contiene el código del sistema base. Las ramas `propuesta_1`, `propuesta_2` y `propuesta_3` contienen la implementación de la Propuesta 1, la Propuesta 2 y la Propuesta 3, respectivamente.

7.2. Entorno de pruebas

Todas las pruebas se ejecutaron con el contenedor de la API limitado a 1 CPU y 4 GB de memoria, configurado en `deploy.resources.limits` del `docker-compose.yml`.

7.3. Cambios en `run-scenario.sh`

El script original solo ejecutaba Artillery. Se lo extendió para que en cada corrida:

- Reconstruya y recree los contenedores de la API, nginx y los que agrega cada propuesta.
- Espere a que la API responda antes de iniciar la carga.
- Al finalizar, descargue de Graphite las series de Artillery, cAdvisor, volumen por moneda y tiempos de respuesta por endpoint correspondientes al intervalo de la prueba.

La invocación no cambia (`./run-scenario.sh <escenario><entorno>` desde `perf/`).

Los datos de cada corrida se guardan en `perf/graficos/data/`, en un directorio `<rama>_<escenario>_<timestamp>`.

7.4. Generación de gráficos

Los gráficos del informe se generan con `matplotlib` a partir de los datos exportados por `run-scenario.sh`. Desde `perf/graficos/`:

```
1 python3 -m venv .venv
2 source .venv/bin/activate
3 pip install -r requirements.txt
4 python3 main.py <rama>_<escenario>_<timestamp>
```

Las figuras se generan en PDF en `perf/graficos/figures/`, en un directorio con el mismo nombre, y desde ahí se copian a `latex/figures/`.

8. USO DE HERRAMIENTAS DE IA

En esta sección se detallan los usos específicos que se dieron a herramientas de inteligencia artificial durante la realización del trabajo. No todos los integrantes del grupo utilizaron estas herramientas, ni lo hicieron de la misma manera.

8.1. Conversión del informe a $\text{\LaTeX}$

En el caso de algunos integrantes, que no tenían experiencia previa con $\text{\LaTeX}$, se utilizó IA para convertir sus aportes redactados inicialmente en un documento de Google Docs a una estructura de secciones, figuras, tablas, bloques de código y referencias cruzadas. En estos casos la herramienta se utilizó únicamente para el cambio de formato; el contenido, el análisis y las conclusiones fueron elaborados por los integrantes.

8.2. Librería para generar gráficos a partir de Graphite

Consideramos que adjuntar capturas de pantalla del dashboard podía resultar poco profesional, difícil de leer al no poder hacer zoom y difícil de automatizar (muchas veces por nuevas consideraciones que surgieron en el medio teníamos que agregar alguna métrica o algún cambio al dashboard e implicaba manualmente tomar todas las capturas).

Considerando que teníamos las series de datos en Graphite, cierta experiencia con numpy pandas y pyplot, decidimos recrear los paneles de Grafana directamente desde los datos.

Resultó que imitar los gráficos de Grafana implicaba muchas consideraciones (como por ejemplo cómo tratar los valores nulos, si como ceros o usar la última medición) que complicaron bastante esta idea. En ciertas partes de la librería usamos código generado por IA, consideramos que no era una parte fundamental del TP y no queríamos perder mucho tiempo en eso. Esto se limita al directorio `/perf/graficos`.

Igualmente `./dashboard` contiene todos los gráficos que usamos como referencia. En el apéndice se incluyen igualmente las capturas junto a los renders para que se puedan corroborar.

A. APÉNDICE

A.1. Resultados completos del escenario de carga sobre el sistema base

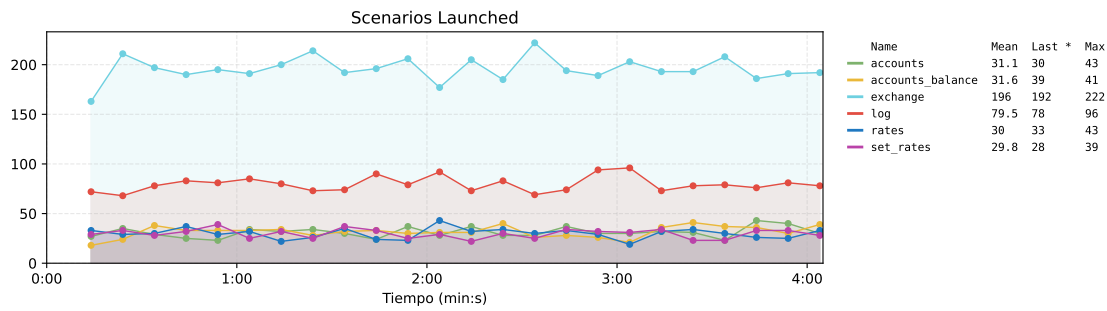


Figura 70. Escenarios del sistema base.

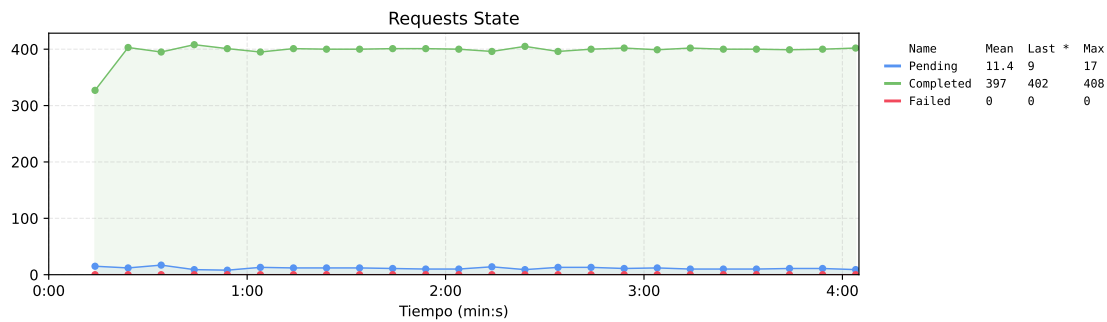


Figura 71. Estado de las requests del sistema base.

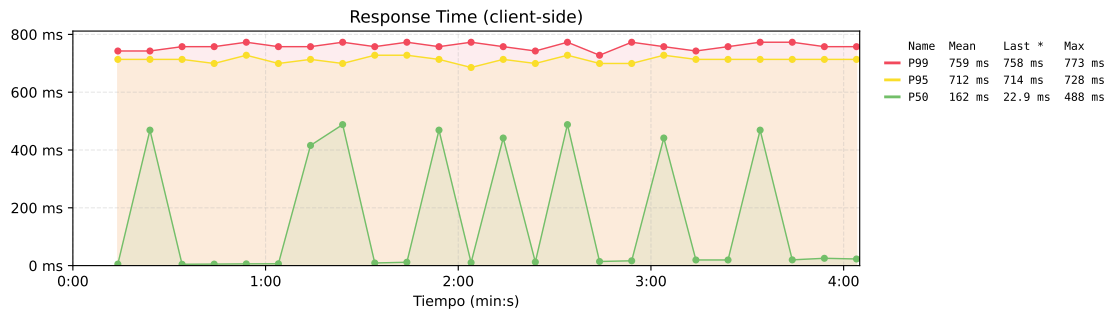


Figura 72. Tiempo de respuesta del sistema base.

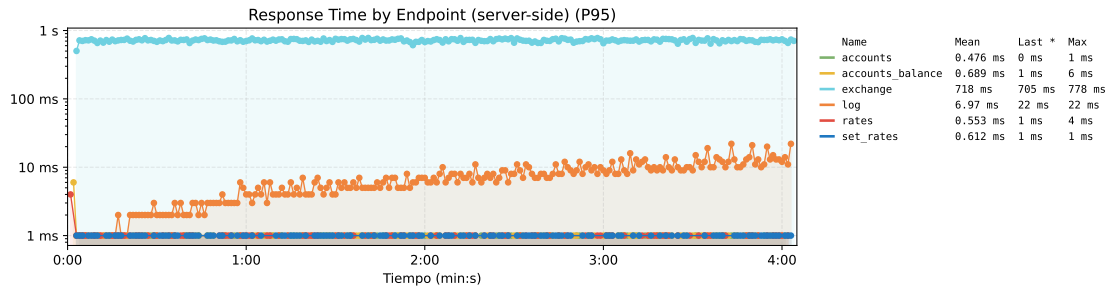


Figura 73. Tiempo de respuesta por endpoint del sistema base. El eje tiene una escala logarítmica.

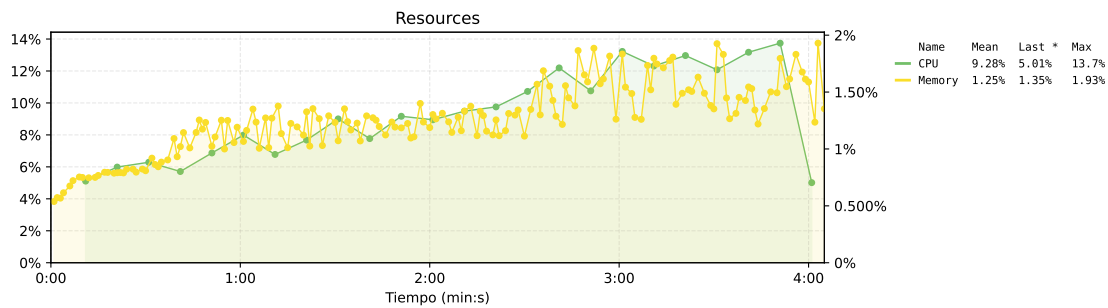


Figura 74. Estado de los recursos del sistema base. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

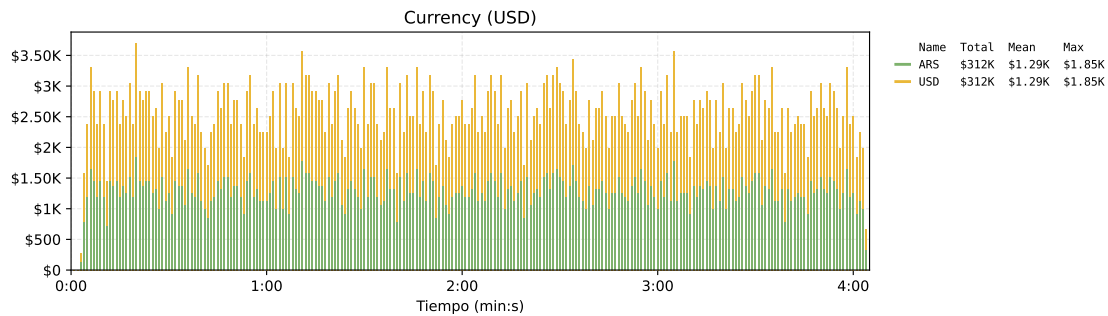


Figura 75. Volumen operado en cada moneda en el sistema base.

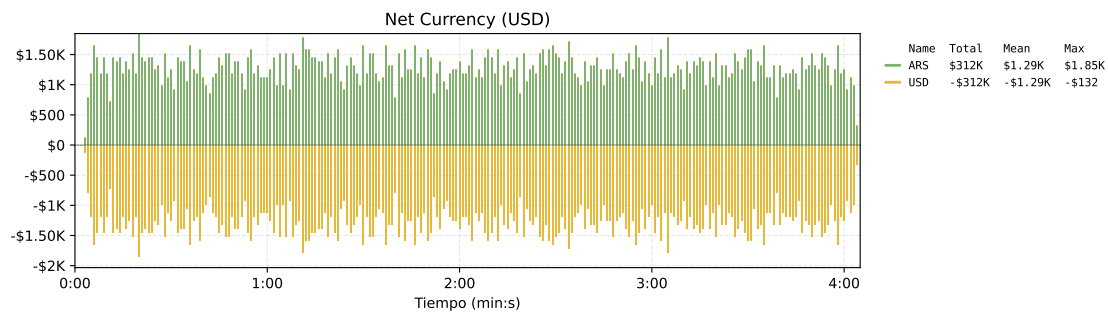


Figura 76. Neto operado en cada moneda en el sistema base.

A.2. Resultados completos del escenario de estrés sobre el sistema base

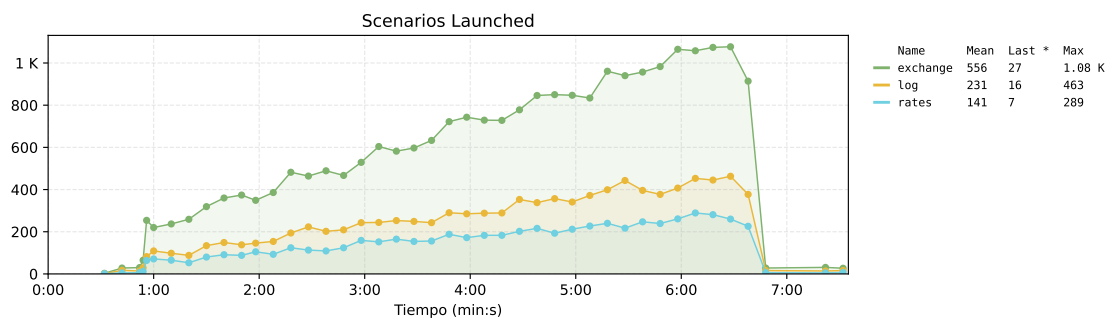


Figura 77. Escenarios del sistema base.

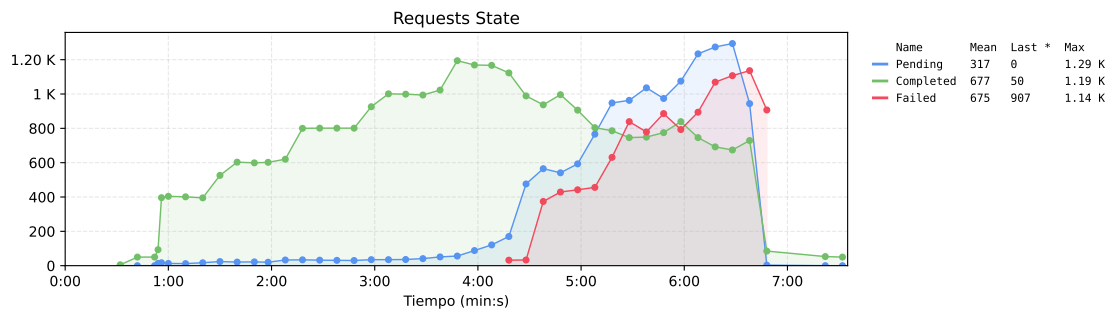


Figura 78. Estado de las requests del sistema base.

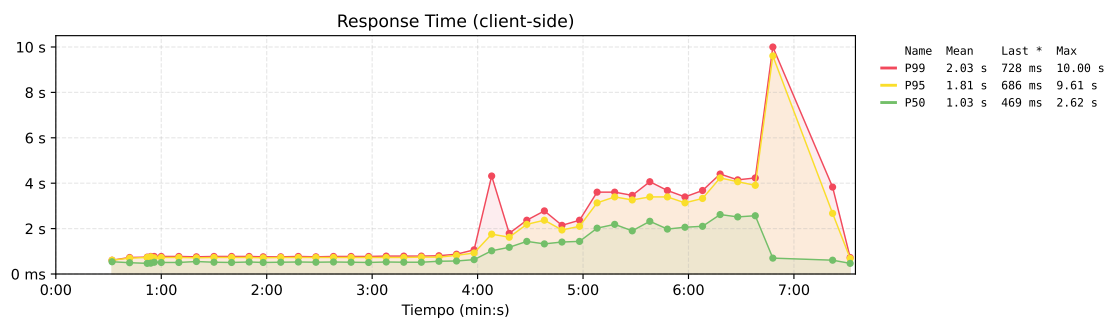


Figura 79. Tiempo de respuesta del sistema base.

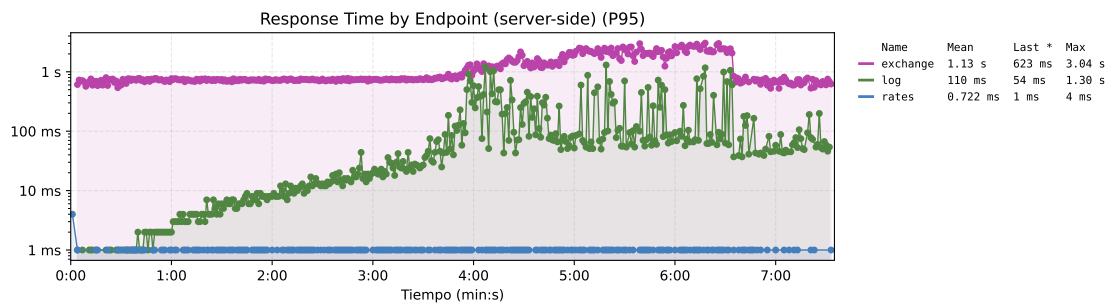


Figura 80. Tiempo de respuesta por endpoint del sistema base. El eje tiene una escala logarítmica.

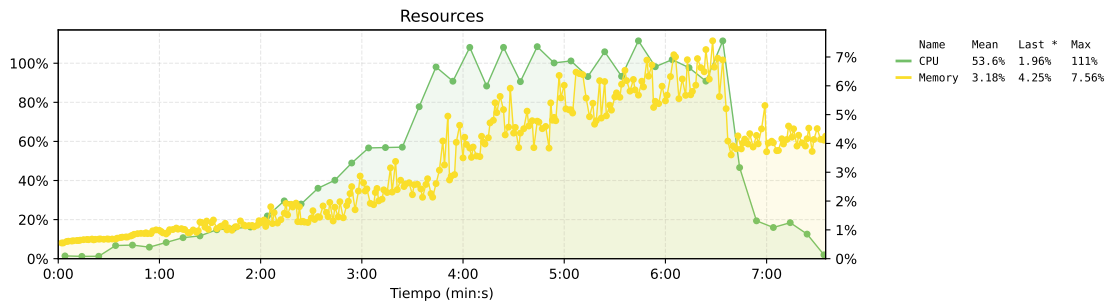


Figura 81. Estado de los recursos del sistema base. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

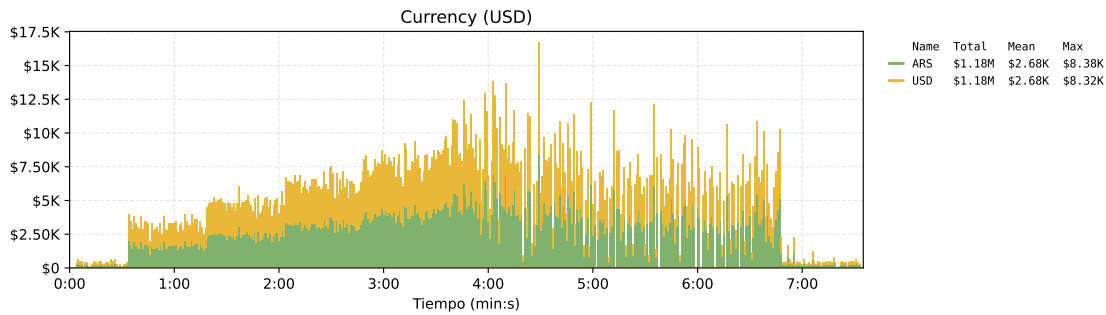


Figura 82. Volumen operado en cada moneda en el sistema base.

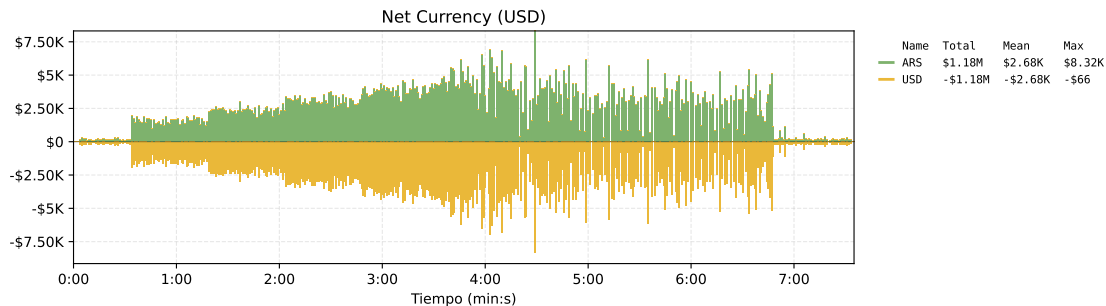


Figura 83. Neto operado en cada moneda en el sistema base.

A.3. Resultados completos del escenario de resistencia sobre el sistema base

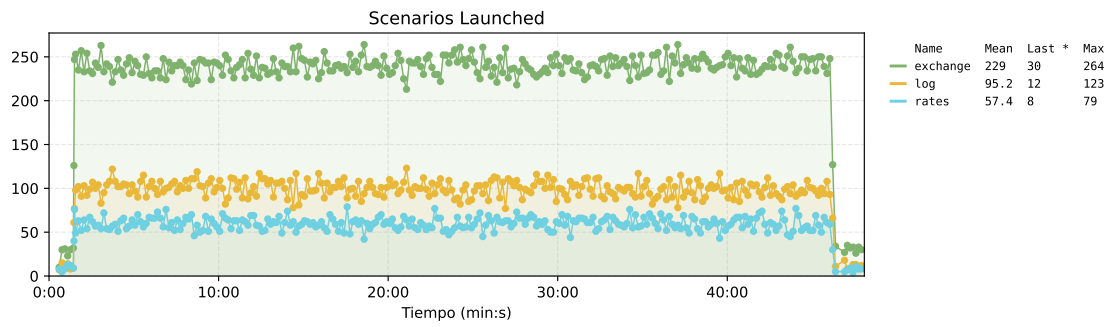


Figura 84. Escenarios del sistema base.

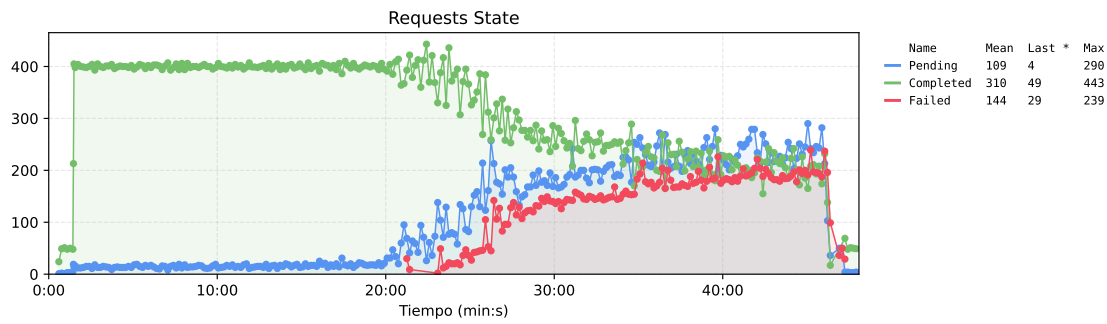


Figura 85. Estado de las requests del sistema base.

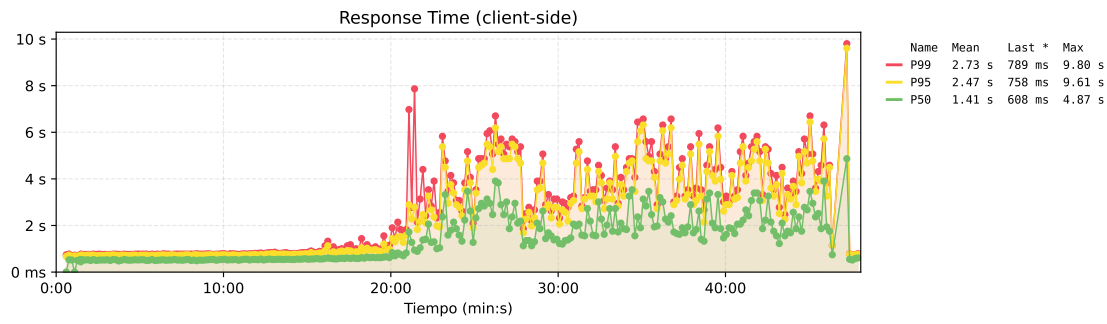


Figura 86. Tiempo de respuesta del sistema base.

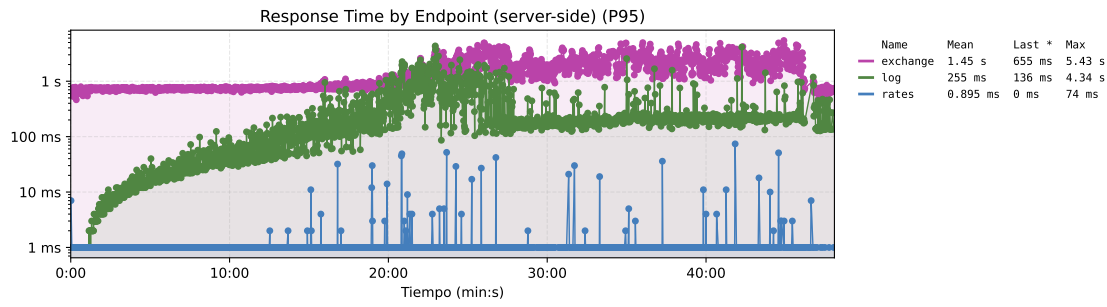


Figura 87. Tiempo de respuesta por endpoint del sistema base. El eje tiene una escala logarítmica.

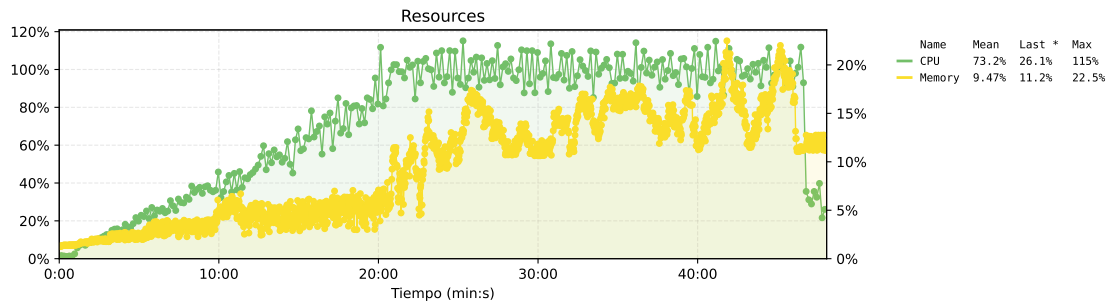


Figura 88. Estado de los recursos del sistema base. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

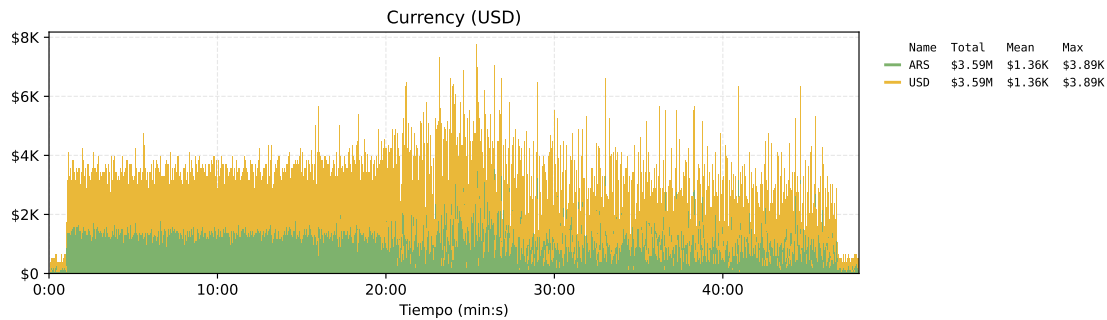


Figura 89. Volumen operado en cada moneda en el sistema base.

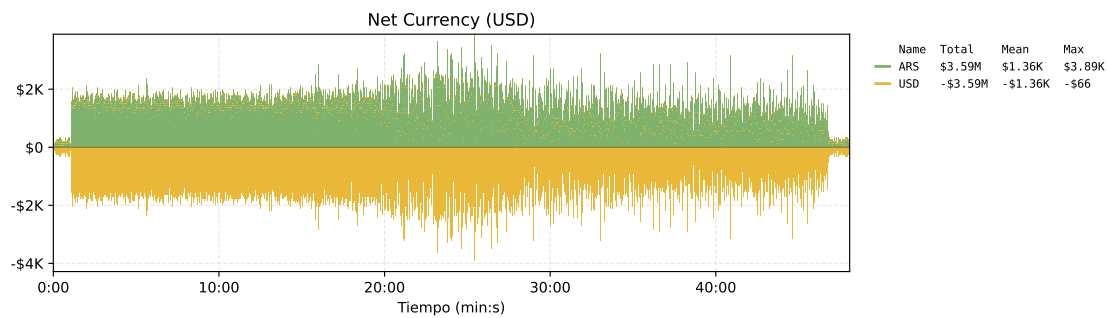


Figura 90. Neto operado en cada moneda en el sistema base.

A.4. Resultados completos del escenario de caos sobre el sistema base

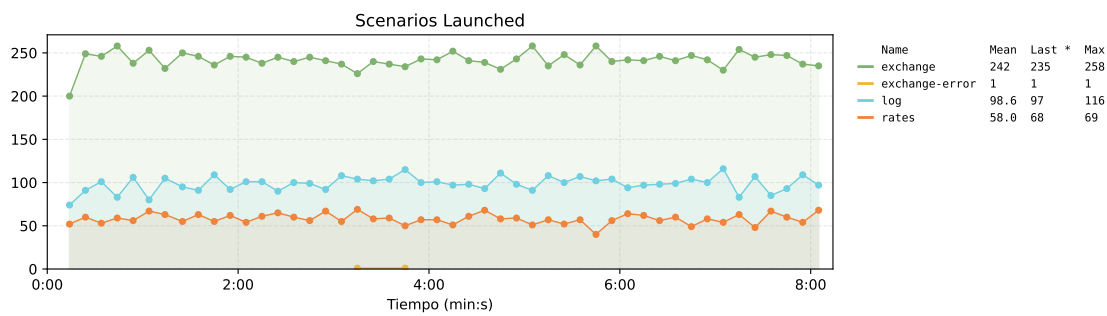


Figura 91. Escenarios del sistema base.

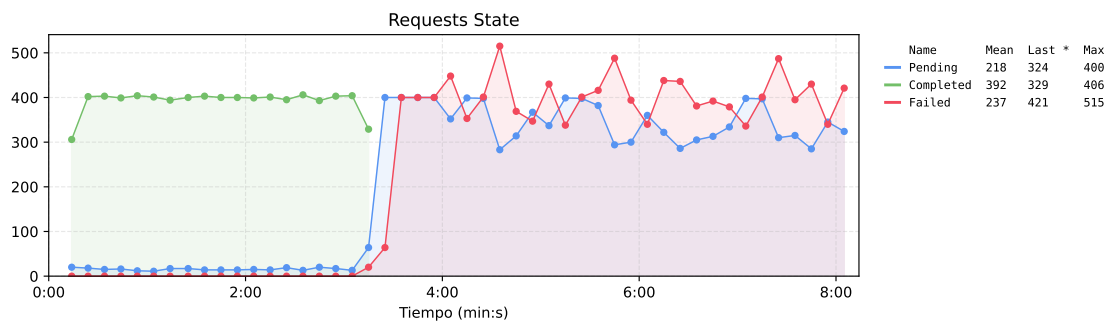


Figura 92. Estado de las requests del sistema base.

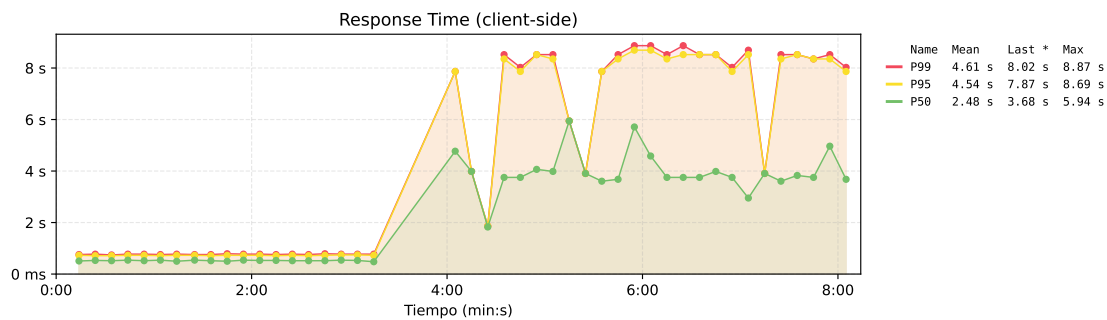


Figura 93. Tiempo de respuesta del sistema base.

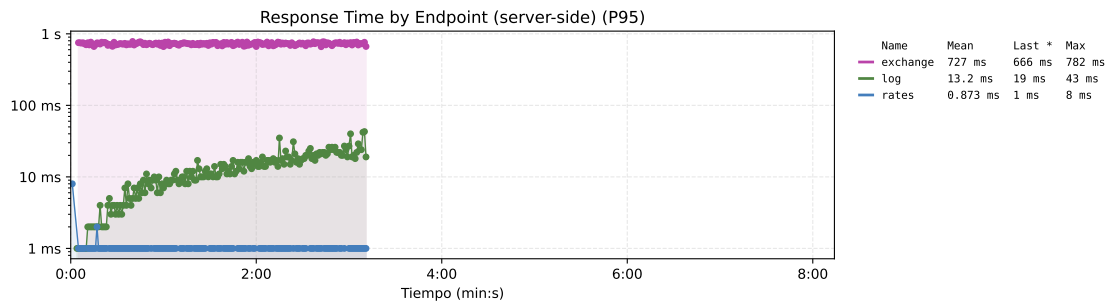


Figura 94. Tiempo de respuesta por endpoint del sistema base. El eje tiene una escala logarítmica.

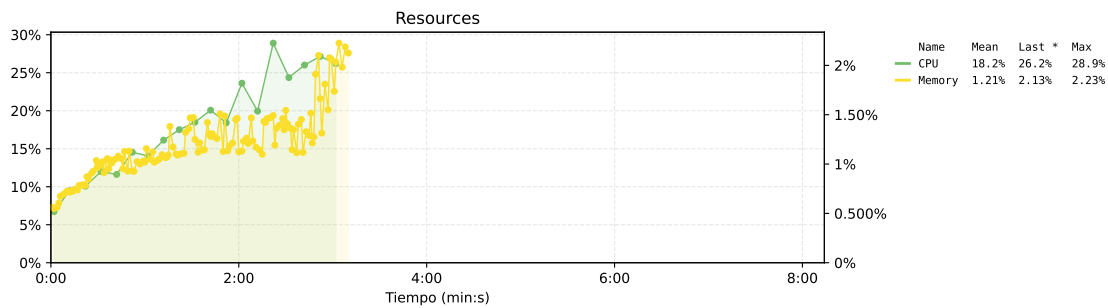


Figura 95. Estado de los recursos del sistema base. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

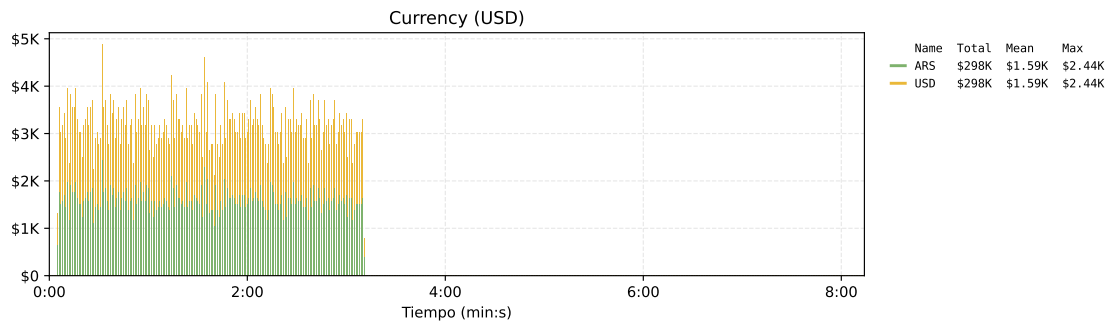


Figura 96. Volumen operado en cada moneda en el sistema base.

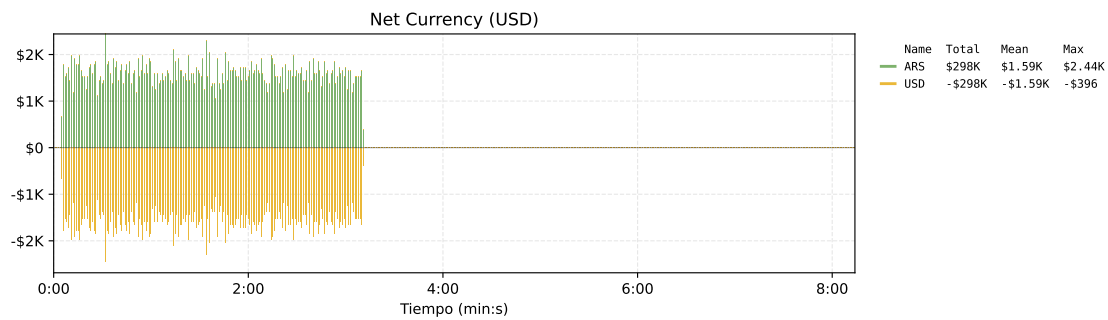


Figura 97. Neto operado en cada moneda en el sistema base.

A.5. Resultados completos del escenario de carga sobre la propuesta 1

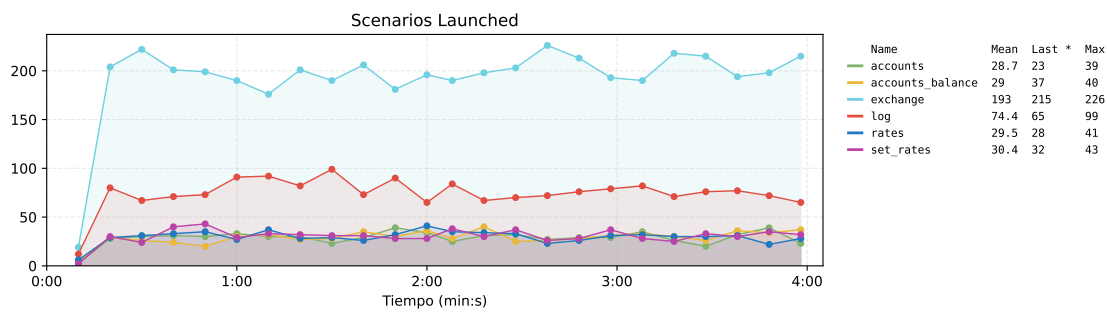


Figura 98. Escenarios de la propuesta 1.

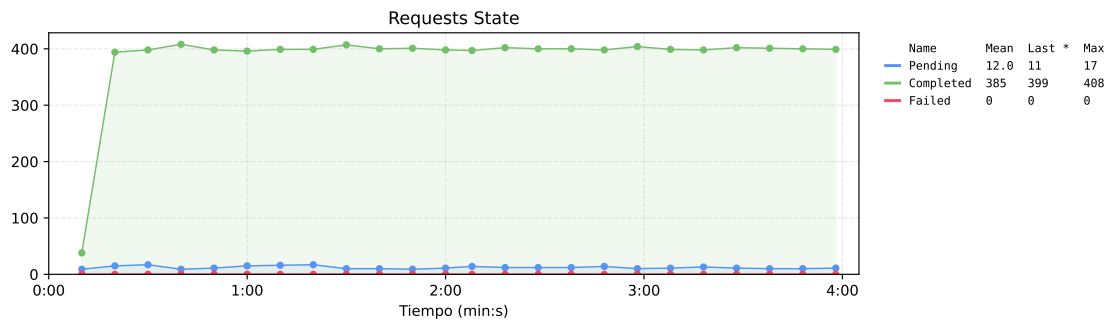


Figura 99. Estado de las requests de la propuesta 1.

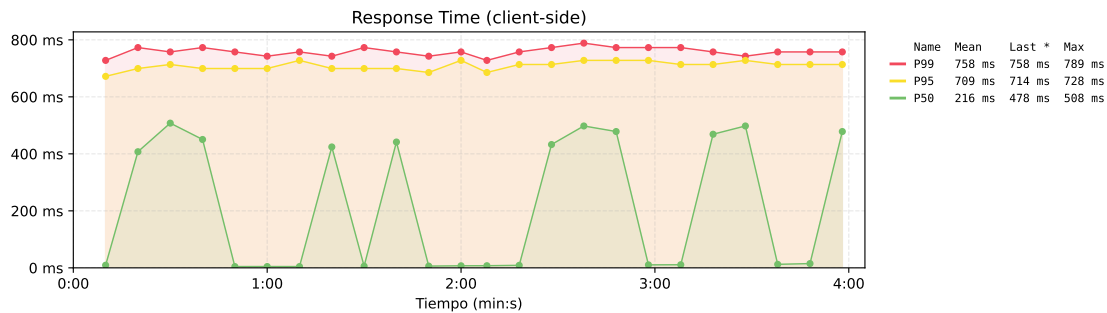


Figura 100. Tiempo de respuesta de la propuesta 1.

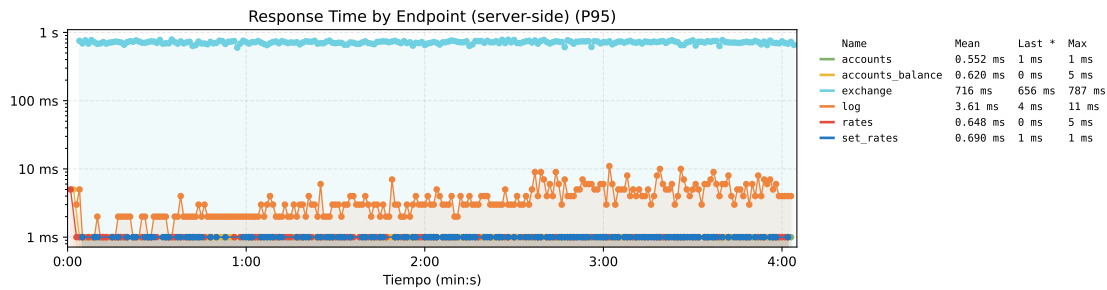


Figura 101. Tiempo de respuesta por endpoint de la propuesta 1. El eje tiene una escala logarítmica.

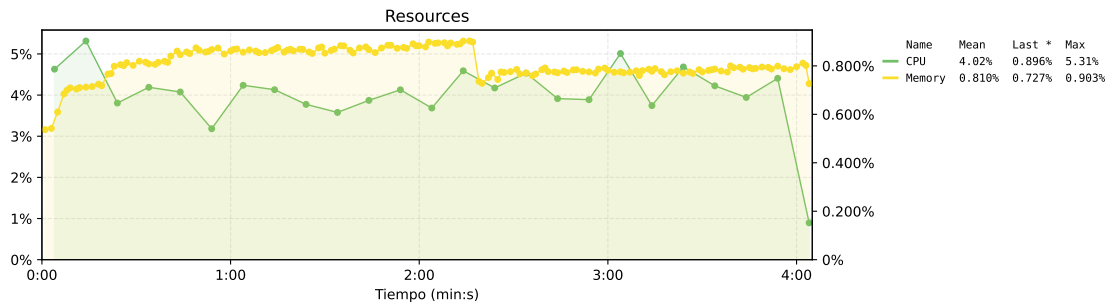


Figura 102. Estado de los recursos del nodo de exchange de la propuesta 1. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

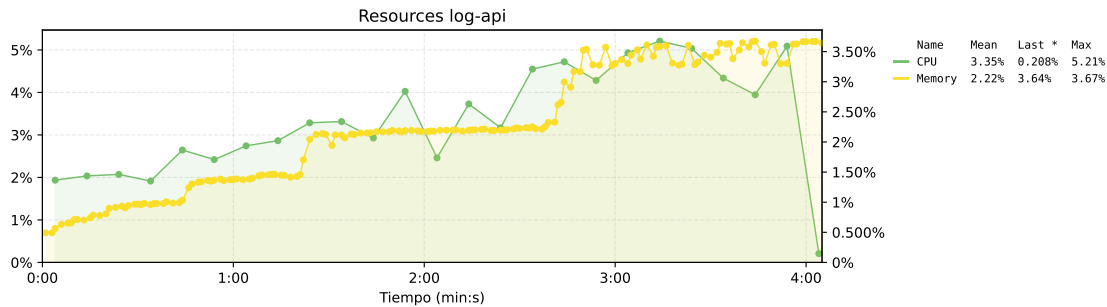


Figura 103. Estado de los recursos del nodo de logs de la propuesta 1. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

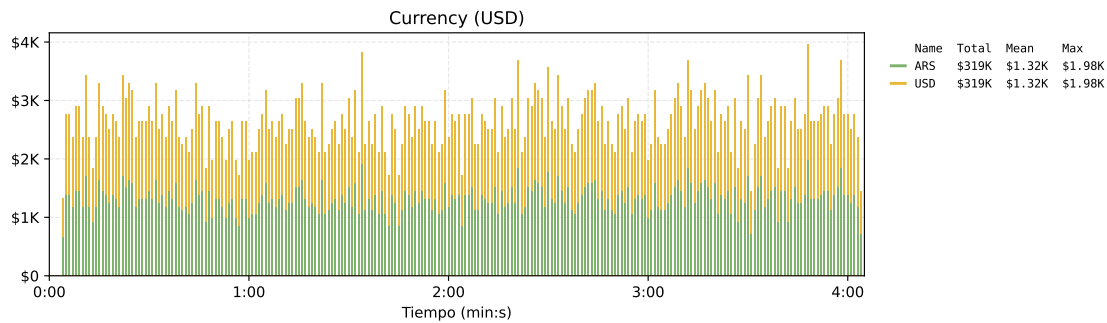


Figura 104. Volumen operado en cada moneda en la propuesta 1.

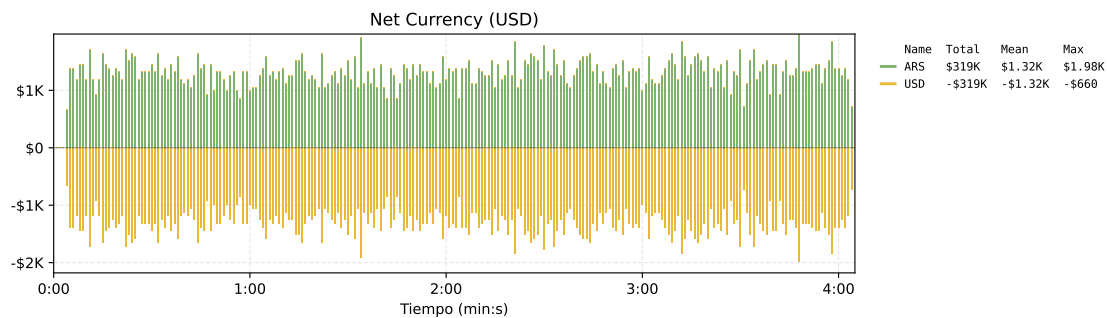


Figura 105. Neto operado en cada moneda en la propuesta 1.

A.6. Resultados completos del escenario de estrés sobre la propuesta 1

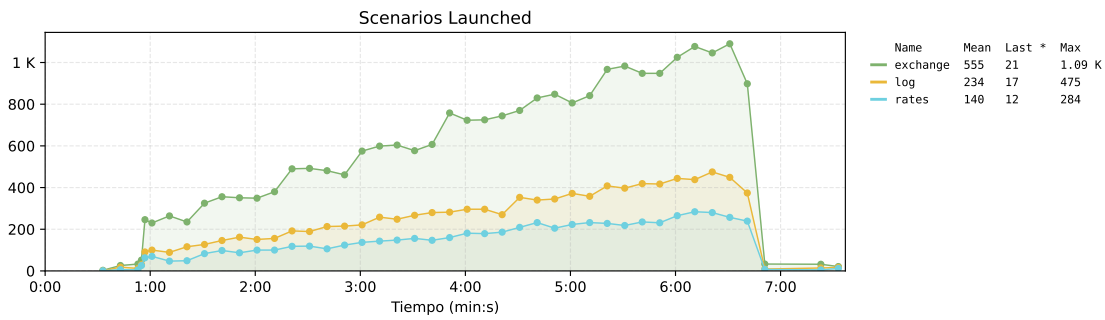


Figura 106. Escenarios de la propuesta 1.

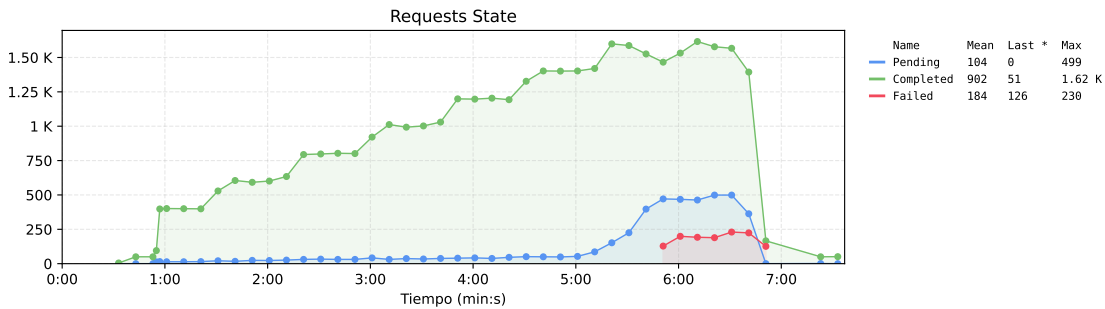


Figura 107. Estado de las requests de la propuesta 1.

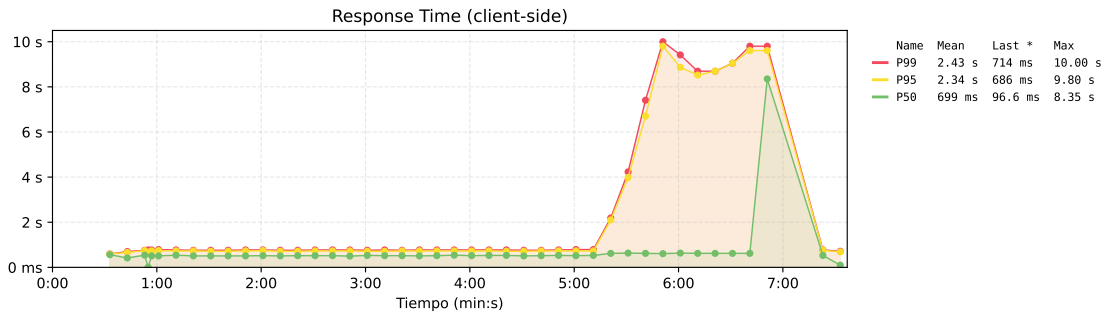


Figura 108. Tiempo de respuesta de la propuesta 1.

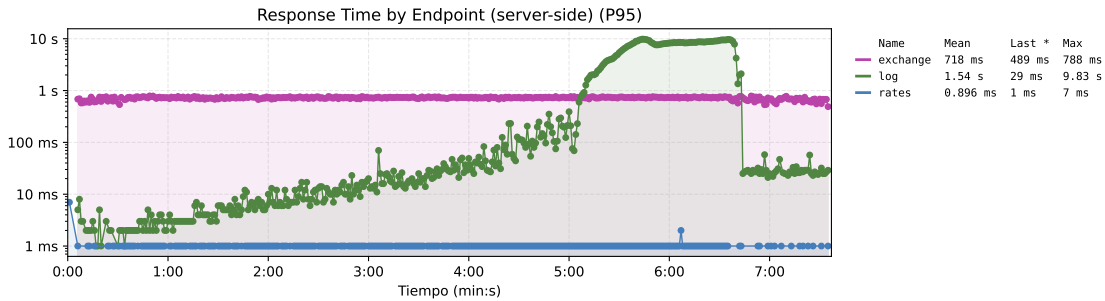


Figura 109. Tiempo de respuesta por endpoint de la propuesta 1. El eje tiene una escala logarítmica.

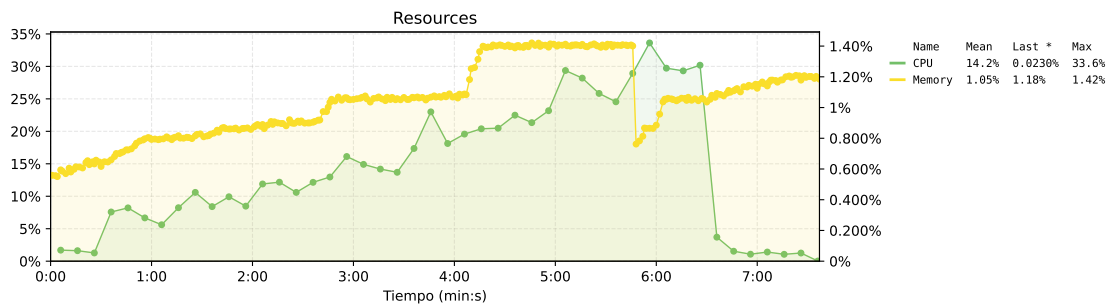


Figura 110. Estado de los recursos del nodo de exchange de la propuesta 1. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

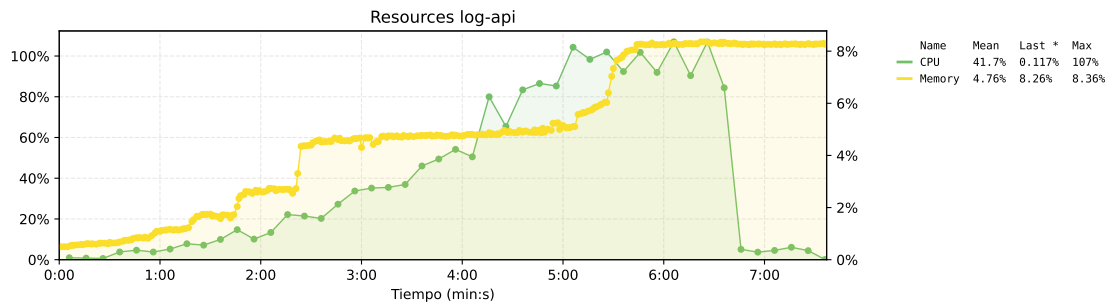


Figura 111. Estado de los recursos del nodo de logs de la propuesta 1. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

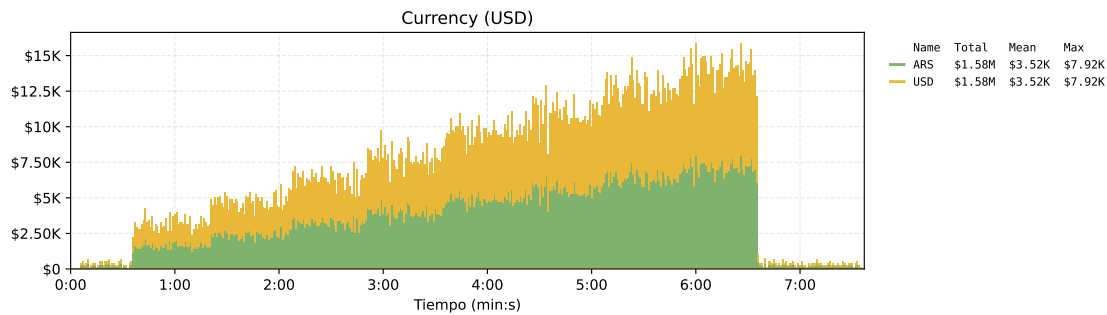


Figura 112. Volumen operado en cada moneda en la propuesta 1.

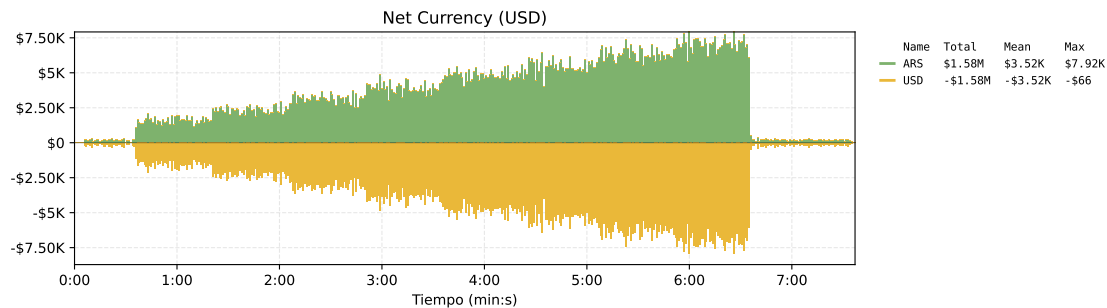


Figura 113. Neto operado en cada moneda en la propuesta 1.

A.7. Resultados completos del escenario de resistencia sobre la propuesta 1

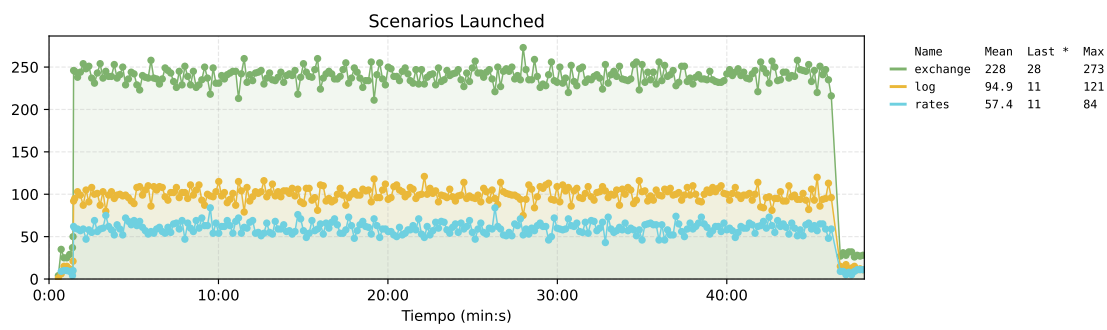


Figura 114. Escenarios de la propuesta 1.

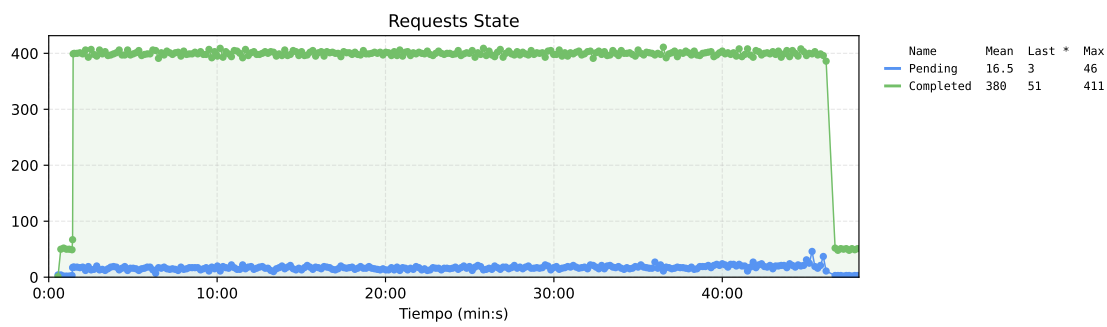


Figura 115. Estado de las requests de la propuesta 1.

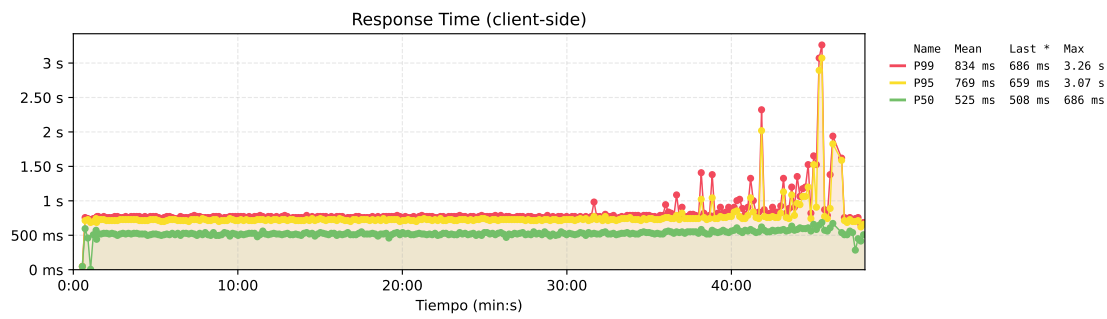


Figura 116. Tiempo de respuesta de la propuesta 1.

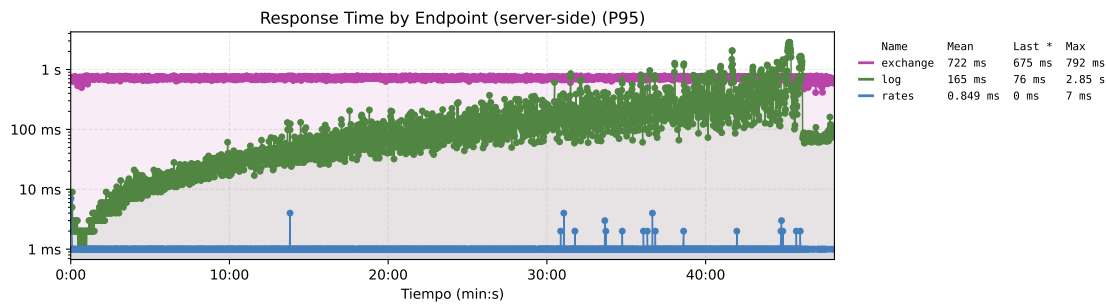


Figura 117. Tiempo de respuesta por endpoint de la propuesta 1. El eje tiene una escala logarítmica.

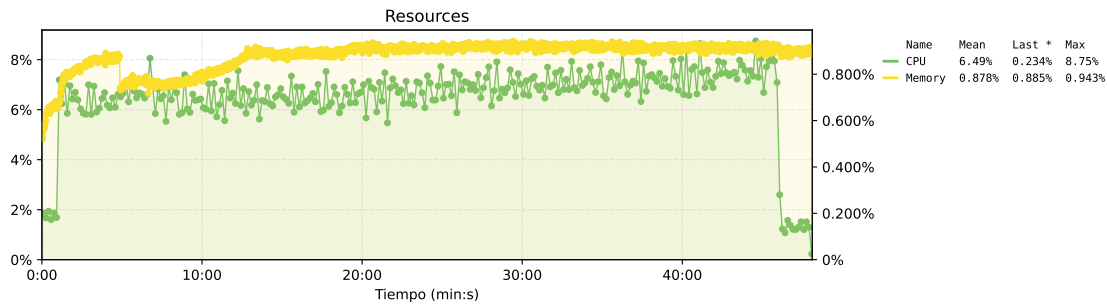


Figura 118. Estado de los recursos del nodo de exchange de la propuesta 1. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

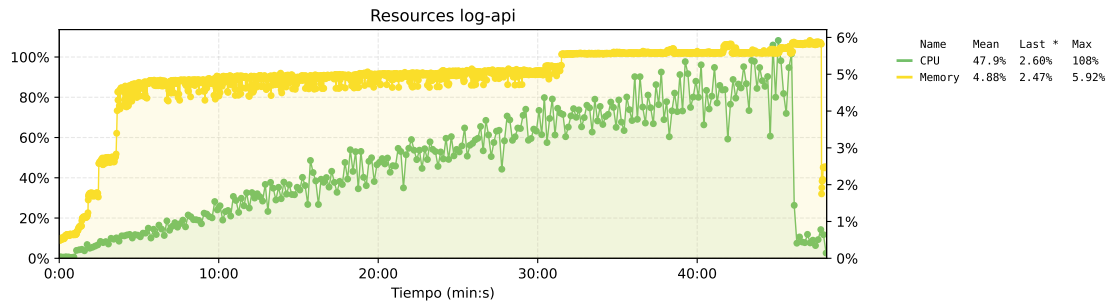


Figura 119. Estado de los recursos del nodo de logs de la propuesta 1. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

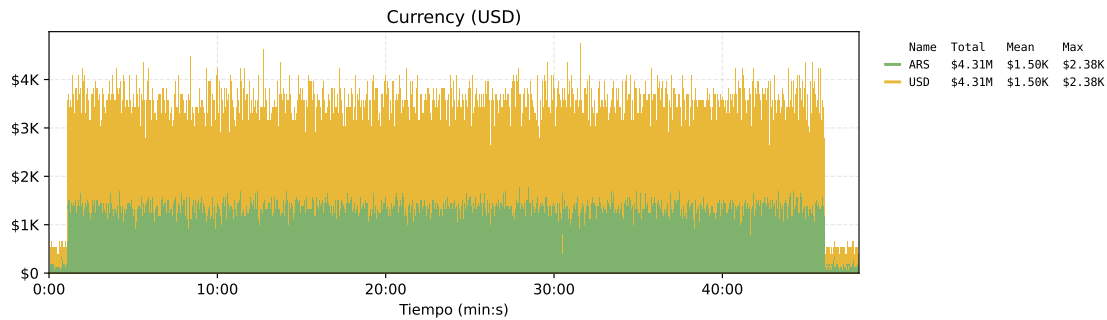


Figura 120. Volumen operado en cada moneda en la propuesta 1.

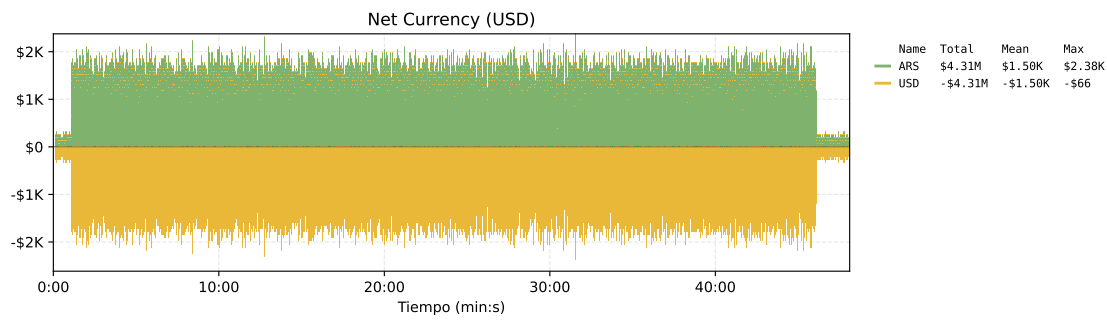


Figura 121. Neto operado en cada moneda en la propuesta 1.

A.8. Resultados completos del escenario de carga sobre la propuesta 2

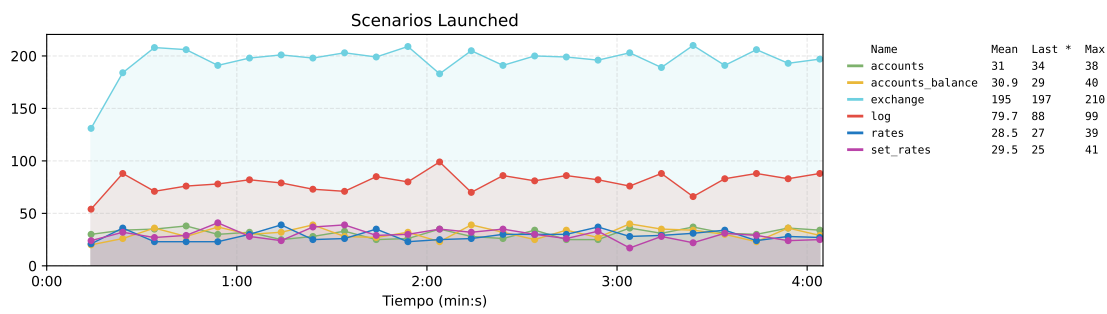


Figura 122. Escenarios de la propuesta 2.

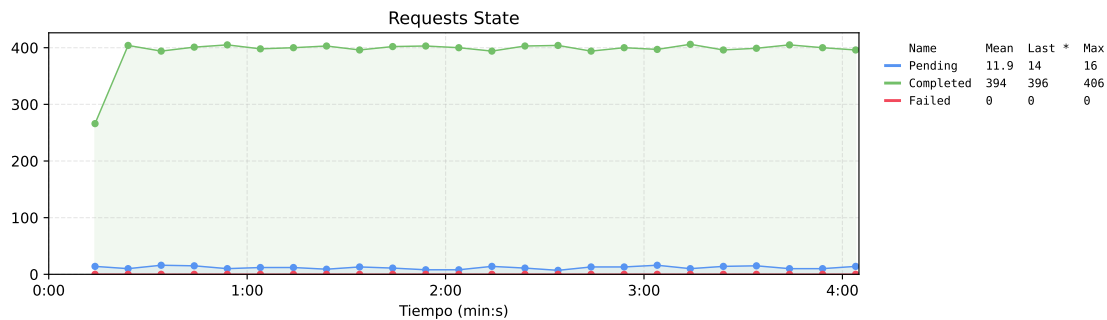


Figura 123. Estado de las requests de la propuesta 2.

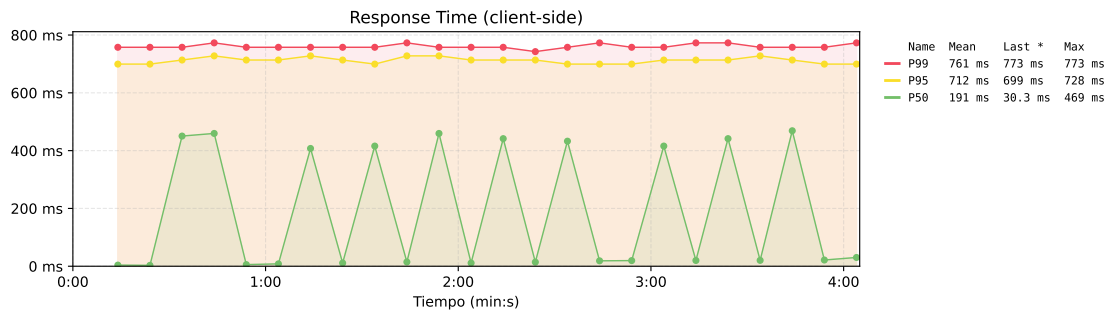


Figura 124. Tiempo de respuesta de la propuesta 2.

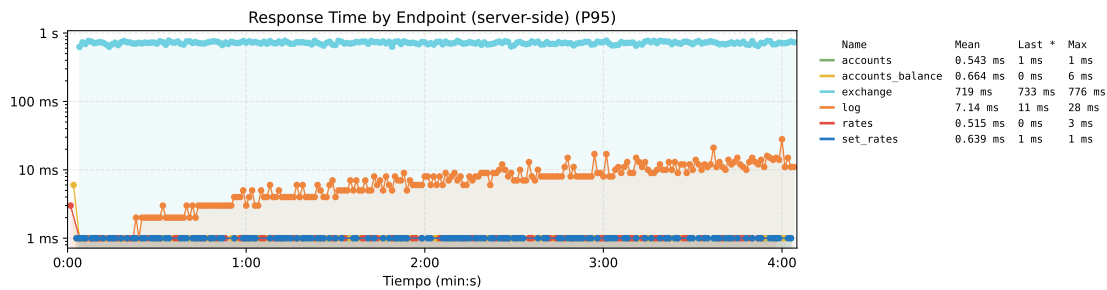


Figura 125. Tiempo de respuesta por endpoint de la propuesta 2. El eje tiene una escala logarítmica.

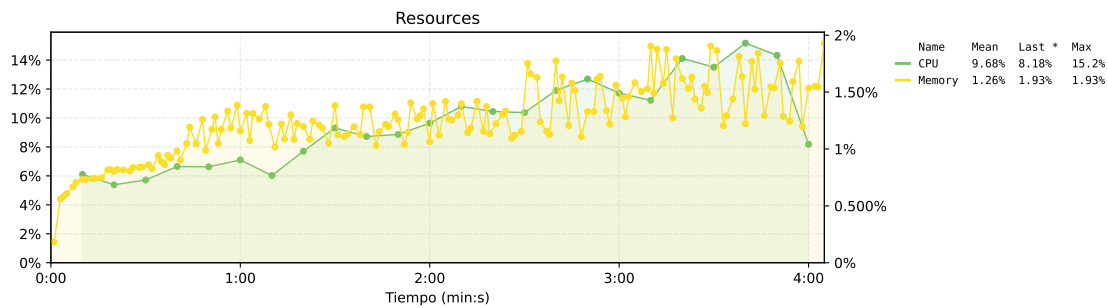


Figura 126. Estado de los recursos de la propuesta 2. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

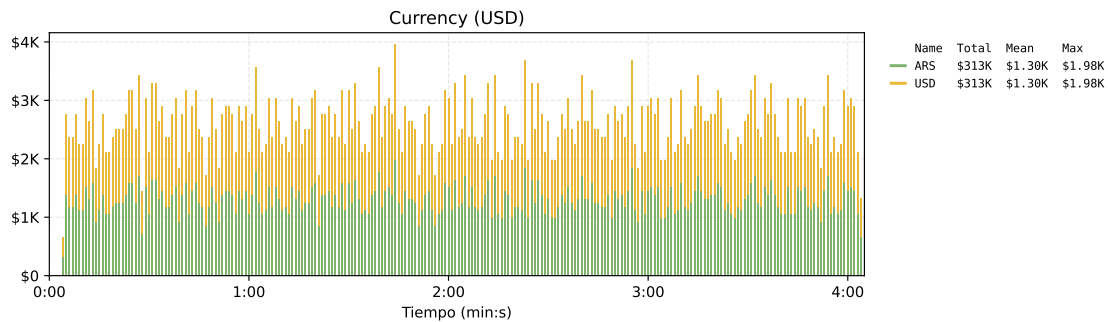


Figura 127. Volumen operado en cada moneda en la propuesta 2.

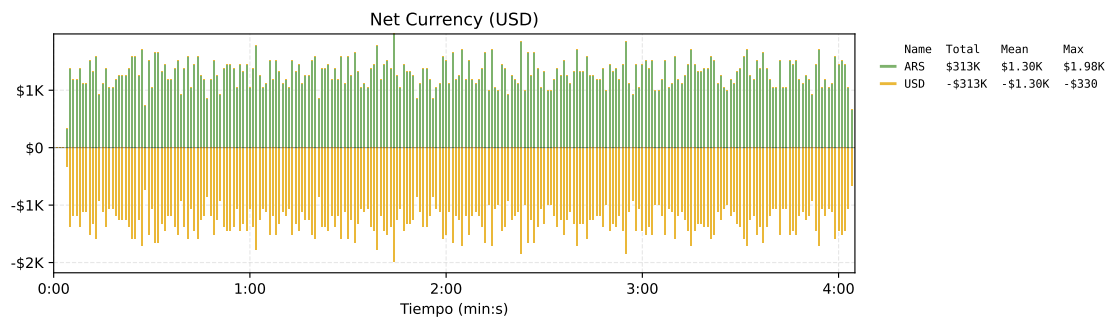


Figura 128. Neto operado en cada moneda en la propuesta 2.

A.9. Resultados completos del escenario de estrés sobre la propuesta 2

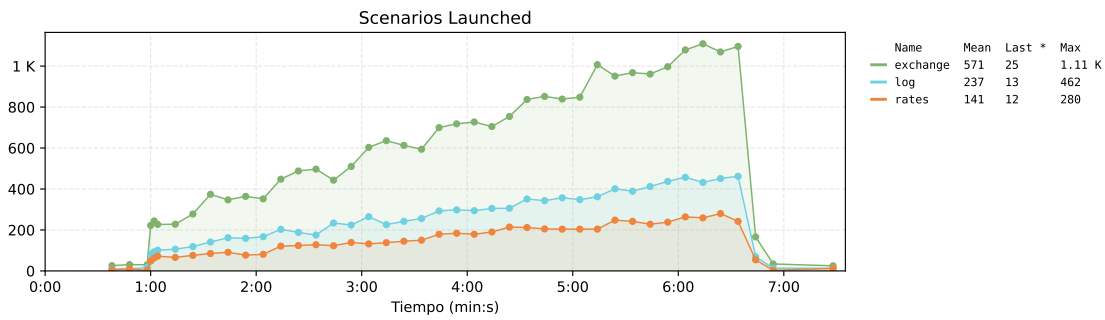


Figura 129. Escenarios de la propuesta 2.

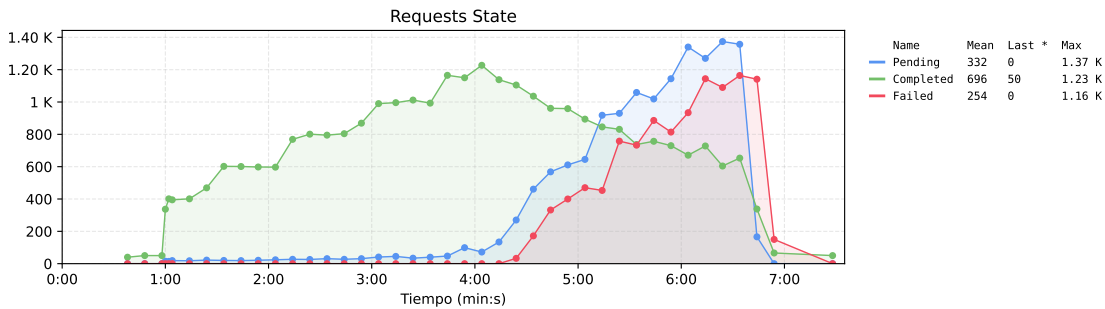


Figura 130. Estado de las requests de la propuesta 2.

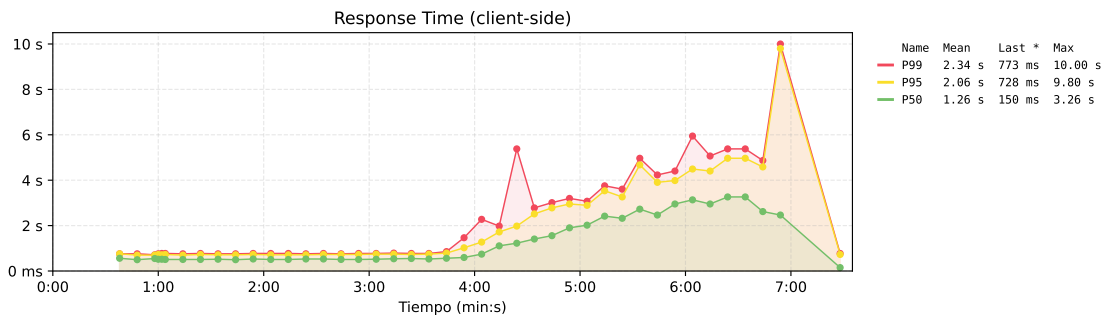


Figura 131. Tiempo de respuesta de la propuesta 2.

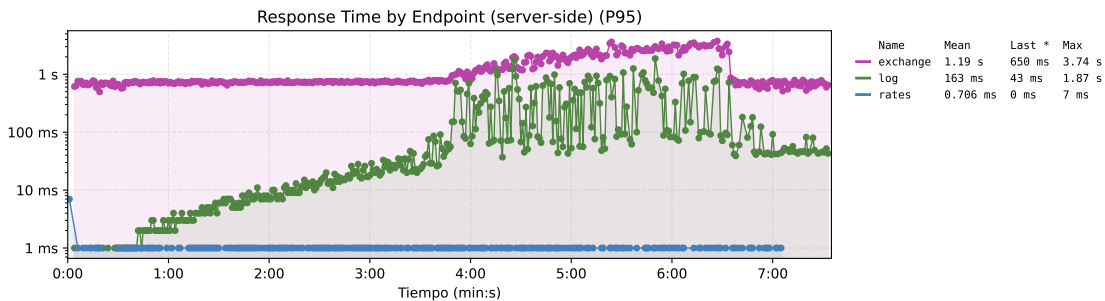


Figura 132. Tiempo de respuesta por endpoint de la propuesta 2. El eje tiene una escala logarítmica.

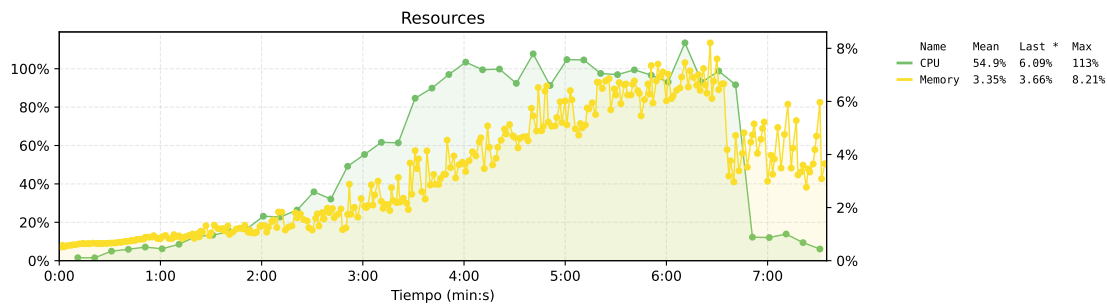


Figura 133. Estado de los recursos de la propuesta 2. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

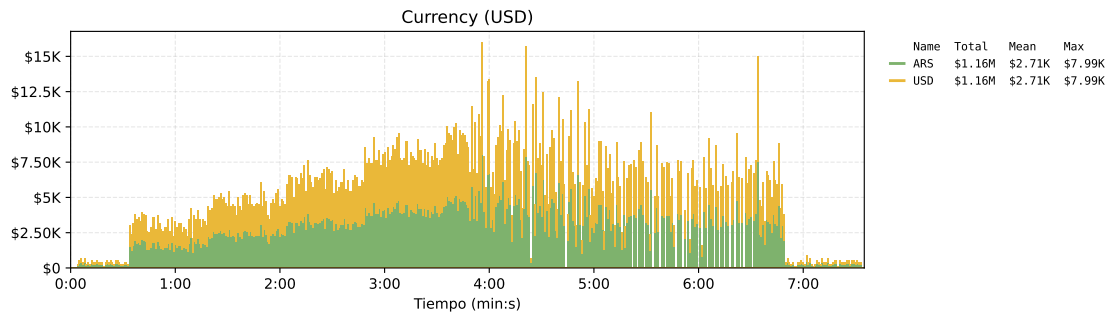


Figura 134. Volumen operado en cada moneda en la propuesta 2.

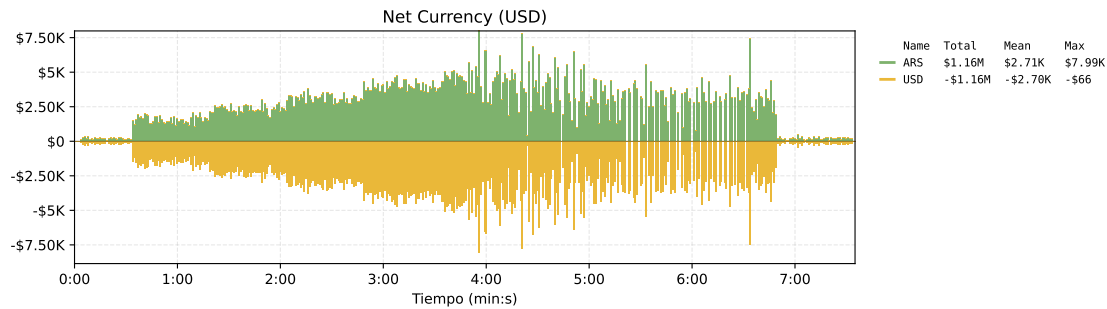


Figura 135. Neto operado en cada moneda en la propuesta 2.

A.10. Resultados completos del escenario de resistencia sobre la propuesta 2

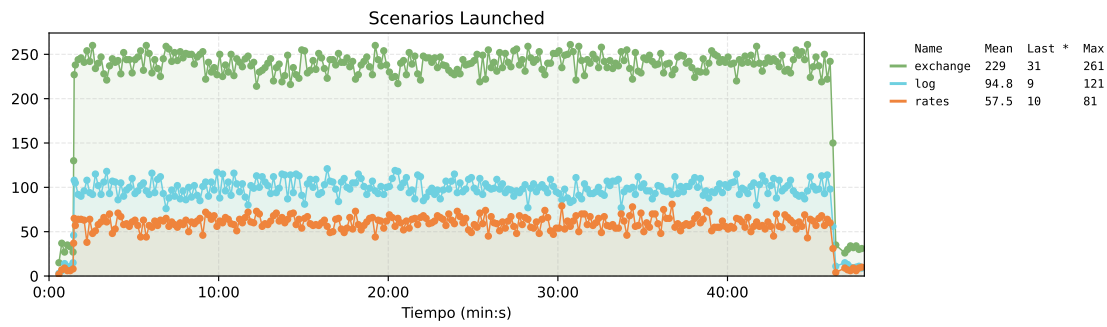


Figura 136. Escenarios de la propuesta 2.

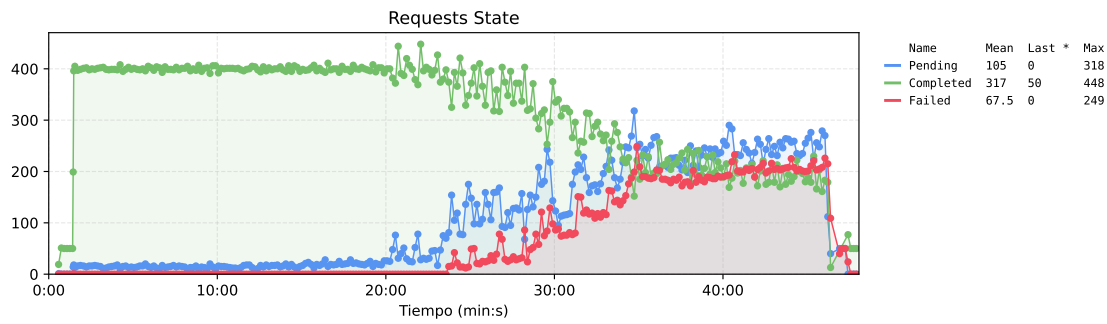


Figura 137. Estado de las requests de la propuesta 2.

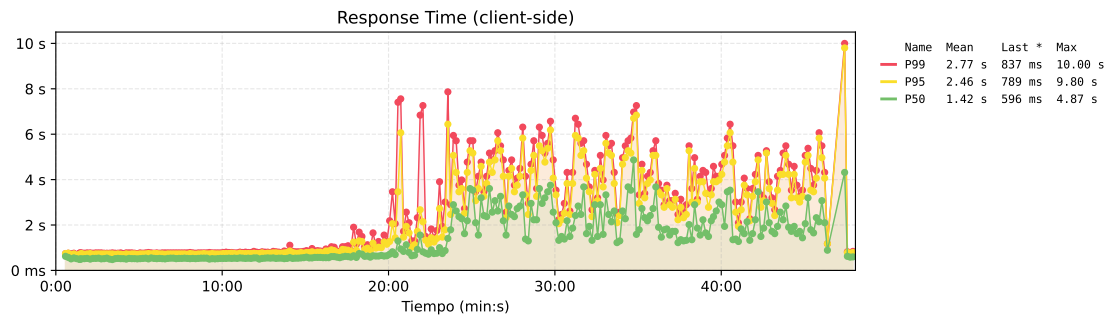


Figura 138. Tiempo de respuesta de la propuesta 2.

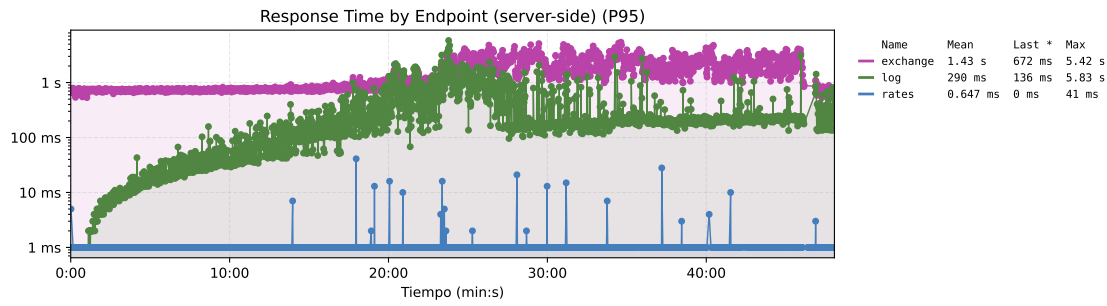


Figura 139. Tiempo de respuesta por endpoint de la propuesta 2. El eje tiene una escala logarítmica.

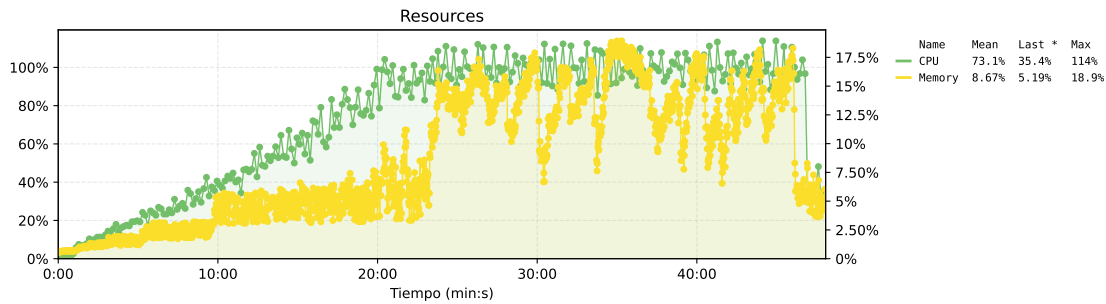


Figura 140. Estado de los recursos de la propuesta 2. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

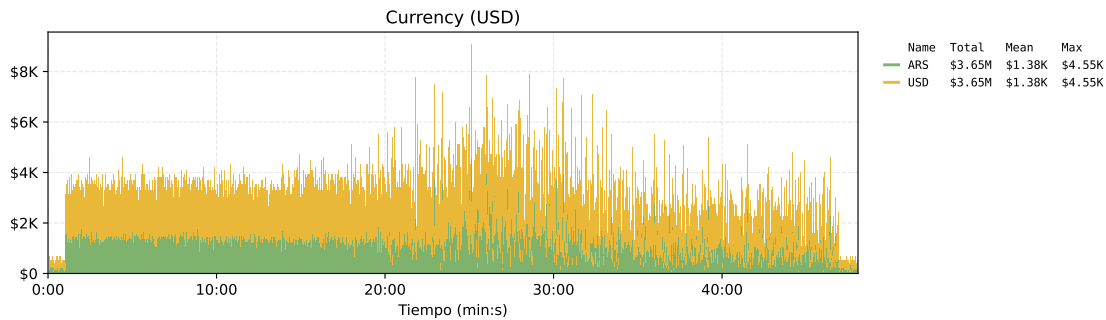


Figura 141. Volumen operado en cada moneda en la propuesta 2.

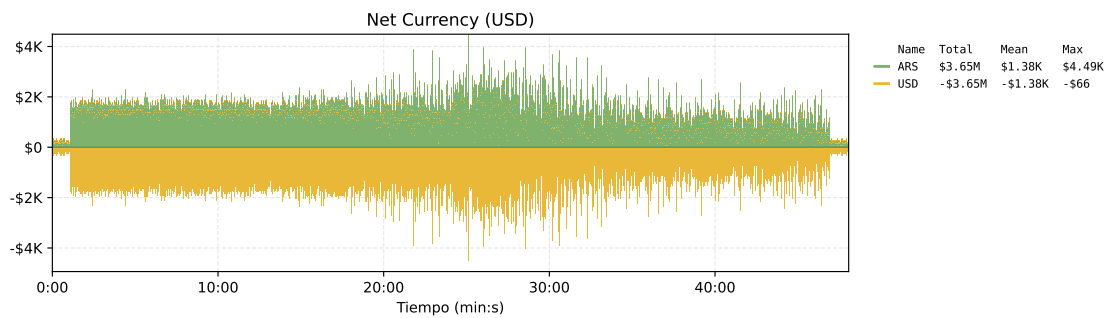


Figura 142. Neto operado en cada moneda en la propuesta 2.

A.11. Resultados completos del escenario de caos sobre la propuesta 2

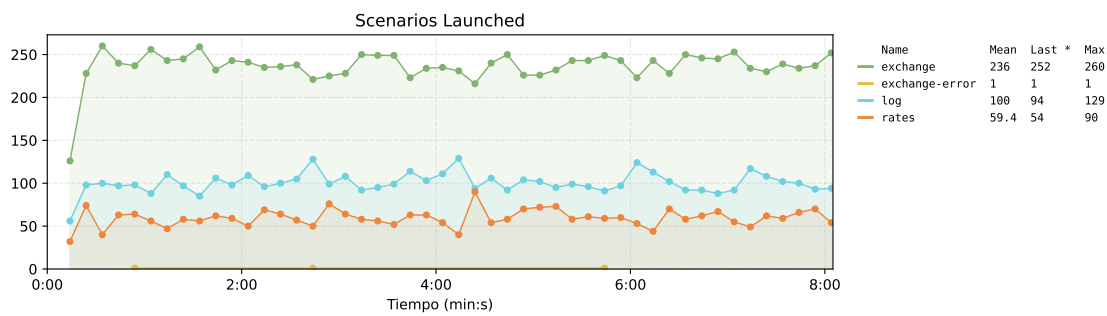


Figura 143. Escenarios de la propuesta 2.

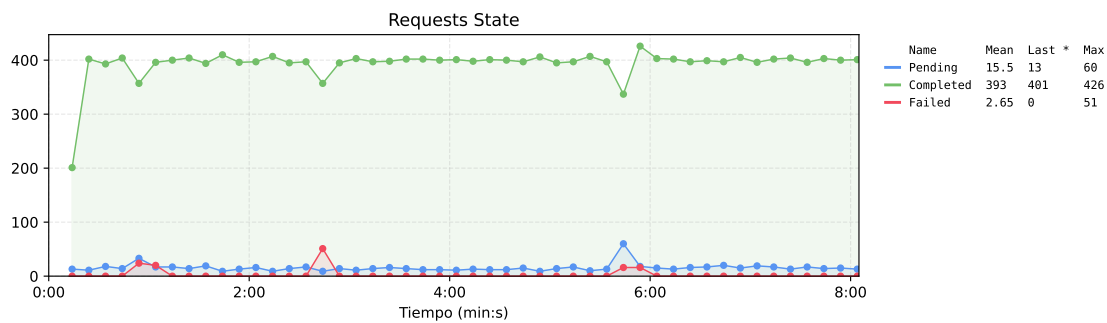


Figura 144. Estado de las requests de la propuesta 2.

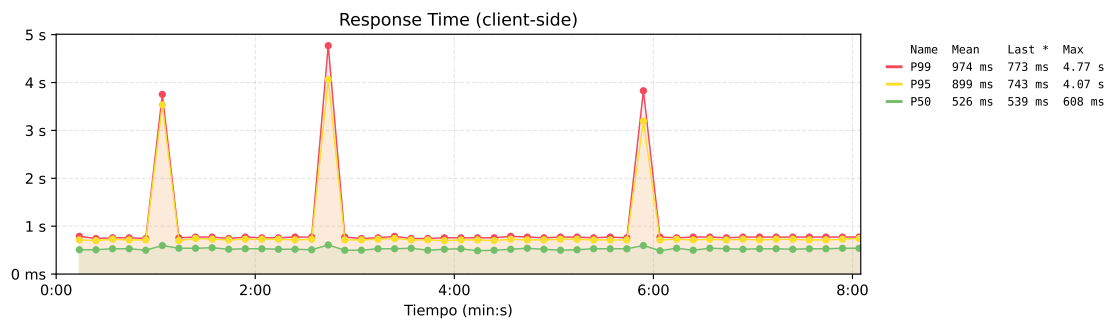


Figura 145. Tiempo de respuesta de la propuesta 2.

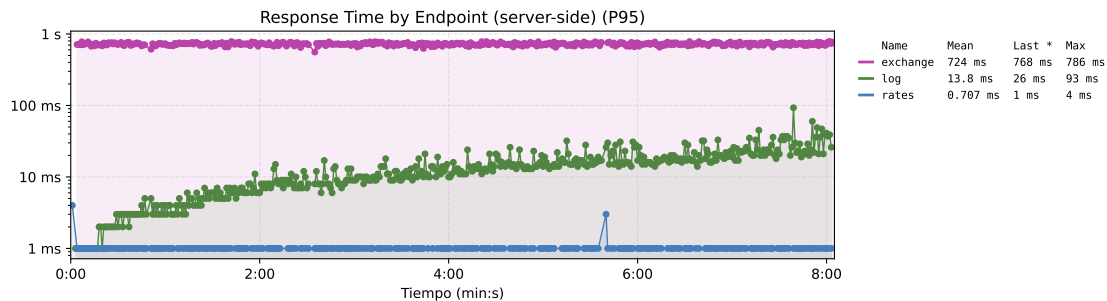


Figura 146. Tiempo de respuesta por endpoint de la propuesta 2. El eje tiene una escala logarítmica.

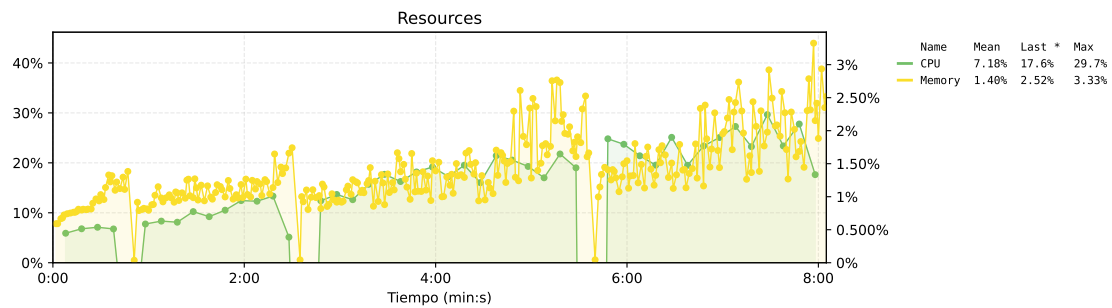


Figura 147. Estado de los recursos de la propuesta 2. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

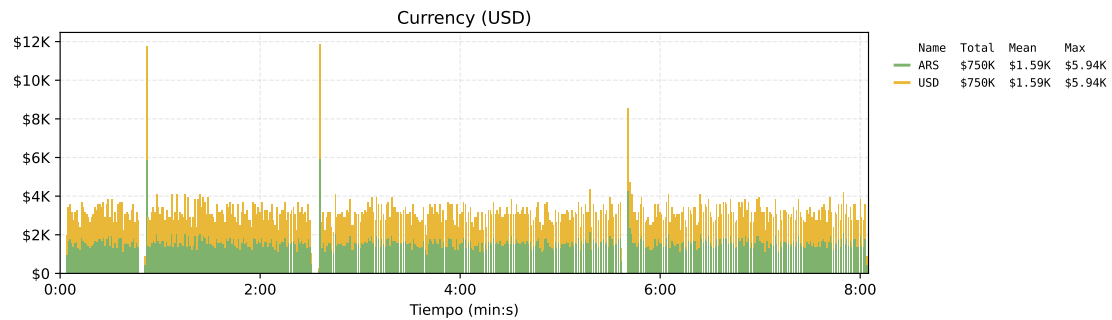


Figura 148. Volumen operado en cada moneda en la propuesta 2.

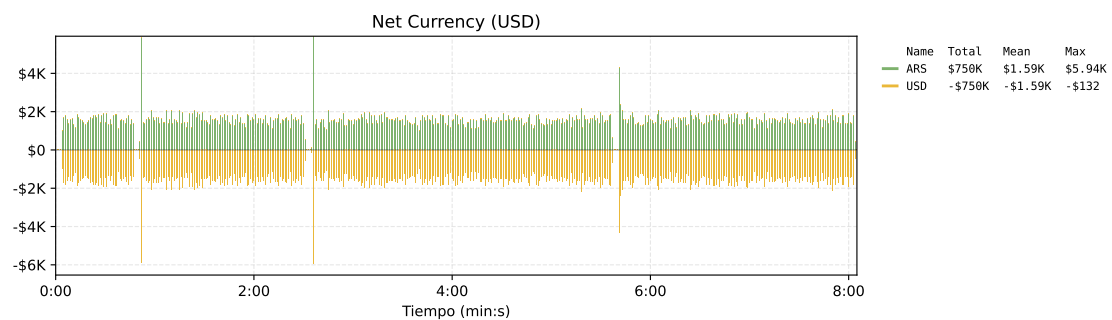


Figura 149. Neto operado en cada moneda en la propuesta 2.

A.12. Resultados completos del escenario de estrés sobre la propuesta 3

TODO: Descomentar las figuras de `figures/propuesta_3/stress/`.

A.13. Resultados completos del escenario de resistencia sobre la propuesta 3

TODO: Descomentar las figuras de `figures/propuesta_3/soak/`.

A.14. Resultados completos del escenario de caos sobre la propuesta 3

TODO: Descomentar las figuras de `figures/propuesta_3/chaos/`.

A.15. Resultados completos del escenario de concurrencia sobre la propuesta 3

TODO: Descomentar las figuras de `figures/propuesta_3/race/`.